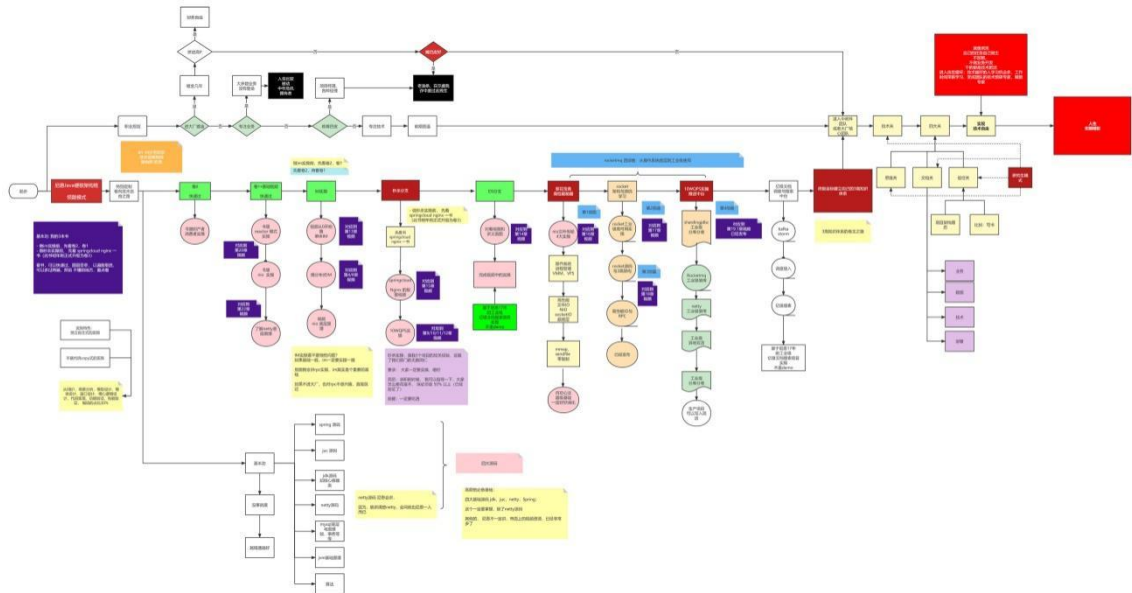


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

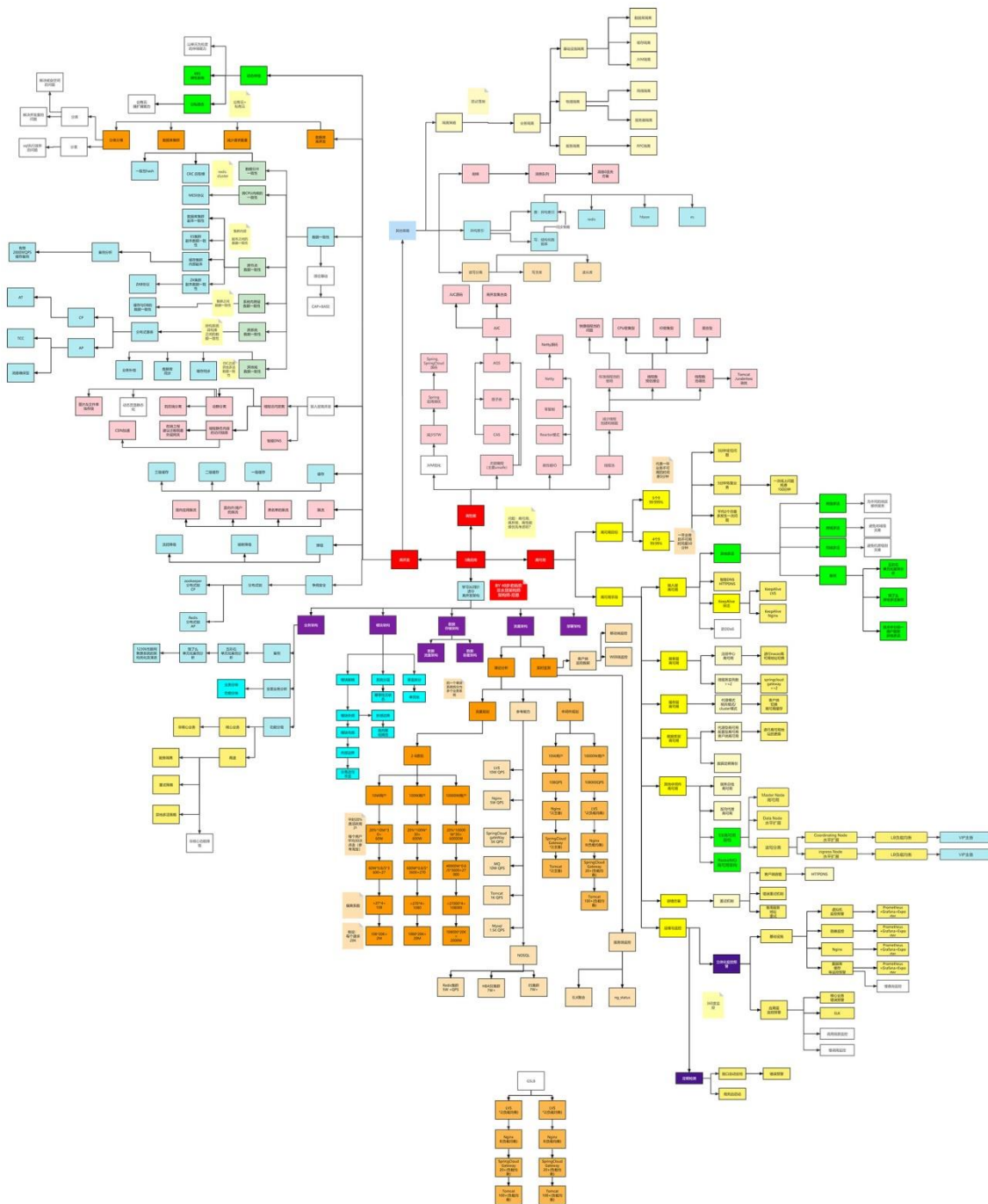
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

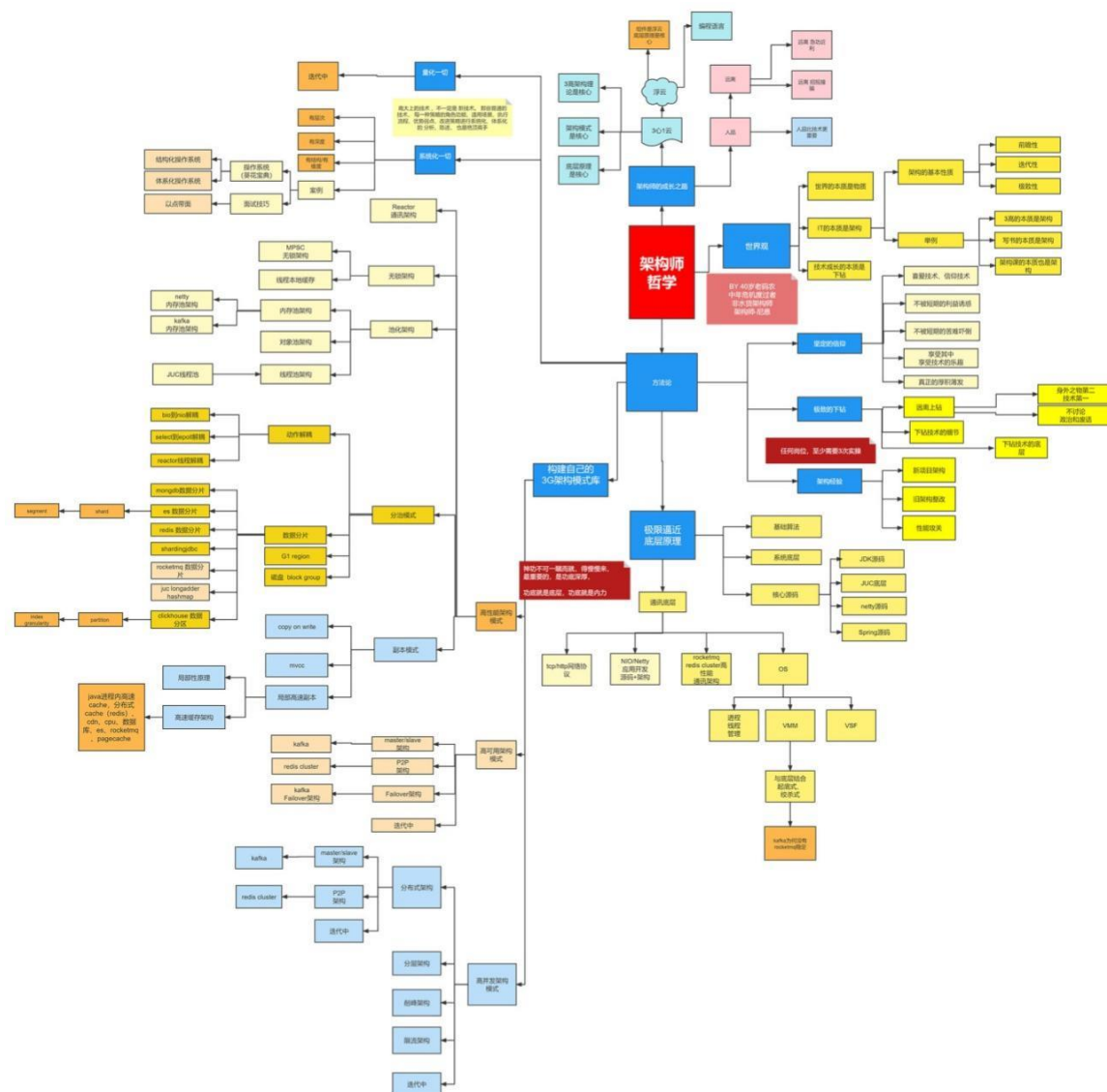
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



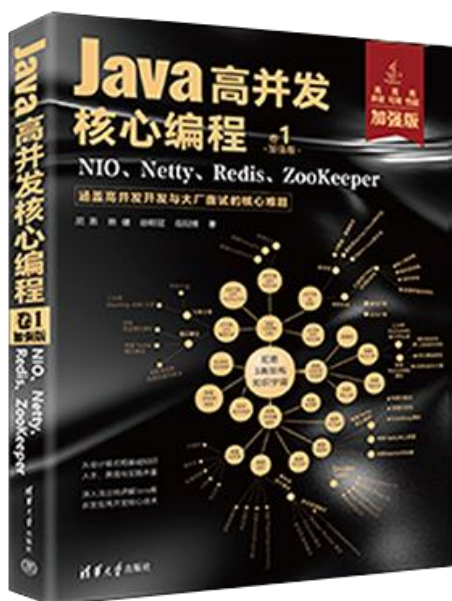
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



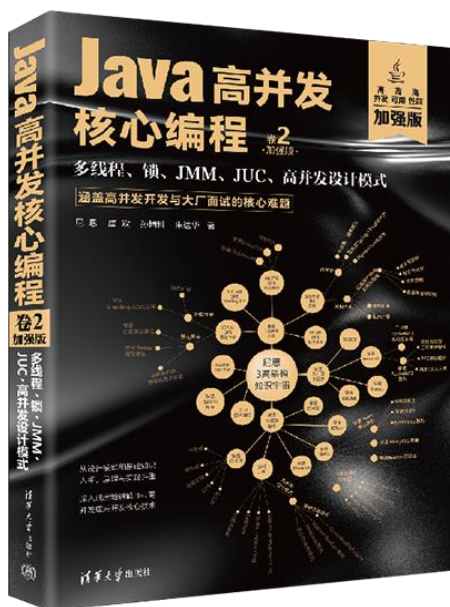
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂的介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



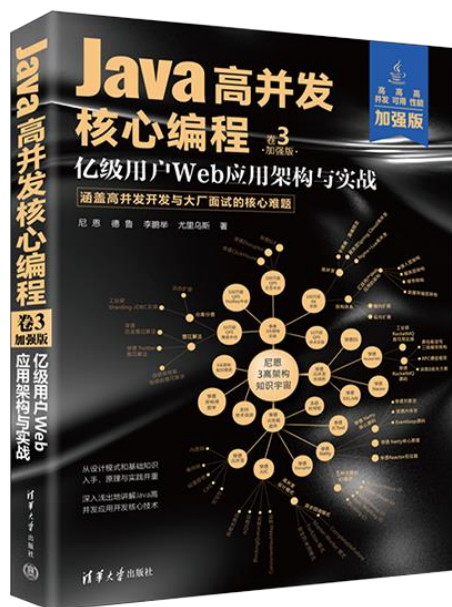
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

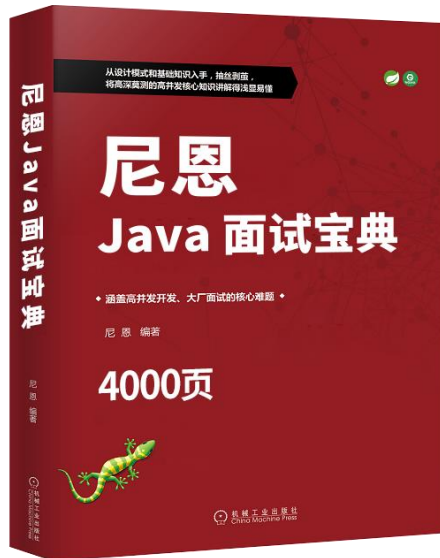
详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

40 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



- 📖 专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V20-from-尼恩Java面试宝典.pdf
- 📖 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf
- 📖 专题03: Java基础面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题04: 架构设计面试题 (卷王专供 + 史上最全 + 2023面试必备) -V24-from-Java面试红宝书-release.pdf
- 📖 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf
- 📖 专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V14-from-Java面试红宝书.pdf
- 📖 专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V22-from-Java面试红宝书-release.pdf
- 📖 专题15: 分布式数据库面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V29-from-尼恩Java面试宝典-release.pdf
- 📖 专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf
- 📖 专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf
- 📖 专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V17-from-Java面试红宝书-release.pdf
- 📖 专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf
- 📖 专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V16-from-Java面试红宝书-release.pdf
- 📖 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf
- 📖 专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf
- 📖 专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf
- 📖 专题36: Dubbo面试题 (卷王专供+ 史上最全 + 2023面试必备) -V21-from-尼恩Java面试宝典-release.pdf
- 📖 专题37: Docker面试题 (卷王专供+ 史上最全 + 2023面试必备) -V26-from-尼恩Java面试宝典-release.pdf
- 📖 专题38: K8S面试题 (卷王专供+ 史上最全 + 2023面试必备) -V26-from-尼恩Java面试宝典-release.pdf
- 📖 专题39: Nginx面试题 (卷王专供+ 史上最全 + 2023面试必备) -V27-from-尼恩Java面试宝典-release.pdf
- 📖 专题40: 操作系统面试题 (卷王专供+ 史上最全 + 2023面试必备) -V28-from-尼恩Java面试宝典-release.pdf

Sentinel 相关面试真题

尼恩说在前面

在40岁老架构师 尼恩的[读者交流群](#)(50+)中，最近有小伙伴拿到了一线互联网企业如阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试资格，遇到很多很重要的面试题：

问题1：Sentinel高可用熔断降级，是如何实现的？

问题2：Sentinel底层滑动时间窗限流算法怎么实现的？

最近又有小伙伴在面试阿里，遇到了相关的面试题。

小伙伴说，Sentinel是自己的盲区，可以说一脸懵逼，面试官不满意，面试挂了，非常可惜，如果早点看看这尼恩的面试宝典，年薪60W+的offer就到手了。

亡羊补牢、为时不晚。

在这里，尼恩给大家做一下系统化、体系化的Sentinel 梳理，使得大家内力猛增，展示一下雄厚的“技术肌肉、技术实力”，**让面试官爱到“不能自己、口水直流”**，然后实现“offer直提，offer自由”。

当然，这道面试题，以及参考答案，也会收入咱们的《[尼恩Java面试宝典PDF](#)》V140版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF，请到文末公号【技术自由圈】获取

阿里面试：Sentinel熔断降级，是如何实现的？

在微服务架构中，Sentinel 作为一种流量控制、熔断降级和服务降级的解决方案，得到了广泛的应用。

Sentinel是一个开源的流量控制和熔断降级库，用于保护分布式系统免受大量请求的影响。

尼恩建议大家，从以下的几个维度去作答：

第一个维度，Sentinel主要功能：

一：熔断机制

1. Sentinel使用滑动窗口统计请求的成功和失败情况。这些统计信息包括成功的请求数、失败的请求数等。
2. 当某个资源（例如一个API接口）的错误率超过阈值或其他指标达到预设的条件，Sentinel将触发熔断机制。
3. 一旦熔断触发，Sentinel将暂时阻止对该资源的请求，防止继续失败的请求对系统造成更大的影响。

二：降级机制

1. Sentinel还提供了降级机制，可以在资源负载过重或其他异常情况下，限制资源的访问速率，以保护系统免受过多的请求冲击。
2. 降级策略可以根据需要定制，可以是慢调用降级、异常比例降级等。

三：高可用性机制

Sentinel的高可用性主要通过以下方式来实现：

- a. 多节点部署：将Sentinel配置为多节点部署，确保即使一个节点发生故障，其他节点仍然能够继续工作。
- b. 持久化配置：Sentinel支持将配置信息持久化到外部存储，如Nacos、Redis等。这样，即使Sentinel节点重启，它可以加载之前的配置信息。
- c. 集群流控规则：Sentinel支持集群流控规则，多个节点可以共享流量控制规则，以协同工作来保护系统。
- d. 实时监控：Sentinel提供了实时监控和仪表盘，可以查看系统的流量控制和熔断降级情况，帮助及时发现问题并采取措施。

四：自适应控制

Sentinel具有自适应控制的功能，它可以根据系统的实际情况自动调整流量控制和熔断降级策略，以适应不同的负载和流量模式。

总的来说，Sentinel的高可用性熔断降级机制是通过多节点部署、持久化配置、实时监控、自适应控制等多种手段来实现的。

这使得Sentinel能够在分布式系统中保护关键资源免受异常流量的影响，并保持系统的稳定性和可用性。

那么，Sentinel是如何实现这些功能的呢？在说说 Sentinel 的基本组件。

第二个维度， Sentinel 的基本组件：

Sentinel 主要包括以下几个部分：资源（Resource）、规则（Rule）、上下文（Context）和插槽（Slot）。

- 资源是我们想要保护的對象，比如一个远程服务、一个数据库连接等。
- 规则是定义如何保护资源的，比如我们可以通过设置阈值、时间窗口等方式来决定何时进行限流、熔断等操作。
- 上下文是一个临时的存储空间，用于存储资源的状态信息，比如当前的 QPS 等。
- 插槽属于责任链模式中的处理器/过滤器，完成资源规则的计算和验证。

第三个维度， Sentinel 的流量治理几个核心步骤：

在 Sentinel 的运行过程中，主要分为以下几个核心步骤：

1. 资源注册：当一个资源被创建时，需要将其注册到 Sentinel。在注册过程中，会为资源创建一个对应的上下文，并将资源的规则存储到插槽中。
2. 流量控制：当有请求访问资源时，Sentinel 会根据资源的规则进行流量控制。如果当前 QPS 超过了规则设定的阈值，Sentinel 就会拒绝请求，以防止系统过载。

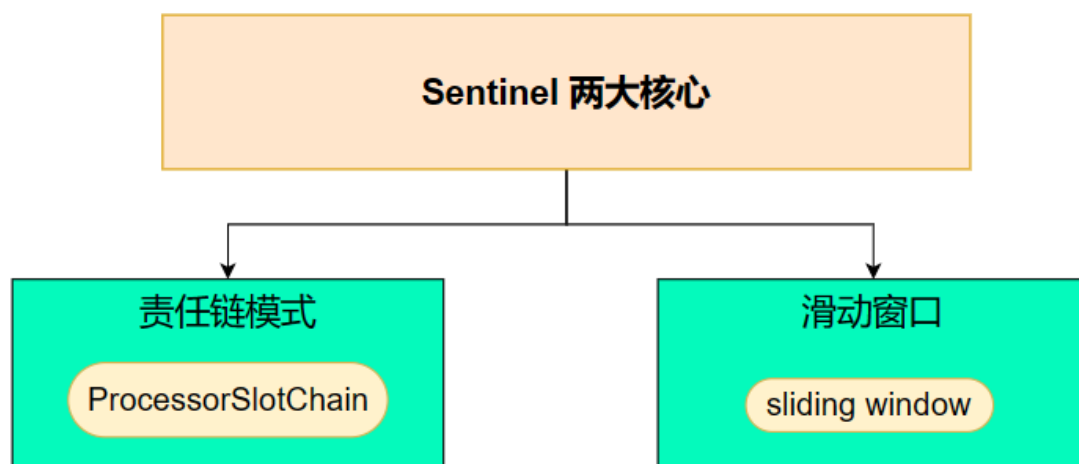
- 熔断降级：当资源出现异常时，Sentinel 会根据规则进行熔断或降级处理。熔断是指暂时切断对资源的访问，以防止异常扩散。降级则是提供一种备用策略，当主策略无法正常工作时，可以切换到备用策略。
- 规则更新：在某些情况下，我们可能需要动态调整资源的规则。Sentinel 提供了 API 接口，可以方便地更新资源的规则。

通过以上分析，我们可以看出，Sentinel 的核心思想是通过规则来管理和控制资源。这种设计使得 Sentinel 具有很强的可扩展性和灵活性。我们可以根据业务需求，定制各种复杂的规则。

第四个维度，Sentinel 的源码层面的两个核心架构：

回到源码层面，在 Sentinel 源码，包括以下二大架构：

- 责任链模式架构
- 滑动窗口数据统计架构



尼恩说明：两大架构的源码，简单说说就可以了，具体可以参见《Sentinel 学习圣经》最新版本。

总指挥，Sentinel 是一种非常强大的流量控制、熔断降级和服务降级的解决方案。已经成为了替代 Hystrix 的主要高可用组件。

说在最后：“offer自由”很容易的

Java Agent、Instrumentation、arthas 相关的面试题，是非常常见的面试题。

以上的内容，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

在面试之前，建议大家系统化的刷一波 5000 页《[尼恩Java面试宝典PDF](#)》，里边有大量的大厂真题、面试难题、架构难题。很多小伙伴刷完后，吊打面试官，大厂横着走。

在刷题过程中，如果有啥问题，大家可以来找 40 岁老架构师尼恩交流。

另外，如果没有面试机会，可以找尼恩来改简历、做帮扶。

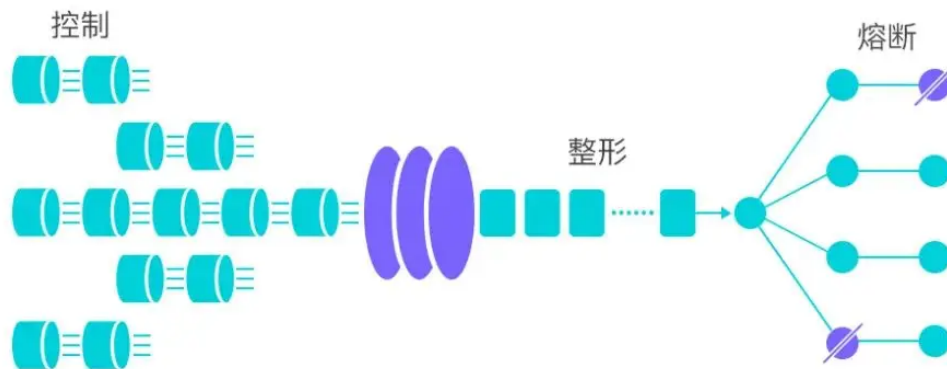
尼恩指导了大量的小伙伴上岸，前段时间，刚指导一个 40 岁+被裁小伙伴，拿到了一个年薪 100W 的 offer。

狠狠卷，实现“offer自由”很容易的，前段时间一个武汉的跟着尼恩卷了 2 年的小伙伴，在极度严寒/痛苦被裁的环境下，offer 拿到手软，实现真正的“offer自由”

美团面试：Sentinel底层滑动时间窗限流算法怎么实现的？

Sentinel是一个系统性的高可用保障工具，提供了限流、降级、熔断等一系列的能力，基于这些能力做了语义化概念抽象，这些概念对于理解实现机制特别有帮助，所以这里也复述一下。

对于流量控制，有个一个模型：



流量控制有以下几个角度：

- 资源的调用关系，例如资源的调用链路，资源和资源之间的关系；
- 运行指标，例如 QPS、线程池、系统负载等；
- 控制的效果，例如直接限流、冷启动、排队等。

Sentinel 的设计理念是让您自由选择控制的角度，并进行灵活组合，从而达到想要的效果。

Sentinel使用滑动时间窗口算法来实现流量控制，流量统计。

滑动时间窗口算法的核心思想是将一段时间划分为多个时间窗口，并在每个时间窗口内对请求进行计数，以确定是否允许继续请求。

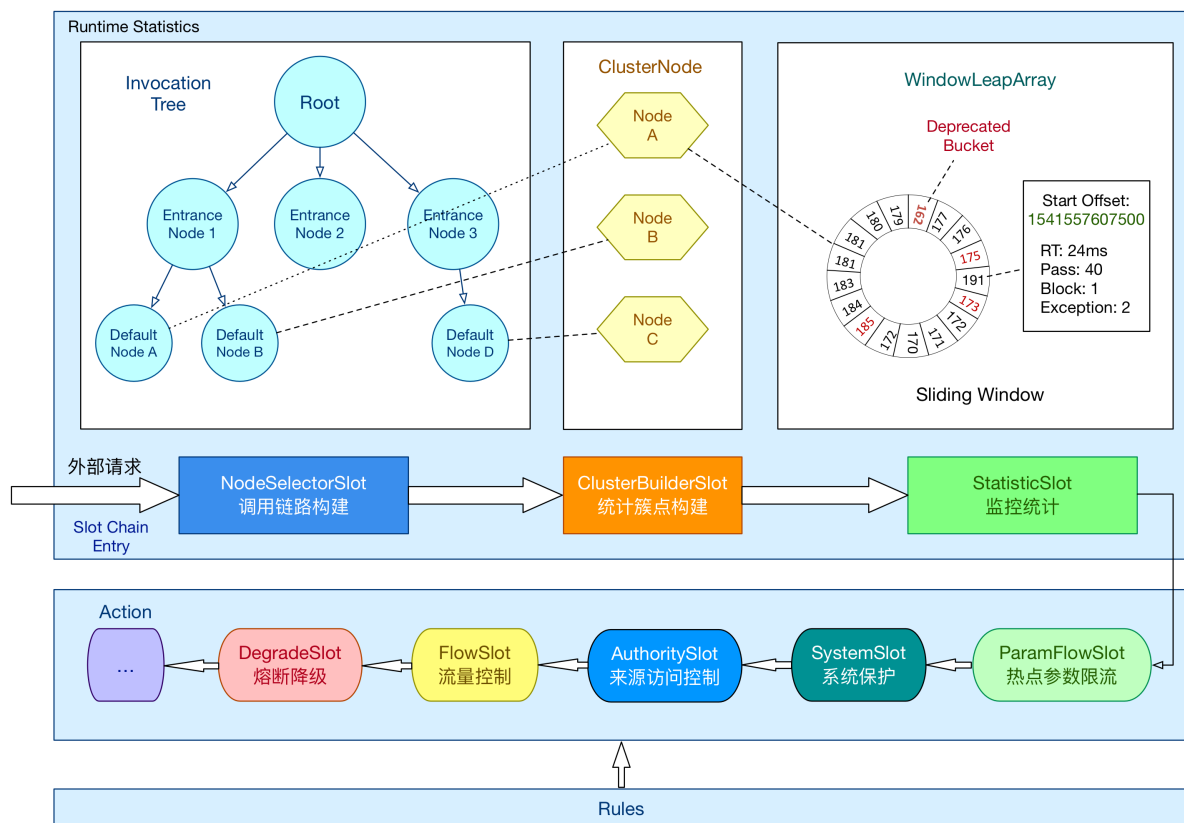
以下是Sentinel底层滑动时间窗口限流算法的简要实现步骤：

- 时间窗口划分：将整个时间范围划分为多个固定大小的时间窗口（例如1秒一个窗口）。这些时间窗口会随着时间的流逝依次滑动。
- 计数器：为每个时间窗口维护一个计数器，用于记录在该时间窗口内的请求数。
- 请求计数：当有请求到来时，将其计入当前时间窗口的计数器中。
- 滑动时间窗口：定期滑动时间窗口，将过期的时间窗口删除，并创建新的时间窗口。这样可以保持时间窗口的滚动。
- 限流判断：当有请求到来时，Sentinel会检查当前时间窗口内的请求数是否超过了预设的限制阈值。如果超过了限制阈值，请求将被拒绝或执行降级策略。
- 计数重置：定期重置过期时间窗口的计数器，以确保计数器不会无限增长。

这种滑动时间窗口算法允许在一段时间内平滑控制请求的流量，而不是仅基于瞬时请求速率进行限流。

它考虑了请求的历史分布，更适用于应对突发流量和周期性负载的情况。

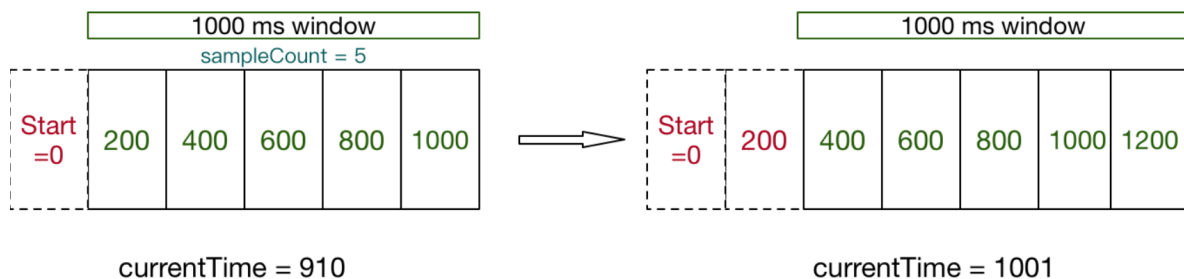
我们知道，Sentinel可以用来帮助我们实现流量控制、服务降级、服务熔断，而这些功能的实现都离不开接口被调用的实时指标数据，本文便是关于 Sentinel 是如何实现指标数据统计的。



上图中的右上角就是滑动窗口的示意图，是 StatisticSlot 的具体实现。

StatisticSlot 是 Sentinel 的核心功能插槽之一，用于统计实时的调用数据。

Sentinel 是基于滑动窗口实现的实时指标数据收集统计，底层采用高性能的滑动窗口数据结构 LeapArray 来统计实时的秒级指标数据，可以很好地支撑写多于读的高并发场景。



滑动窗口的核心数据结构

- **ArrayMetric:** 滑动窗口核心实现类。
- **LeapArray:** 滑动窗口顶层数据结构，包含一个一个的窗口数据。
- **WindowWrap:** 每一个滑动窗口的包装类，其内部的数据结构用 MetricBucket 表示。
- **MetricBucket:** 指标桶，例如通过数量、阻塞数量、异常数量、成功数量、响应时间，已通过未来配额（抢占下一个滑动窗口的数量）。
- **MetricEvent:** 指标类型，例如通过数量、阻塞数量、异常数量、成功数量、响应时间等。

ArrayMetric 源码

滑动窗口的入口类为 ArrayMetric，实现了 Metric 指标收集核心接口，该接口主要定义一个滑动窗口中成功的数量、异常数量、阻塞数量，TPS、响应时间等数据。

```

public class ArrayMetric implements Metric {

    private final LeapArray<MetricBucket> data;

    public ArrayMetric(int sampleCount, int intervalInMs, boolean enableOccupy)
    {
        if (enableOccupy) {
            this.data = new OccupiableBucketLeapArray(sampleCount,
intervalInMs);
        } else {
            this.data = new BucketLeapArray(sampleCount, intervalInMs);
        }
    }
}

```

- `int intervalInMs`：表示一个采集的时间间隔，即滑动窗口的总时间，例如 1 分钟。
- `int sampleCount`：在一个采集间隔中抽样的个数，默认为 2，即一个采集间隔中会包含两个相等的区间，一个区间就是一个窗口。
- `boolean enableOccupy`：是否允许抢占，即当前时间戳已经达到限制后，是否可以占用下一个时间窗口的容量。

LeapArray 源码

LeapArray 用来承载滑动窗口，即成员变量 `array`，类型为

`AtomicReferenceArray<WindowWrap<T>>`，保证创建窗口的原子性（CAS）。

```

public abstract class LeapArray<T> {

    //每一个窗口的时间间隔，单位为毫秒
    protected int windowLengthInMs;
    //抽样个数，就一个统计时间间隔中包含的滑动窗口个数
    protected int sampleCount;
    //一个统计的时间间隔
    protected int intervalInMs;
    //滑动窗口的数组，滑动窗口类型为 WindowWrap<MetricBucket>
    protected final AtomicReferenceArray<WindowWrap<T>> array;
    private final ReentrantLock updateLock = new ReentrantLock();

    public LeapArray(int sampleCount, int intervalInMs) {
        this.windowLengthInMs = intervalInMs / sampleCount;
        this.intervalInMs = intervalInMs;
        this.sampleCount = sampleCount;
        this.array = new AtomicReferenceArray<>(sampleCount);
    }
}

```

MetricBucket 源码

Sentinel 使用 MetricBucket 统计一个窗口时间内的各项指标数据，这些指标数据包括请求总数、成功总数、异常总数、总耗时、最小耗时、最大耗时等，而一个 Bucket 可以是记录一秒内的数据，也可以是 10 毫秒内的数据，这个时间长度称为窗口时间。

```

public class MetricBucket {
    /**
     * 存储各事件的计数，比如异常总数、请求总数等
     */
    private final LongAdder[] counters;
    /**
     * 这段事件内的最小耗时
     */
    private volatile long minRt;
}

```

Bucket 记录一段时间内的各项指标数据用的是一个 LongAdder 数组，数组的每个元素分别记录一个时间窗口内的请求总数、异常数、总耗时。也就是说：MetricBucket 包含一个 LongAdder 数组，数组的每个元素代表一类 MetricEvent。

LongAdder 保证了数据修改的原子性，并且性能比 AtomicLong 表现更好。

```

public enum MetricEvent {
    PASS,
    BLOCK,
    EXCEPTION,
    SUCCESS,
    RT,
    OCCUPIED_PASS
}

```

当需要获取 Bucket 记录总的成功请求数或者异常总数、总的请求处理耗时，可根据事件类型 (MetricEvent) 从 Bucket 的 LongAdder 数组中获取对应的 LongAdder，并调用 sum 方法获取总数。

```

public long get(MetricEvent event) {
    return counters[event.ordinal()].sum();
}

```

当需要 Bucket 记录一个成功请求或者一个异常请求、处理请求的耗时，可根据事件类型 (MetricEvent) 从 LongAdder 数组中获取对应的 LongAdder，并调用其 add 方法。

```

public void add(MetricEvent event, long n) {
    counters[event.ordinal()].add(n);
}

```

WindowWrap 源码

因为 Bucket 自身并不保存时间窗口信息，所以 Sentinel 给 Bucket 加了一个包装类 WindowWrap。Bucket 用于统计各项指标数据，WindowWrap 用于记录 Bucket 的时间窗口信息（窗口的开始时间、窗口的大小），WindowWrap 数组就是一个滑动窗口。

```

public class WindowWrap<T> {
    /**
     * 单个窗口的时间长度（毫秒）
     */
    private final long windowLengthInMs;
}

```

```

    /**
     * 窗口的开始时间戳（毫秒）
     */
    private long windowStart;
    /**
     * 统计数据
     */
    private T value;
}

```

总的来说：

- WindowWrap 用于包装 Bucket，随着 Bucket 一起创建。
- WindowWrap 数组实现滑动窗口，Bucket 只负责统计各项指标数据，WindowWrap 用于记录 Bucket 的时间窗口信息。
- 定位 Bucket 实际上是定位 WindowWrap，拿到 WindowWrap 就能拿到 Bucket。

滑动窗口 统计 源码实现

如果我们希望能够知道某个接口的每秒处理成功请求数（成功 QPS）、每秒处理失败请求数（失败 QPS），以及处理每个成功请求的平均耗时（avg RT），注意这里我们只需要控制 Bucket 统计一秒钟的指标数据即可，但如何才能确保 Bucket 存储的就是精确到 1 秒内的数据呢？

Sentinel 是这样实现的：定义一个 Bucket 数组，根据时间戳来定位到数组的下标。

由于只需要保存最近一分钟的数据。那么 Bucket 数组的大小就可以设置为 60，每个 Bucket 的 windowLengthInMs（窗口时间）大小就是 1 秒。

内存资源是有限的，而这个数组可以循环使用，并且永远只保存最近 1 分钟的数据，这样可以避免频繁的创建 Bucket，减少内存资源的占用。

那如何定位 Bucket 呢？我们只需要将当前时间戳减去毫秒部分，得到当前的秒数，再将得到的秒数与数组长度取余数，就能得到当前时间窗口的 Bucket 在数组中的位置（索引）。

calculateTimeIdx 方法中，取余数就是实现循环利用数组。

如果想要获取连续的一分钟的 Bucket 数据，就不能简单的从头开始遍历数组，而是指定一个开始时间和结束时间，从开始时间戳开始计算 Bucket 存放在数组中的下标，然后循环每次将开始时间戳加上 1 秒，直到开始时间等于结束时间。

```

private int calculateTimeIdx(long timeMillis) {
    long timeId = timeMillis / windowLengthInMs;
    return (int)(timeId % array.length());
}

```

由于循环使用的问题，当前时间戳与一分钟之前的时间戳和一分钟之后的时间戳都会映射到数组中的同一个 Bucket，

因此，必须要能够判断取得的 Bucket 是否是统计当前时间窗口内的指标数据，这便要数组每个元素都存储 Bucket 时间窗口的开始时间戳。

比如当前时间戳是 1577017626812，Bucket 统计一秒的数据，将时间戳的毫秒部分全部替换为 0，就能得到 Bucket 时间窗口的开始时间戳为 1577017626000。

```
//计算时间窗口开始时间戳
protected long calculateWindowStart(long timeMillis) {
    return timeMillis - timeMillis % windowLengthInMs;
}

//判断时间戳是否在当前时间窗口内
public boolean isTimeInWindow(long timeMillis) {
    return windowStart <= timeMillis && timeMillis < windowStart +
windowLengthInMs;
}
```

如何定位 Bucket?

通过时间戳 定位 Bucket的。

当接收到一个请求时，可根据接收到请求的时间戳计算出一个数组索引，从滑动窗口（WindowWrap 数组）中获取一个 WindowWrap，从而获取 WindowWrap 包装的 Bucket，调用 Bucket 的 add 方法记录相应的事件。

```
/**
 * 根据时间戳获取 bucket
 * @param timeMillis 时间戳（毫秒）
 * @return 如果时间有效，则在提供的时间戳处显示当前存储桶项；如果时间无效，则为空
 */
public WindowWrap<T> currentWindow(long timeMillis) {
    if (timeMillis < 0) {
        return null;
    }
    // 获取时间戳映射到的数组索引
    int idx = calculateTimeIdx(timeMillis);
    // 计算 bucket 时间窗口的开始时间
    long windowStart = calculateWindowStart(timeMillis);

    // 从数组中死循环查找当前的时间窗口，因为可能多个线程都在获取当前时间窗口
    while (true) {
        WindowWrap<T> old = array.get(idx);
        // 一般是项目启动时，时间未到达一个周期，数组还没有存储满，没有到复用阶段，所以数组元素可能为空
        if (old == null) {
            // 创建新的 bucket，并创建一个 bucket 包装器
            WindowWrap<T> window = new WindowWrap<T>(windowLengthInMs,
windowStart, newEmptyBucket(timeMillis));
            // cas 写入，确保线程安全，期望数组下标的元素是空的，否则就不写入，而是复用
            if (array.compareAndSet(idx, null, window)) {
                return window;
            } else {
                Thread.yield();
            }
        }
        // 如果 WindowWrap 的 windowStart 正好是当前时间戳计算出的时间窗口的开始时间，则就是我们想要的 bucket
        else if (windowStart == old.windowStart()) {
            return old;
        }
        // 复用旧的 bucket
        else if (windowStart > old.windowStart()) {
```

```

        if (updateLock.tryLock()) {
            try {
                // 重置 bucket, 并指定 bucket 的新时间窗口的开始时间
                return resetWindowTo(old, windowStart);
            } finally {
                updateLock.unlock();
            }
        } else {
            Thread.yield();
        }
    }

    // 计算出来的当前 bucket 时间窗口的开始时间比数组当前存储的 bucket 的时间窗口开始时间还小,
    // 直接返回一个空的 bucket 就行
    else if (windowStart < old.windowStart()) {
        return new WindowWrap<T>(windowLengthInMs, windowStart,
            new EmptyBucket(timeMillis));
    }
}
}
}

```

上面代码的实现是：通过当前时间戳计算出当前时间窗口的 (new) Bucket 在数组中的索引，通过索引从数组中取得 (old) Bucket；并计算 (new) Bucket 时间窗口的开始时间，与 (old) Bucket 时间窗口的开始时间作对比。

1. 如果旧的 bucket 不存在，那么我们在 windowStart 处创建一个新的 bucket，然后尝试通过 CAS 操作更新环形数组。只有一个线程可以成功更新，保证原子性。
2. 如果当前 windowStart 等于旧桶的开始时间戳，表示时间在桶内，所以直接返回桶。
3. 如果旧桶的开始时间戳落后于所提供的时间，这意味着桶已弃用，我们可以复用该桶，并将桶重置为当前的 windowStart。注意重置和清理操作很难是原子的，所以我们需要一个更新锁来保证桶更新的正确性。只有当 bucket 已弃用才会上锁，所以在大多数情况下它不会导致性能损失。
4. 不应该通过这里，因为提供的时间已经落后了，一般是时钟回拨导致的。

《尼恩学习圣经 系列PDF》内容、目标、意义：

现在**拿到offer超级难**，甚至连面试电话，一个都搞不到。

尼恩的技术社群中（50+），很多小伙伴凭借“左手云原生 + 右手大数据 + SpringCloud Alibaba 微服务”三大绝活，拿到了offer，并且是非常优质的offer，据说**年终奖都足足18个月**，非常令人羡慕。

问题是：“左手云原生 + 右手大数据 + SpringCloud Alibaba 微服务”**内容非常多，实操的环境非常复杂，底层原理很深。**

米饭要一口一口的吃，不能急。在这里，尼恩从架构师视角出发，左手云原生 + 右手大数据 + SpringCloud Alibaba 微服务 核心原理做一个宏观的介绍。

由于内容确实太多，所以写多个pdf电子书：

- (1) 《**Docker 学习圣经**》PDF （V1已经完成）
- (2) 《**SpringCloud Alibaba 微服务 学习圣经**》PDF （V2已经完成）
- (3) 《**Sentinel 学习圣经**》PDF （V3已经完成）
- (3) 《**K8S 学习圣经**》PDF （V1已经完成）

以上学习圣经，并且后续会持续升级，从V1版本一直迭代发布。就像咱们的《[尼恩 Java 面试宝典](#)》一样，已经迭代到V62啦。

40岁老架构师尼恩的掏心窝：通过一系列的学习圣经，带大家穿透“左手云原生+右手大数据+SpringCloud Alibaba 微服务”，实现技术自由，走向颠覆人生，让大家不迷路。

以上学习圣经的基础知识是尼恩的《高并发三部曲》，建议在看学习圣经之前，一定把尼恩的Java高并发三部曲过一遍，切记，切记。

《sentinel 学习圣经》说明：

尼恩建议：Hystrix和Sentinel对比学习

hystrix 服务保护

hystrix 作为老牌 SpringCloud 微服务保护的组件，很多项目仍然在使用，另外，底层原理都是想通的，大家可以和 sentinel 对比学习

hystrix 的详细介绍，请阅读《[Java 高并发核心编程 卷3 加强版](#)》

Sentinel 服务保护

sentinel 是SpringCloud 阿里巴巴的 微服务保护组件，所以，在学习的时候，sentinel 最好与 hystrix 对比学习，

《Sentinel 学习圣经》版本升级说明

本文《Sentinel 学习圣经》为《SpringCloud Alibaba 学习圣经》的 Sentinel 部分的升级版本。

在原书的基础上，完善和修改了 5000字左右。

最新的、完整的PDF，可以在文末公众号【技术自由圈】获取。

开始《sentinel 学习圣经》：一组核心基本概念

开发的原因，需要对吞吐量（TPS）、QPS、并发数、响应时间（RT）几个概念做下了解，查自百度百科，记录如下：

1. 响应时间(RT)

响应时间是指系统对请求作出响应的的时间。直观上看，这个指标与人对软件性能的主观感受是非常一致的，因为它完整地记录了整个计算机系统处理请求的时间。由于一个系统通常会提供许多功能，而不同功能的处理逻辑也千差万别，因而不同功能的响应时间也不尽相同，甚至同一功能在不同输入数据的情况下响应时间也不相同。所以，在讨论一个系统的响应时间时，人们通常是指该系统所有功能的平均时间或者所有功能的最大响应时间。当然，往往也需要对每个或每组功能讨论其平均响应时间和最大响应时间。

对于单机的没有并发操作的应用系统而言，人们普遍认为响应时间是一个合理且准确的性能指标。需要指出的是，响应时间的绝对值并不能直接反映软件的性能的高低，软件性能的高低实际上取决于用户对该响应时间的接受程度。对于一个游戏软件来说，响应时间小于100毫秒应该是不错的，响应时间在1秒左右可能属于勉强可以接受，如果响应时间达到3秒就完全难以接受了。而对于编译系统来说，完整编译一个较大规模软件的源代码可能需要几十分钟甚至更长时间，但这些响应时间对于用户来说都是可以接受的。

2. 吞吐量(Throughput)

吞吐量是指系统在单位时间内处理请求的数量。对于无并发的应用系统而言，吞吐量与响应时间成严格的反比关系，实际上此时吞吐量就是响应时间的倒数。前面已经说过，对于单用户的系统，响应时间（或者系统响应时间和应用延迟时间）可以很好地度量系统的性能，但对于并发系统，通常需要用吞吐量作为性能指标。

对于一个多用户的系统，如果只有一个用户使用系统的平均响应时间是 t ，当有 n 个用户使用，每个用户看到的响应时间通常并不是 $n \times t$ ，而往往比 $n \times t$ 小很多（当然，在某些特殊情况下也可能比 $n \times t$ 大，甚至大很多）。这是因为处理每个请求需要用到很多资源，由于每个请求的处理过程中有许多不难以并发执行，这导致在具体的一个时间点，所占资源往往并不多。也就是说在处理单个请求时，在每个时间点都可能有许多资源被闲置，当处理多个请求时，如果资源配置合理，每个用户看到的平均响应时间并不随用户数的增加而线性增加。实际上，不同系统的平均响应时间随用户数增加而增长的速度也不大相同，这也是采用吞吐量来度量并发系统的性能的主要原因。一般而言，吞吐量是一个比较通用的指标，两个具有不同用户数和用户使用模式的系统，如果其最大吞吐量基本一致，则可以判断两个系统的处理能力基本一致。

3. 并发用户数

并发用户数是指系统可以同时承载的正常使用系统功能的用户的数量。与吞吐量相比，并发用户数是一个更直观但也更笼统的性能指标。实际上，并发用户数是一个非常不准确的指标，因为用户不同的使用模式会导致不同用户在单位时间发出不同数量的请求。以网站系统为例，假设用户只有注册后才能使用，但注册用户并不是每时每刻都在使用该网站，因此具体一个时刻只有部分注册用户同时在线，在线用户就在浏览网站时会花很多时间阅读网站上的信息，因而具体一个时刻只有部分在线用户同时向系统发出请求。这样，对于网站系统我们会有三个关于用户数的统计数字：注册用户数、在线用户数和同时发请求用户数。由于注册用户可能长时间不登陆网站，使用注册用户数作为性能指标会造成很大的误差。而在线用户数和同时发请求用户数都可以作为性能指标。相比而言，以在线用户作为性能指标更直观些，而以同时发请求用户数作为性能指标更准确些。

4. QPS每秒查询率(Query Per Second)

每秒查询率QPS是对一个特定的查询服务器在规定时间内所处理流量多少的衡量标准，在因特网上，作为域名系统服务器的机器的性能经常用每秒查询率来衡量。对应fetches/sec，即每秒的响应请求数，也即是最大吞吐能力。（看来是类似于TPS，只是应用于特定场景的吞吐量）

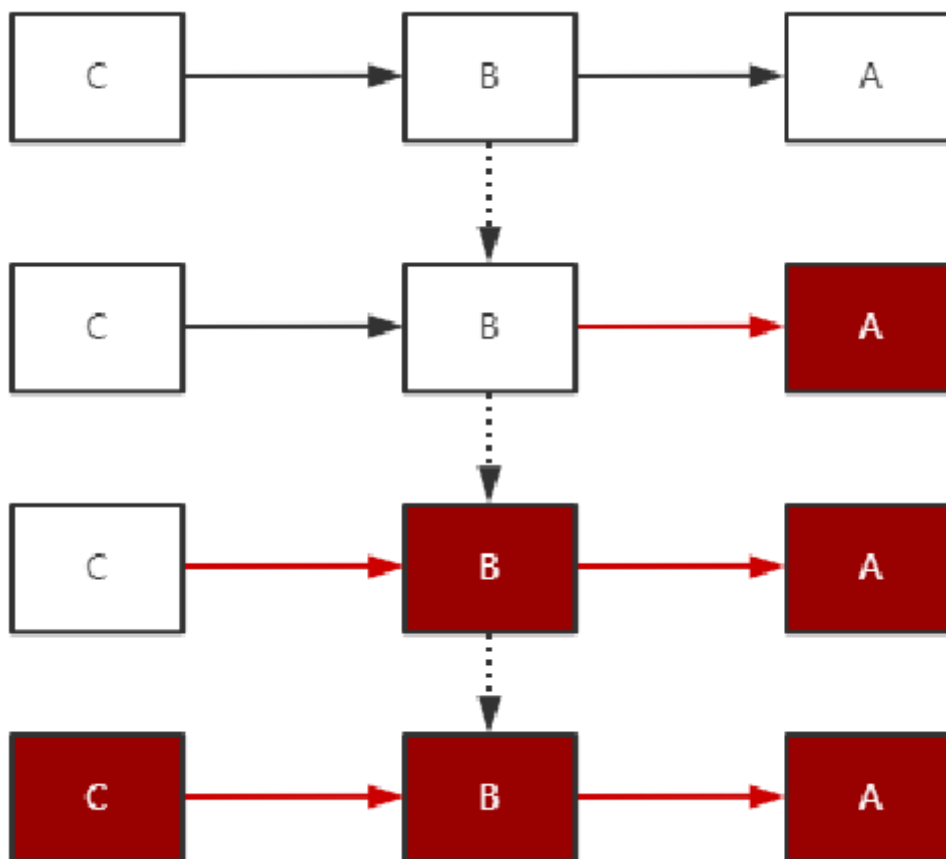
什么是服务雪崩效应？

在微服务架构系统中通常会有多个服务，在服务调用中如果出现基础服务故障，可能会导致级联故障，即一个服务不可用，可能导致所有调用它或间接调用它的服务都不可用，进而造成整个系统不可用的情况，这种现象也被称为服务雪崩效应。

服务雪崩效应是一种因“服务提供者不可用”（原因）导致“服务调用者不可用”（结果），并将不可用逐渐放大的现象。

服务雪崩效应示意如图所示，A为服务提供者，B为A的服务调用者，C为B的服务调用者。

当服务A因为某些原因导致不可用时，会引起服务B的不可用，并将不可用放大到服务C进而导致整个系统瘫痪，这样就形成了服务雪崩效应。



出现服务雪崩效应的原因如下：

- 硬件故障：如服务器宕机、机房断电、光纤被挖断等。
- 流量激增：如异常流量、重试加大流量等。
- 缓存穿透：一般发生在应用重启，所有缓存失效时，以及短时间内大量缓存失效时，因大量的缓存不命中，使请求直击后端服务，造成服务提供者超负荷运行，引起服务不可用。
- 程序bug：如程序逻辑导致死循环或者内存泄漏等。

如何解决服务器雪崩的方法有以下这些：

- 超时机制：在上游服务调用下游服务的时候，设置一个最大响应时间，如果超过这个时间，下游未作出反应，就断开请求，释放掉线程。
- 限流机制：限流就是限制系统的输入和输出流量已达到保护系统的目的。为了保证系统的稳固运行，一旦达到的需要限制的阈值，就需要限制流量并采取少量措施以完成限制流量的目的。
- 熔断机制：在互联网系统中，当下游服务因访问压力过大而响应变慢或失败，上游服务为了保护系统整体的可用性，可以暂时切断对下游服务的调用。这种牺牲局部，保全整体的措施就叫做熔断。

- 降级机制：降级是从系统功能优先级的角度考虑如何应对系统故障。服务降级指的是当服务器压力剧增的情况下，根据当前业务情况及流量对一些服务和页面有策略的降级，以此释放服务器资源以保证核心任务的正常运行。降级其实就是为服务提供一个兜底方案，一旦服务无法正常调用，就使用兜底方案。

Sentinel 为我们提供了多种的解决服务雪崩的方法：如超时机制、限流机制、熔断机制、降级机制等等，后面会为大家进行介绍

1、什么是Sentinel:

Sentinel是阿里开源的项目，提供了流量控制、熔断降级、系统负载保护等多个维度来保障服务之间的稳定性。

官网：<https://github.com/alibaba/Sentinel/wiki>

2012年，Sentinel诞生于阿里巴巴，其主要目标是流量控制。2013-2017年，Sentinel迅速发展，并成为阿里巴巴所有微服务的基本组成部分。它已在6000多个应用程序中使用，涵盖了几乎所有核心电子商务场景。2018年，Sentinel演变为一个开源项目。2020年，Sentinel Golang发布。

Sentinel的官方使用手册

<https://sentinelguard.io/zh-cn/docs/quick-start.html>

Sentinel 具有以下特征:

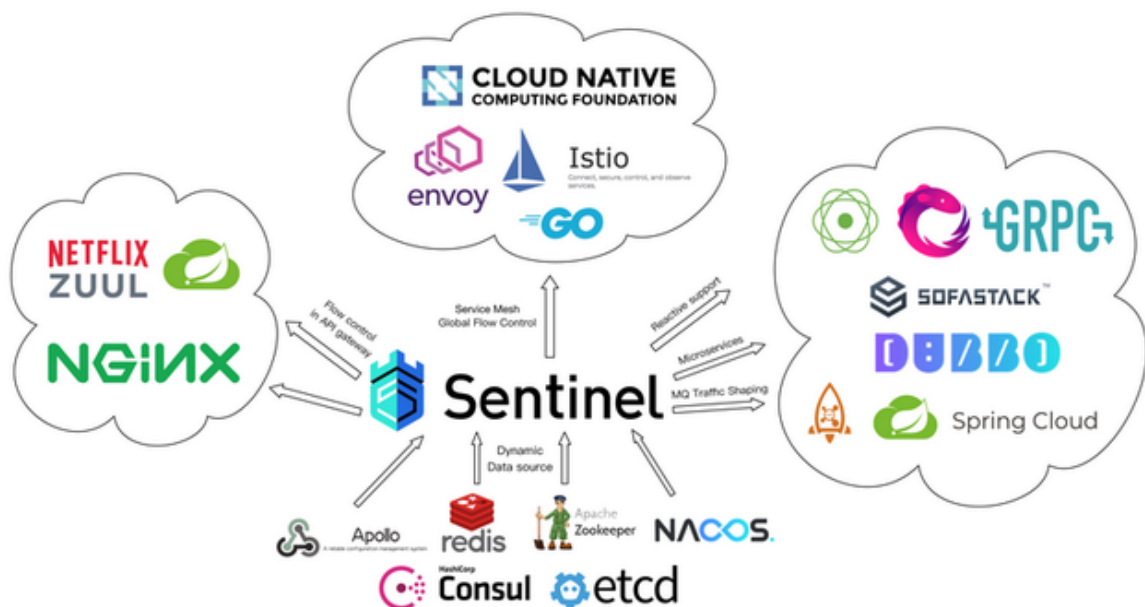
丰富的应用场景： Sentinel 承接了阿里巴巴近 10 年的双十一大促流量的核心场景，例如秒杀（即突发流量控制在系统容量可以承受的范围）、消息削峰填谷、集群流量控制、实时熔断下游不可用应用等。

完备的实时监控： Sentinel 同时提供实时的监控功能。您可以在控制台中看到接入应用的单台机器秒级数据，甚至 500 台以下规模的集群的汇总运行情况。

广泛的开源生态： Sentinel 提供开箱即用的与其它开源框架/库的整合模块，例如与 Spring Cloud、Dubbo、gRPC 的整合。您只需要引入相应的依赖并进行简单的配置即可快速地接入Sentinel。

完善的 SPI 扩展点： Sentinel 提供简单易用、完善的 SPI 扩展接口。您可以通过实现扩展接口来快速地定制逻辑。例如定制规则管理、适配动态数据源等。

Sentinel的生态圈



Sentinel主要特性:



关于Sentinel与Hystrix的区别见: <https://yq.aliyun.com/articles/633786/>

Sentinel很多的特性和Hystrix有很多类似的功能。以下是Sentinel和Hystrix的对比。

Sentinel与Hystrix的区别

Sentinel	Hystrix	
隔离策略	基于并发数	线程池隔离/信号量隔离
熔断降级策略	基于响应时间或失败比率	基于失败比率
实时指标实现	滑动窗口	滑动窗口（基于 RxJava）
规则配置	支持多种数据源	支持多种数据源
扩展性	多个扩展点	插件的形式
基于注解的支持	即将发布	支持
调用链路信息	支持同步调用	不支持
限流	基于 QPS / 并发数，支持基于调用关系的限流	不支持
流量整形	支持慢启动、匀速器模式	不支持
系统负载保护	支持	不支持
实时监控 API	各式各样	较为简单
控制台	开箱即用，可配置规则、查看秒级监控、机器发现等	不完善
常见框架的适配	Servlet、Spring Cloud、Dubbo、gRPC 等	Servlet、Spring Cloud Netflix

Hystrix 迁移Sentinel 方案

Sentinel 官方提供了详细的由Hystrix 迁移到Sentinel 的方法

Hystrix 功能	迁移方案
线程池隔离/信号量隔离	Sentinel 不支持线程池隔离；信号量隔离对应 Sentinel 中的线程数限流，详见 此处
熔断器	Sentinel 支持按平均响应时间、异常比率、异常数来进行熔断降级。从 Hystrix 的异常比率熔断迁移的步骤详见 此处
Command 创建	直接使用 Sentinel <code>sphu</code> API 定义资源即可，资源定义与规则配置分离，详见 此处
规则配置	在 Sentinel 中可通过 API 硬编码配置规则，也支持多种动态规则源
注解支持	Sentinel 也提供注解支持，可以很方便地迁移，详见 此处
开源框架支持	Sentinel 提供 Servlet、Dubbo、Spring Cloud、gRPC 的适配模块，开箱即用；若之前使用 Spring Cloud Netflix，可迁移至 Spring Cloud Alibaba

sentinel组件介绍

Sentinel两个部分:

- 控制台 (Dashboard) : Sentinel 提供的一个轻量级的开源控制台, 它为用户提供了机器自发现、簇点链路自发现、监控、规则配置等功能。控制台主要负责管理推送规则、监控、集群限流分配管理、机器发现等。
- 核心库 (Java 客户端) : 不依赖任何框架/库, 能够运行于 Java 7 及以上的版本的运行时环境, 同时对 Dubbo / Spring Cloud 等框架也有较好的支持。

sentinel 核心概念

① 资源:

资源是 Sentinel 的关键概念。它可以是 Java 应用程序中的任何内容, 例如由应用程序提供的服务或者是服务里的方法, 甚至可以是一段代码。

Sentinel 定义资源的方式有下面几种: 适配主流框架自动定义资源、通过 SphU 手动定义资源、通过 SphO 手动定义资源、注解方式定义资源。这个稍后会有使用方法教程。

其中注解方式定义资源@SentinelResource参数介绍如下:

参数	解释
value	Sentinel资源的名称，我们不仅可以通过url进行限流，也可以把此值作为资源名配置，一样可以限流。
entryType	条目类型（入站或出站），默认为出站（EntryType.OUT）
resourceType	资源的分类（类型）
blockHandler	块异常函数的名称，默认为空
blockHandlerClass	指定块处理方法所在的类。默认情况下，blockHandler与原始方法位于同一类中。但是，如果某些方法共享相同的签名并打算设置相同的块处理程序，则用户可以设置存在块处理程序的类。请注意，块处理程序方法必须是静态的。
fallback	后备函数的名称，默认为空
defaultFallback	默认后备方法的名称，默认为空
defaultFallback用作默认的通用后备方法。它不应接受任何参数，并且返回类型应与原始方法兼容	
fallbackClass	fallback方法所在的类（仅单个类）。默认情况下，fallback与原始方法位于同一类中。但是，如果某些方法共享相同的签名并打算设置相同的后备，则用户可以设置存在后备功能的类。请注意，共享的后备方法必须是静态的。
exceptionsToTrace	异常类的列表追查，默认 Throwable
exceptionsToIgnore	要忽略的异常类列表，默认情况下为空

②规则：围绕资源而设定的规则。

Sentinel 支持流量控制、熔断降级、系统保护、来源访问控制和热点参数等多种规则，所有这些规则都可以动态实时调整。

Sentinel 的使用

Sentinel中的管理控制台

1) 获取 Sentinel 控制台

您可以从 <https://github.com/alibaba/Sentinel/releases> 下载最新版本的控制台 jar 包。

您可以从官方 网站中 下载最新版本的控制台 jar 包，下载地址如下：

```
https://github.com/alibaba/Sentinel/releases/download/1.6.3/sentinel-dashboard-1.7.1.jar
```

您也可以从最新版本的源码自行构建 Sentinel 控制台：

- 下载 [控制台](#) 工程
- 使用以下命令将代码打包成一个 fat jar: `mvn clean package`

2) sentinel服务启动

启动 sentinel

普通进程

```
java -server -Xms64m -Xmx256m -Dserver.port=8849 -Dcsp.sentinel.dashboard.server=localhost:8849 -Dproject.name=sentinel-dashboard -jar /work/sentinel-dashboard-1.8.6.jar
```

守护进程

```
nohup java -server -Xms64m -Xmx256m -Dserver.port=8849 -Dcsp.sentinel.dashboard.server=localhost:8849 -Dproject.name=sentinel-dashboard -jar /work/sentinel-dashboard-1.8.6.jar 2>&1 &
```

或者

```
/usr/bin/su - root -c "nohup java -server -Xms64m -Xmx256m -Dserver.port=8849 -Dcsp.sentinel.dashboard.server=localhost:8849 -Dproject.name=sentinel-dashboard -jar /work/sentinel-dashboard-1.8.6.jar 2>&1 &"
```

开机启动：启动命令可以加入到启动的 rc.local 配置文件，之后做到开机启动

```
1 Last login: Thu Oct 19 10:38:32 2023
2 [root@centos1 ~]# cat /etc/rc.local
# /bin/bash
# THIS FILE IS ADDED FOR COMPATIBILITY PURPOSES
#
# It is highly advisable to create own systemd services or udev rules
# to run scripts during boot instead of using this file.
#
# In contrast to previous versions due to parallel execution during boot
# this script will NOT be run after all other services.
#
# Please note that you must run 'chmod +x /etc/rc.d/rc.local' to ensure
# that this script will be executed during boot.
touch /var/lock/subsys/local

#start nginx
# /usr/bin/su - root -c "/usr/local/openresty/nginx/sbin/nginx -c /usr/local/openresty/nginx/conf/nginx.conf"

# 启动 Nacos
# /usr/bin/su - root -c "/work/nacos/nacos-server-2.0.3/bin/startup.sh -m standalone"

#start springcloud
sleep 10s
# /usr/bin/su - root -c "/work/cloud-eureka-1.0-SNAPSHOT/bin/deploy.sh start"
sleep 20s
# /usr/bin/su - root -c "/work/cloud-config-1.0-SNAPSHOT/bin/deploy.sh start"

#启动 sentinel
# /usr/bin/su - root -c "nohup java -server -Xms64m -Xmx256m -Dserver.port=8849 -Dcsp.sentinel.dashboard.server=localhost:8849 -Dproject.name=sentinel-dashboard -jar /work/sentinel-dashboard-1.7.1.jar 2>&1 &"
```

控制台端口:

启动 Sentinel 控制台需要 JDK 版本为 1.8 及以上版本,

`-Dserver.port=8849` 用于指定 Sentinel 控制台端口为 8849

控制台登录

从 Sentinel 1.6.0 起, Sentinel 控制台引入基本的登录功能, 默认用户名和密码都是 `sentinel`。

可以参考 [鉴权模块文档](#) 配置用户名和密码。

注: 若您的应用为 Spring Boot 或 Spring Cloud 应用, 您可以通过 Spring 配置文件来指定配置, 详情请参考 [Spring Cloud Alibaba Sentinel 文档](#)。

启动日志

使用如下命令启动控制台:

其中 `-Dserver.port=8849` 用于指定 Sentinel 控制台端口为 8849。

从 Sentinel 1.6.0 起, Sentinel 控制台引入基本的登录功能, 默认用户名和密码都是 `sentinel`。可以参考 [鉴权模块文档](#) 配置用户名和密码。

启动日志如下

```
[root@192 ~]# java -Dserver.port=8888 -Dcsp.sentinel.dashboard.server=localhost:8888 -Dproject.name=sentinel-dashboard -jar sentinel-dashboard-1.8.6.jar
INFO: log base dir is: /root/logs/csp/
INFO: log name use pid is: false
```

```
. _ _ _ _ _
/\ / _ _ _ _ _ ( ) _ _ _ _ _ \ \ \ \ \
( ( ) _ _ _ _ _ | ' _ | ' _ | ' _ \ / _ _ \ \ \ \ \
```

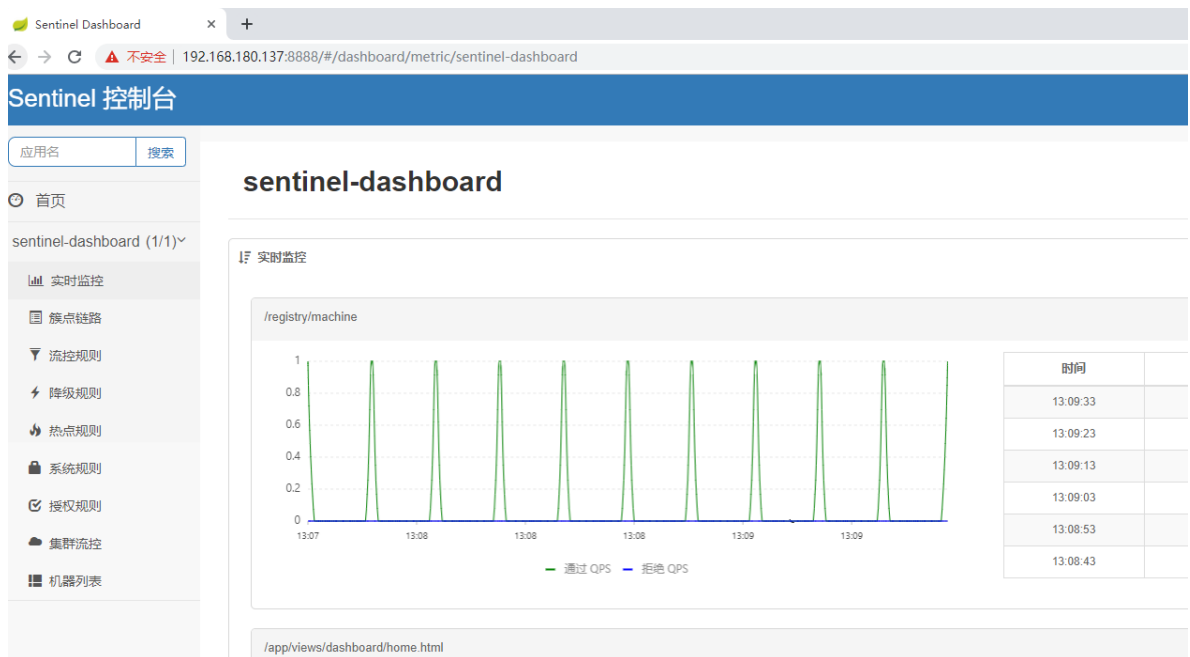
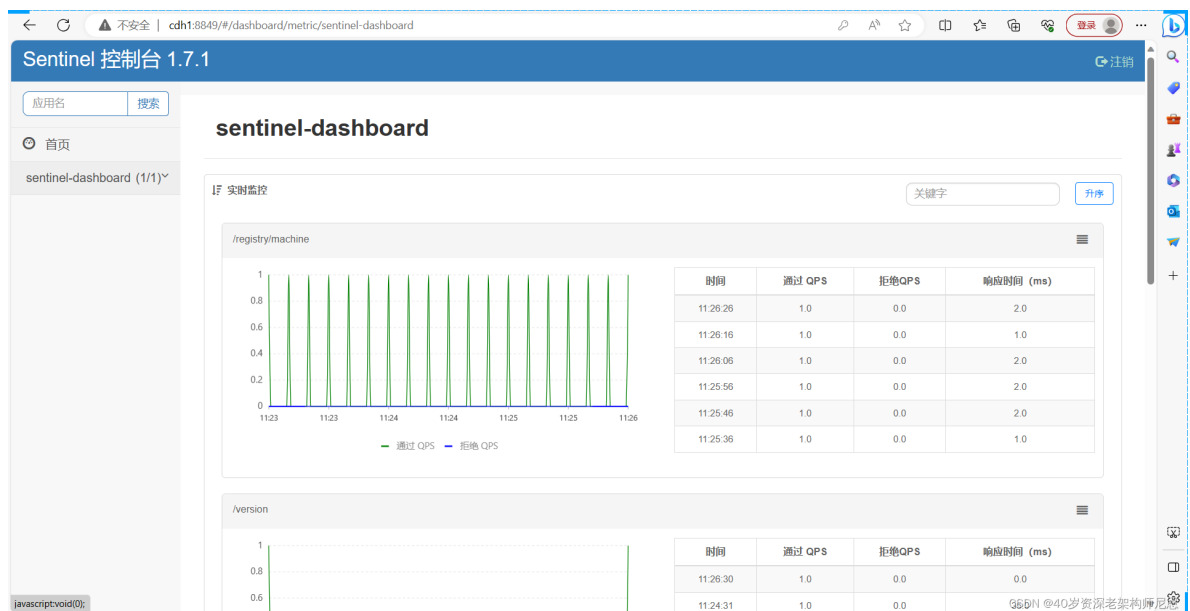
```
2020-02-08 13:07:29.316 INFO 114031 --- [          main]
c.a.c.s.dashboard.DashboardApplication : Starting DashboardApplication on
192.168.180.137 with PID 114031 (/root/sentinel-dashboard-1.6.3.jar started by
root in /root)
2020-02-08 13:07:29.319 INFO 114031 --- [          main]
c.a.c.s.dashboard.DashboardApplication : No active profile set, falling back
to default profiles: default
2020-02-08 13:07:29.456 INFO 114031 --- [          main]
ConfigServletWebServerApplicationContext : Refreshing
org.springframework.boot.web.servlet.context.AnnotationConfigServletWebServerApp
licationContext@59690aa4: startup date [Sat Feb 08 13:07:29 CST 2020]; root of
context hierarchy
2020-02-08 13:07:33.783 INFO 114031 --- [          main]
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8888
(http)
```

Sentinel 控制台使用

登录地址 <http://cdh1:8849/#/login>



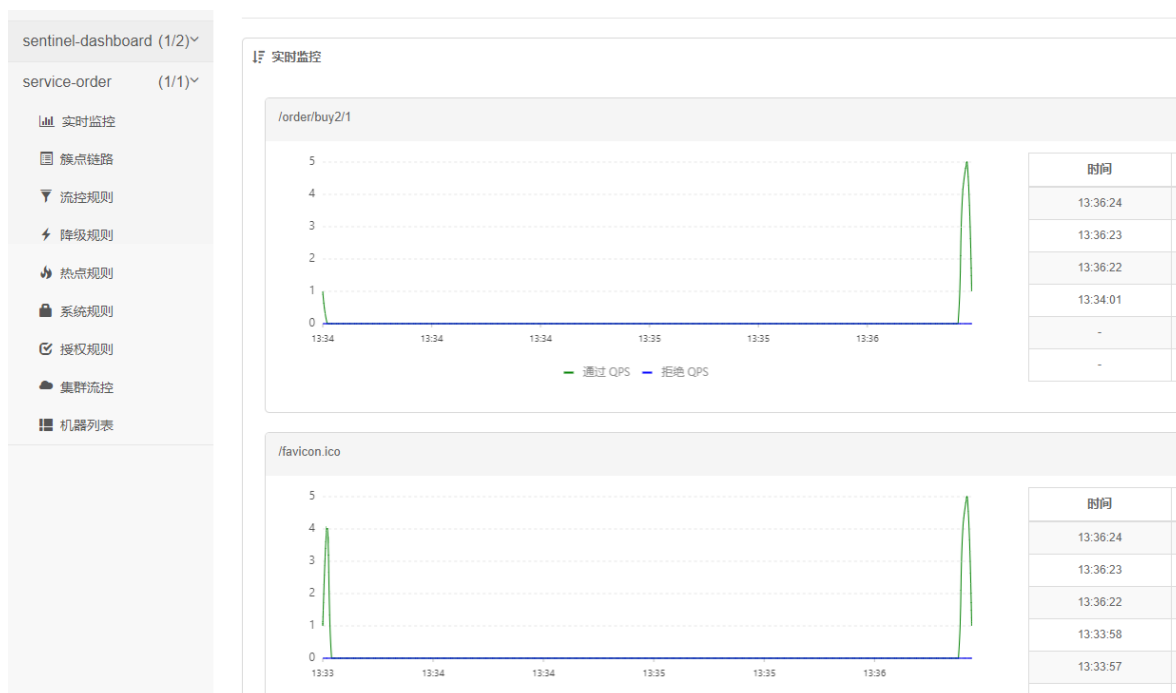
默认用户名和密码都是 sentinel



查看机器列表以及健康情况

默认情况下Sentinel 会在客户端首次调用的时候进行初始化，开始向控制台发送心跳包。

也可以配置 `sentinel.eager=true` ,取消Sentinel控制台懒加载。



SpringCloud客户端能接入控制台

控制台启动后，客户端需要按照以下步骤接入到控制台。

父工程引入 alibaba实现的SpringCloud

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>Greenwich.RELEASE</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
    <dependency>
      <groupId>com.alibaba.cloud</groupId>
      <artifactId>spring-cloud-alibaba-dependencies</artifactId>
      <version>2.1.0.RELEASE</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

子工程中引入 sentinel

```
<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
</dependency>
```

配置启动参数

在工程的application.yml中添加Sentinel 控制台配置信息

```
spring:
  cloud:
    sentinel:
      transport:
        dashboard: 192.168.180.137:8849    #sentinel控制台的请求地址
```

这里的 spring.cloud.sentinel.transport.dashboard 配置控制台的请求路径。

2、使用 Sentinel 来进行熔断与限流

Sentinel 可以简单的分为 Sentinel 核心库和 Dashboard。核心库不依赖 Dashboard，但是结合 Dashboard 可以取得最好的效果。

使用 Sentinel 来进行熔断保护，主要分为几个步骤：

1. 定义资源

资源：可以是任何东西，一个服务，服务里的方法，甚至是一段代码。

2. 定义规则

规则：Sentinel 支持以下几种规则：流量控制规则、熔断降级规则、系统保护规则、来源访问控制规则和热点参数规则。

3. 检验规则是否生效

Sentinel 的所有规则都可以在内存态中动态地查询及修改，修改之后立即生效。先把可能需要保护的资源定义好，之后再配置规则。

也可以理解为，只要有了资源，我们就可以在任何时候灵活地定义各种流量控制规则。在编码的时候，只需要考虑这个代码是否需要保护，如果需要保护，就将之定义为一个资源。

2.1 Java普通应用限流

1. 引入 Sentinel 依赖

如果您的应用使用了 Maven，则在 pom.xml 文件中加入以下代码即可：

```
<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-core</artifactId>
  <version>1.8.6</version>
</dependency>
```

如果您未使用依赖管理工具，请到 [Maven Center Repository](#) 直接下载 JAR 包。

2. 定义资源

资源 是 Sentinel 中的核心概念之一。最常用的资源是我们代码中的 Java 方法。当然，您也可以更灵活的定义你的资源，例如，把需要控制流量的代码用 Sentinel API `SphU.entry("HelloWorld")` 和 `entry.exit()` 包围起来即可。在下面的例子中，我们将 `System.out.println("hello world");` 作为资源（被保护的逻辑），用 API 包装起来。参考代码如下：

```
public static void main(String[] args) {
    // 配置规则.
    initFlowRules();

    while (true) {
        // 1.5.0 版本开始可以直接利用 try-with-resources 特性
        try (Entry entry = SphU.entry("HelloWorld")) {
            // 被保护的逻辑
            System.out.println("hello world");
        } catch (BlockException ex) {
            // 处理被流控的逻辑
            System.out.println("blocked!");
        }
    }
}
```

完成以上两步后，代码端的改造就完成了。

您也可以通过我们提供的 [注解支持模块](#)，来定义我们的资源，类似于下面的代码：

```
@SentinelResource("HelloWorld")
public void helloWorld() {
    // 资源中的逻辑
    System.out.println("hello world");
}
```

这样，`helloWorld()` 方法就成了我们的一个资源。注意注解支持模块需要配合 Spring AOP 或者 AspectJ 一起使用。

3. 定义规则

接下来，通过流控规则来指定允许该资源通过的请求次数，例如下面的代码定义了资源 `HelloWorld` 每秒最多只能通过 20 个请求。

```
private static void initFlowRules(){
    List<FlowRule> rules = new ArrayList<>();
    FlowRule rule = new FlowRule();
    rule.setResource("HelloWorld");
    rule.setGrade(RuleConstant.FLOW_GRADE_QPS);
    // Set limit QPS to 20.
    rule.setCount(20);
    rules.add(rule);
    FlowRuleManager.loadRules(rules);
}
```

完成上面 3 步，Sentinel 就能够正常工作了。更多的信息可以参考 [使用文档](#)。

4. 检查效果

Demo 运行之后，我们可以在日志 `~/logs/csp/${appName}-metrics.log.xxx` 里看到下面的输出：

```
|--timestamp-|-----date time----|--resource-|p|block|s|e|rt
1529998904000|2018-06-26 15:41:44|hello world|20|0|20|0|0
1529998905000|2018-06-26 15:41:45|hello world|20|5579|20|0|728
1529998906000|2018-06-26 15:41:46|hello world|20|15698|20|0|0
1529998907000|2018-06-26 15:41:47|hello world|20|19262|20|0|0
1529998908000|2018-06-26 15:41:48|hello world|20|19502|20|0|0
1529998909000|2018-06-26 15:41:49|hello world|20|18386|20|0|0
```

其中 `p` 代表通过的请求，`block` 代表被阻止的请求，`s` 代表成功执行完成的请求个数，`e` 代表用户自定义的异常，`rt` 代表平均响应时长。

可以看到，这个程序每秒稳定输出 "hello world" 20 次，和规则中预先设定的阈值是一样的。

更详细的说明可以参考: [如何使用](#)

更多的例子可以参考: [Sentinel Demo 集锦](#)

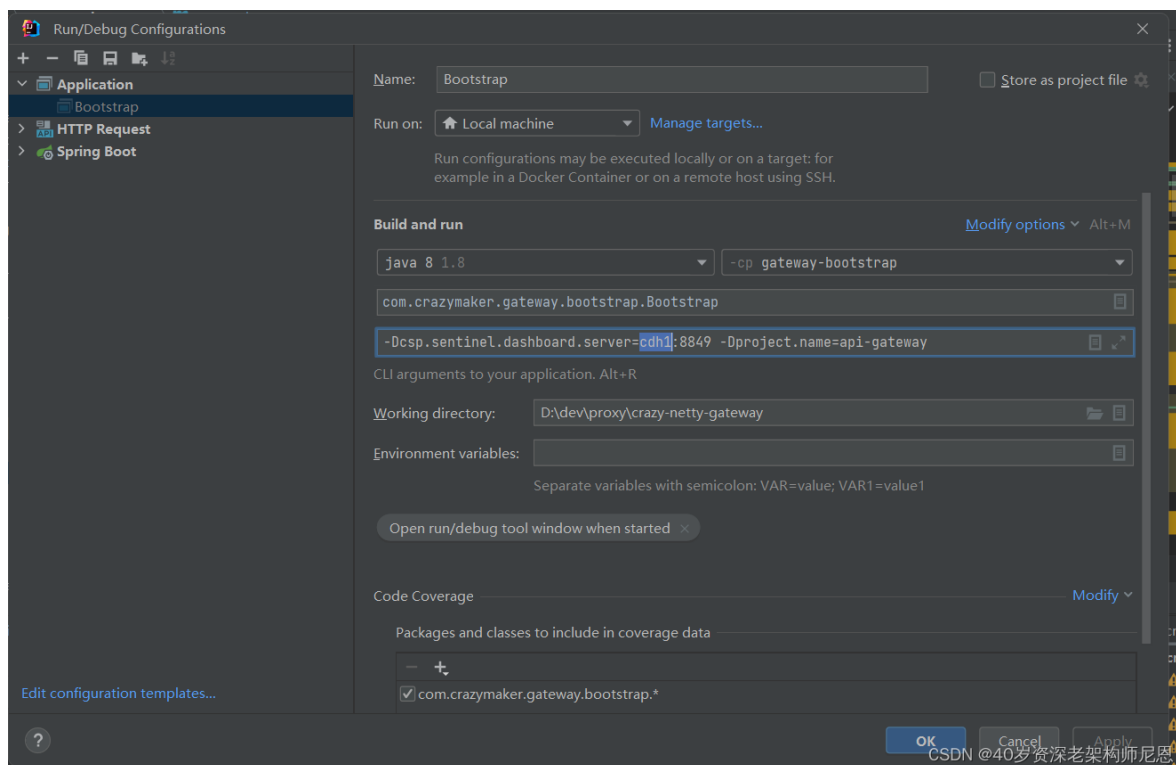
5. 接入控制台

客户端需要引入 Transport 模块来与 Sentinel 控制台进行通信。您可以通过 `pom.xml` 引入 JAR 包：

```
<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-transport-simple-http</artifactId>
</dependency>
```

启动时加入 JVM 参数 `-Dcsp.sentinel.dashboard.server=consoleIp:port` 指定控制台地址和端口。

```
-Dcsp.sentinel.dashboard.server=cdh1:8849 -Dproject.name=api-gateway
-Dcsp.sentinel.dashboard.server=192.169.56.121:8849 -Dproject.name=api-gateway
telnet 192.168.56.121 8849
```



2.1 定义资源

资源是 Sentinel 的关键概念。

它可以是 Java 应用程序中的任何内容，例如，由应用程序提供的服务，或由应用程序调用的其它应用提供的服务，RPC接口方法，甚至可以是一段代码。

只要通过 Sentinel API 定义的代码，就是资源，能够被 Sentinel 保护起来。

大部分情况下，可以使用方法签名，URL，甚至服务名称作为资源名来标示资源。

把需要控制流量的代码用 Sentinel的关键代码 `SphU.entry("资源名")` 和 `entry.exit()` 包围起来即可。

实例代码：

```
Entry entry = null;
try {
    // 定义一个sentinel保护的资源，名称为test-sentinel-api
    entry = SphU.entry(resourceName);
    // 模拟执行被保护的逻辑耗时
    Thread.sleep(100);
    return a;
} catch (BlockException e) {
    // 如果被保护的资源被限流或者降级了，就会抛出BlockException
    log.warn("资源被限流或降级了", e);
    return "资源被限流或降级了";
} catch (InterruptedException e) {
    return "发生InterruptedException";
} finally {
    if (entry != null) {
        entry.exit();
    }
}
```

```

        ContextUtil.exit();
    }
}

```

在下面的例子中，用 try-with-resources 来定义资源。参考代码如下：

```

public static void main(String[] args) {
    // 配置规则.
    initFlowRules();

    while (true) {
        // 1.5.0 版本开始可以直接利用 try-with-resources 特性
        try (Entry entry = SphU.entry("HelloWorld")) {
            // 被保护的逻辑
            System.out.println("hello world");
        } catch (BlockException ex) {
            // 处理被流控的逻辑
            System.out.println("blocked!");
        }
    }
}

```

资源注解@SentinelResource

也可以使用Sentinel提供的注解@SentinelResource来定义资源，实例如下：

```

@SentinelResource("HelloWorld")
public void helloworld() {
    // 资源中的逻辑
    System.out.println("hello world");
}

```

@SentinelResource 注解

注意：注解方式埋点不支持 private 方法。

@SentinelResource 用于定义资源，并提供可选的异常处理和 fallback 配置项。

@SentinelResource 注解包含以下属性：

- value：资源名称，必需项（不能为空）
- entryType：entry 类型，可选项（默认为 EntryType.OUT）
- blockHandler / blockHandlerClass：

blockHandler 对应处理 BlockException的函数名称，可选项。blockHandler 函数访问范围需要是 public，返回类型需要与原方法相匹配，参数类型需要和原方法相匹配并且最后加一个额外的参数，类型为 BlockException。blockHandler 函数默认需要和原方法在同一个类中。若希望使用其他类的函数，则可以指定 blockHandlerClass 为对应的类的 Class 对象，注意对应的函数必需为 static 函数，否则无法解析。

- fallback /fallbackClass

fallback 函数名称，可选项，用于在抛出异常的时候提供 fallback 处理逻辑。fallback 函数可以针对所有类型的异常（除了exceptionsToIgnore里面排除掉的异常类型）进行处理。

- defaultFallback

(since 1.6.0)：默认的 fallback 函数名称，可选项，通常用于通用的 fallback 逻辑（即可以用于很多服务或方法）。默认 fallback 函数可以针对所有类型的异常（除了exceptionsToIgnore里面排除掉的异常类型）进行处理。若同时配置了 fallback 和 defaultFallback，则只有 fallback 会生效。

fallback 函数签名和位置要求：

- 返回值类型必须与原函数返回值类型一致；
- 方法参数列表需要和原函数一致，或者可以额外多一个 Throwable 类型的参数用于接收对应的异常。
- fallback 函数默认需要和原方法在同一个类中。若希望使用其他类的函数，则可以指定 fallbackClass为对应的类的 Class 对象，注意对应的函数必需为 static 函数，否则无法解析。

defaultFallback 函数签名要求：

- 返回值类型必须与原函数返回值类型一致；
- 方法参数列表需要为空，或者可以额外多一个 Throwable 类型的参数用于接收对应的异常。
- defaultFallback 函数默认需要和原方法在同一个类中。若希望使用其他类的函数，则可以指定 fallbackClass 为对应的类的 Class 对象，注意对应的函数必需为 static 函数，否则无法解析。
- exceptionsToIgnore (since 1.6.0)：用于指定哪些异常被排除掉，不会计入异常统计中，也不会进入 fallback 逻辑中，而是会原样抛出。

2.3 定义规则

规则主要有流控规则、熔断降级规则、系统规则、权限规则、热点参数规则等：

一段硬编码的方式定义流量控制规则如下：

```
private void initSystemRule() {
    List<SystemRule> rules = new ArrayList<>();
    SystemRule rule = new SystemRule();
    rule.setHighestSystemLoad(10);
    rules.add(rule);
    SystemRuleManager.loadRules(rules);
}
```

加载规则：

```
FlowRuleManager.loadRules(List<FlowRule> rules); // 修改流控规则
DegradRuleManager.loadRules(List<DegradRule> rules); // 修改降级规则
SystemRuleManager.loadRules(List<SystemRule> rules); // 修改系统规则
AuthorityRuleManager.loadRules(List<AuthorityRule> rules); // 修改授权规则
```

3、sentinel 熔断降级

3.1 什么是熔断降级

熔断降级对调用链路中不稳定的资源进行熔断降级是保障高可用的重要措施之一。

由于调用关系的复杂性，如果调用链路中的某个资源不稳定，最终会导致请求发生堆积。

Sentinel 熔断降级会在调用链路中某个资源出现不稳定状态时（例如调用超时或异常比例升高），对这个资源的调用进行限制，让请求快速失败，避免影响到其它的资源而导致级联错误。当资源被降级后，在接下来的降级时间窗口之内，对该资源的调用都自动熔断（默认行为是抛出 `DegradeException`）

3.2 熔断降级规则

熔断降级规则包含下面几个重要的属性：

Field	说明	默认值
resource	资源名，即规则的作用对象	
grade	熔断策略，支持慢调用比例/异常比例/异常数量策略	慢调用比例
count	慢调用比例模式下为慢调用临界 RT（超出该值计为慢调用）；异常比例/异常数模式下为对应的阈值	
timeWindow	熔断时长，单位为 s	
minRequestAmount	熔断触发的最小请求数，请求数小于该值时即使异常比率超出阈值也不会熔断（1.7.0 引入）	5
statIntervalMs	统计时长（单位为 ms），如 60*1000 代表分钟级（1.8.0 引入）	1000 ms
slowRatioThreshold	慢调用比例阈值，仅慢调用比例模式有效（1.8.0 引入）	

3.3 几种降级策略

我们通常用以下几种降级策略：

- 平均响应时间 (DEGRADE_GRADE_RT)：

当资源的平均响应时间超过阈值（`DegradeRule` 中的 `count`，以 ms 为单位）之后，资源进入准降级状态。接下来如果持续进入 5 个请求，它们的 RT 都持续超过这个阈值，那么在接下的时间窗口（`DegradeRule` 中的 `timewindow`，以 s 为单位）之内，对这个方法的调用都会自动地返回（抛出 `DegradeException`）。在下一个时间窗口到来时，会接着再放入 5 个请求，再重复上面的判断。

```
// 初始化降级规则
DegradeRule degradeRule = new DegradeRule();
degradeRule.setResource("queryGoodsInfo");
//响应时间
degradeRule.setGrade(RuleConstant.DEGRADE_GRADE_RT);
//超过100ms
degradeRule.setCount(100);
//熔断打开后，经过多少时间进入半打开
degradeRule.setTimewindow(10);
```

注意 Sentinel 默认统计的 RT 上限是 4900 ms，超出此阈值的都会算作 4900 ms，若需要变更此上限可以通过启动配置项 -Dcsp.sentinel.statistic.max.rt=xxx 来配置。

- 异常比例 (DEGRADE_GRADE_EXCEPTION_RATIO):

当资源的每秒请求量 $\geq N$ (可配置)，并且每秒异常总数占通过量的比值超过阈值 (DegradeRule 中的 count) 之后，资源进入降级状态，即在接下的时间窗口 (DegradeRule 中的 timeWindow，以 s 为单位) 之内，对这个方法的调用都会自动地返回。异常比率的阈值范围是 [0.0, 1.0]，代表 0% - 100%。

当资源的每秒异常总数占通过量的比值超过阈值 (DegradeRule 中的 count) 之后，资源进入降级状态，即在接下的时间窗口 (DegradeRule 中的 timeWindow，以 s 为单位) 之内，对这个方法的调用都会自动地返回。

异常比率的阈值范围是 [0.0, 1.0]，代表 0% - 100%。

- 异常数 (DEGRADE_GRADE_EXCEPTION_COUNT):

当资源近 1 分钟的异常数目超过阈值之后会进行熔断。

当资源近 1 分钟的异常数目超过阈值之后会进行熔断。注意由于统计时间窗口是分钟级别的，若 timeWindow 小于 60s，则结束熔断状态后仍可能再进入熔断状态。

注意由于统计时间窗口是分钟级别的，若 timeWindow 小于 60s，则结束熔断状态后仍可能再进入熔断状态。

3.4 熔断降级代码实现

可以通过调用 DegradeRuleManager.loadRules() 方法来用硬编码的方式定义流量控制规则。

异常数 (DEGRADE_GRADE_EXCEPTION_COUNT) 熔断:

```
@PostConstruct
public void initSentinelRule()
{
    //熔断规则：5s内调用接口出现异常次数超过5的时候，进行熔断
    List<DegradeRule> degradeRules = new ArrayList<>();
    DegradeRule rule = new DegradeRule();
    rule.setGrade(RuleConstant.DEGRADE_GRADE_EXCEPTION_COUNT); //熔断规则
    rule.setResource("queryGoodsInfo");
    rule.setCount(5);
}
```

```

rule.setTimewindow(5);
degradeRules.add(rule);
DegradeRuleManager.loadRules(degradeRules);
}

```

具体源码，请参见疯狂创客圈crazy-springcloud 源码工程

平均响应时间 (DEGRADE_GRADE_RT) 熔断

```

// 初始化降级规则
DegradeRule degradeRule = new DegradeRule();
degradeRule.setResource("queryGoodsInfo");
//响应时间
degradeRule.setGrade(RuleConstant.DEGRADE_GRADE_RT);
//平均响应时间超过100ms
degradeRule.setCount(100);
//熔断打开后，经过多少时间进入半打开
degradeRule.setTimewindow(10);
DegradeRuleManager.loadRules(Collections.singletonList(degradeRule));

```

3.5 控制台降级规则

配置

新增降级规则

资源名

资源名

降级策略

☒ RT
 ☐ 异常比例
 ☐ 异常数

RT

毫秒

时间窗口

降级时间间隔, 单位秒

新增

取消

参数

Field	说明	默认值
resource	资源名，即限流规则的作用对象	
count	阈值	
grade	降级模式，根据 RT 降级还是根据异常比例降级	RT
timeWindow	降级的时间，单位为 s	

3.6 与Hystrix的熔断对比：

Hystrix常用的线程池隔离会造成线程上下切换的overhead比较大；

Hystrix使用的信号量隔离对某个资源调用的并发数进行控制，效果不错，但是无法对慢调用进行自动降级；

Sentinel通过并发线程数的流量控制提供信号量隔离的功能；

此外，Sentinel支持的熔断降级维度更多，可对多种指标进行流控、熔断，且提供了实时监控和控制面板，功能更为强大。

4、Sentinel 流控（限流）

流量控制(Flow Control)，原理是监控应用流量的QPS或并发线程数等指标，当达到指定阈值时对流量进行控制，避免系统被瞬时的流量高峰冲垮，保障应用高可用性。

通过流控规则来指定允许该资源通过的请求次数，例如下面的代码定义了资源 HelloWorld 每秒最多只能通过 20 个请求。

参考的规则定义如下：

```
private static void initFlowRules(){
    List<FlowRule> rules = new ArrayList<>();
    FlowRule rule = new FlowRule();
    rule.setResource("HelloWorld");
    rule.setGrade(RuleConstant.FLOW_GRADE_QPS);
    // Set limit QPS to 20.
    rule.setCount(20);
    rules.add(rule);
    FlowRuleManager.loadRules(rules);
}
```

一条限流规则主要由下面几个因素组成，我们可以组合这些元素来实现不同的限流效果：

- **resource**：资源名，即限流规则的作用对象
- **count**：限流阈值
- **grade**：限流阈值类型（QPS 或并发线程数）

- `limitApp`: 流控针对的调用来源, 若为 `default` 则不区分调用来源
- `strategy`: 调用关系限流策略
- `controlBehavior`: 流量控制效果 (直接拒绝、Warm Up、匀速排队)

基本的参数

资源名: 唯一名称, 默认请求路径

针对来源: Sentinel可以针对调用者进行限流, 填写微服务名, 默认为default(不区分来源)

阈值类型/单机阈值:

1. QPS: 每秒请求数, 当前调用该api的QPS到达阈值的时候进行限流
2. 线程数: 当调用该api的线程数到达阈值的时候, 进行限流

是否集群: 是否为集群

流控的几种 `strategy`:

1. 直接: 当api大达到限流条件时, 直接限流
2. 关联: 当关联的资源到达阈值, 就限流自己
3. 链路: 只记录指定路上的流量, 指定资源从入口资源进来的流量, 如果达到阈值, 就进行限流, api级别的限流

4.1 直接失败模式

使用API进行资源定义

```
/**
 * 限流实现方式一: 抛出异常的方式定义资源
 *
 * @param orderId
 * @return
 */
@ApiOperation(value = "纯代码限流")
@GetMapping("/getOrder")
@ResponseBody
public String getOrder(@RequestParam(value = "orderId", required = false)String
orderId)
{
    Entry entry = null;
    // 资源名
    String resourceName = "getOrder";
    try
    {
        // entry可以理解成入口登记
        entry = SphU.entry(resourceName);
        // 被保护的逻辑, 这里为订单查询接口
        return "正常的业务逻辑 OrderInfo : " + orderId;
    }
}
```

```

    } catch (BlockException blockException)
    {
        // 接口被限流的时候，会进入到这里
        log.warn("---getOrder1接口被限流了---，exception: ", blockException);
        return "接口限流，返回空";
    } finally
    {
        // SphU.entry(xxx) 需要与 entry.exit() 成对出现,否则会导致调用链记录异常
        if (entry != null)
        {
            entry.exit();
        }
    }
}

```

代码限流规则

```

//限流规则 QPS mode,
List<FlowRule> rules = new ArrayList<FlowRule>();
FlowRule rule1 = new FlowRule();
rule1.setResource("getOrder");
// QPS控制在2以内
rule1.setCount(2);
// QPS限流
rule1.setGrade(RuleConstant.FLOW_GRADE_QPS);
rule1.setLimitApp("default");
rules.add(rule1);
FlowRuleManager.loadRules(rules);

```

网页限流规则配置

选择QPS，直接，快速失败，单机阈值为2。

配置

新增流控规则

资源名

资源名

来源应用

default

阈值类型

QPS

线程数

单机阈值

单机阈值

是否集群

☐

高级选项

新增

取消

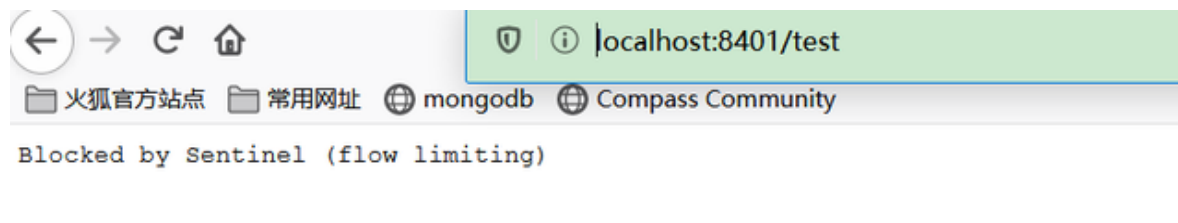
<https://blog.csdn.net/crazymakercircle>

参数

Field	说明	默认值
resource	资源名，资源名是限流规则的作用对象	
count	限流阈值	
grade	限流阈值类型，QPS 或线程数模式	QPS 模式
limitApp	流控针对的调用来源	default，代表不区分调用来源
strategy	判断的根据是资源自身，还是根据其它关联资源 (refResource)，还是根据链路入口	根据资源本身
controlBehavior	流控效果（直接拒绝 / 排队等待 / 慢启动模式）	直接拒绝

测试

频繁刷新请求，1秒访问2次请求，正常，超过设置的阈值，将报默认的错误。



再次的1秒访问2次请求，访问正常。超过2次，访问异常

4.2 关联模式

调用关系包括调用方、被调用方；一个方法又可能会调用其它方法，形成一个调用链路的层次关系。Sentinel 通过 `NodeSelectorSlot` 建立不同资源间的调用的关系，并且通过 `ClusterBuilderSlot` 记录每个资源的实时统计信息。

当两个资源之间具有资源争抢或者依赖关系的时候，这两个资源便具有了关联。

比如对数据库同一个字段的读操作和写操作存在争抢，读的速度过高会影响写得速度，写的速度过高会影响读的速度。如果放任读写操作争抢资源，则争抢本身带来的开销会降低整体的吞吐量。可使用关联限流来避免具有关联关系的资源之间过度的争抢。

举例来说，`read_db` 和 `write_db` 这两个资源分别代表数据库读写，我们可以给 `read_db` 设置限流规则来达到写优先的目的。具体的方法：

```
设置 `strategy` 为 `RuleConstant.STRATEGY_RELATE`  
设置 `refResource` 为 `write_db`。  
这样当写库操作过于频繁时，读数据的请求会被限流。
```

还有一个例子，电商的下订单和支付两个操作，需要优先保障支付，可以根据支付接口的流量阈值，来对订单接口进行限制，从而保护支付的目的。

使用注解进行资源定义

添加2个请求

```
@SentinelResource(value = "test1", blockHandler = "exceptionHandler")  
@GetMapping("/test1")  
public String test1()  
{  
    log.info(Thread.currentThread().getName() + "\t" + "...test1");  
    return "-----hello baby, i am test1";  
}  
  
// Block 异常处理函数，参数最后多一个 BlockException，其余与原函数一致。  
public String exceptionHandler(BlockException ex)  
{  
    // Do some log here.  
    ex.printStackTrace();  
}
```

```

        log.info(Thread.currentThread().getName() + "\t" + "...exceptionHandler");
        return String.format("error: test1 is not OK");
    }

    @SentinelResource(value = "test1_ref")
    @GetMapping("/test1_ref")
    public String test1_ref()
    {
        log.info(Thread.currentThread().getName() + "\t" + "...test1_related");
        return "-----hello baby, i am test1_ref";
    }
}

```

代码配置关联限流规则

```

// 关联模式流控 QPS控制在1以内
String refResource = "test1_ref";
FlowRule rRule = new FlowRule("test1")
    .setCount(1) // QPS控制在1以内
    .setStrategy(RuleConstant.STRATEGY_RELATE)
    .setRefResource(refResource);

rules.add(rRule);
FlowRuleManager.loadRules(rules);

```

网页限流规则配置

编辑流控规则

资源名

test1

针对来源

default

阈值类型

☒ QPS ☐ 线程数

单机阈值

1

是否集群

☐

流控模式

☐ 直接 ☒ 关联 ☐ 链路

关联资源

test1_ref

流控效果

☒ 快速失败 ☐ Warm Up ☐ 排队等待

关闭高级选项

保存

取消

<https://blog.csdn.net/crazymakercircle>

测试

选择QPS，单机阈值为1，选择关联，关联资源为/test_ref，这里用Jmeter模拟高并发，请求/test_ref。

线程属性

线程数

100

Ramp-Up时间(秒)

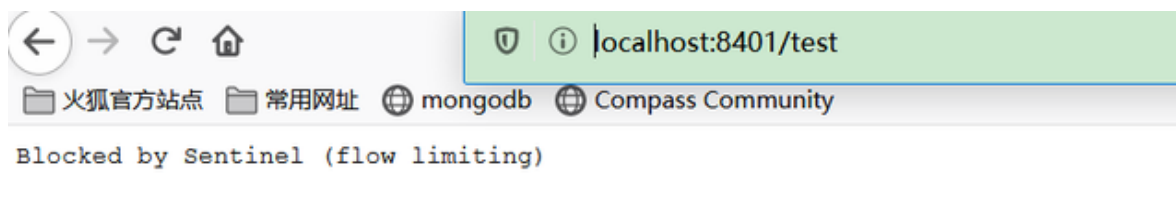
1

循环次数

☐ 永远 ☒ 10

☒ Same user on each iteration

在大批量线程高并发访问/test_ref，导致/test失效了



链路类型的关联也类似，就不再演示了。多个请求调用同一微服务。

4.3 Warm up（预热）模式

场景1：JVM刚启动场景需要预热，具体的原因，去阅读 下面的博客

[微博一面：JVM预热，你的方案是啥？ - 疯狂创客圈 - 博客园 \(cnblogs.com\)](#)

场景2：空闲状态到繁忙状态的切换，也需要时间

- db连接池，最少的连接数
- 线程池，很少线程数
- 其他的池

当流量突然增大的时候，我们常常会希望系统从空闲状态到繁忙状态的切换的时间长一些。

即如果系统在此之前长期处于空闲的状态，我们希望处理请求的数量是缓步的增多，经过预期的时间以后，到达系统处理请求个数的最大值。

如秒杀系统在开启瞬间，会有很多流量上来，很可能把系统打死，预热方式就是为了保护系统，可慢慢的把流量放进来，慢慢的把阈值增长到设置的阈值。

Warm Up（冷启动，预热）模式就是为了实现这个保护系统目的。

默认 coldFactor 为 3，即请求 QPS 从 threshold / 3 开始，经预热时长逐渐升至设定的 QPS 阈值。

使用注解定义资源

```
@SentinelResource(value = "testWarmUP", blockHandler =  
    "exceptionHandlerOfWarmUp")  
@GetMapping("/testWarmUP")  
public String testWarmUP()  
{  
    log.info(Thread.currentThread().getName() + "\t" + "...test1");  
    return "-----hello baby, i am testWarmUP";  
}
```

代码预热规则

案例：阈值为10 qps，预热时长设置为 5s；

```
FlowRule warmUPRule = new FlowRule();
warmUPRule.setResource("testWarmUP");
warmUPRule.setCount(10);
warmUPRule.setGrade(RuleConstant.FLOW_GRADE_QPS);
warmUPRule.setLimitApp("default");
warmUPRule.setControlBehavior(RuleConstant.CONTROL_BEHAVIOR_WARM_UP);
warmUPRule.setWarmUpPeriodSec(5);
```

1) 默认冷加载因子 codeFactor为3，即请求QPS从 (threshold / 3) 开始，经过多少预热时长才逐渐升至设定的 QPS 阈值；

系统初始化 阈值为 10/3 约等于3，即阈值刚开始为3；然后过了5s后，阈值才慢慢升到10；

就是5s前阈值是3，5s后阈值为10；

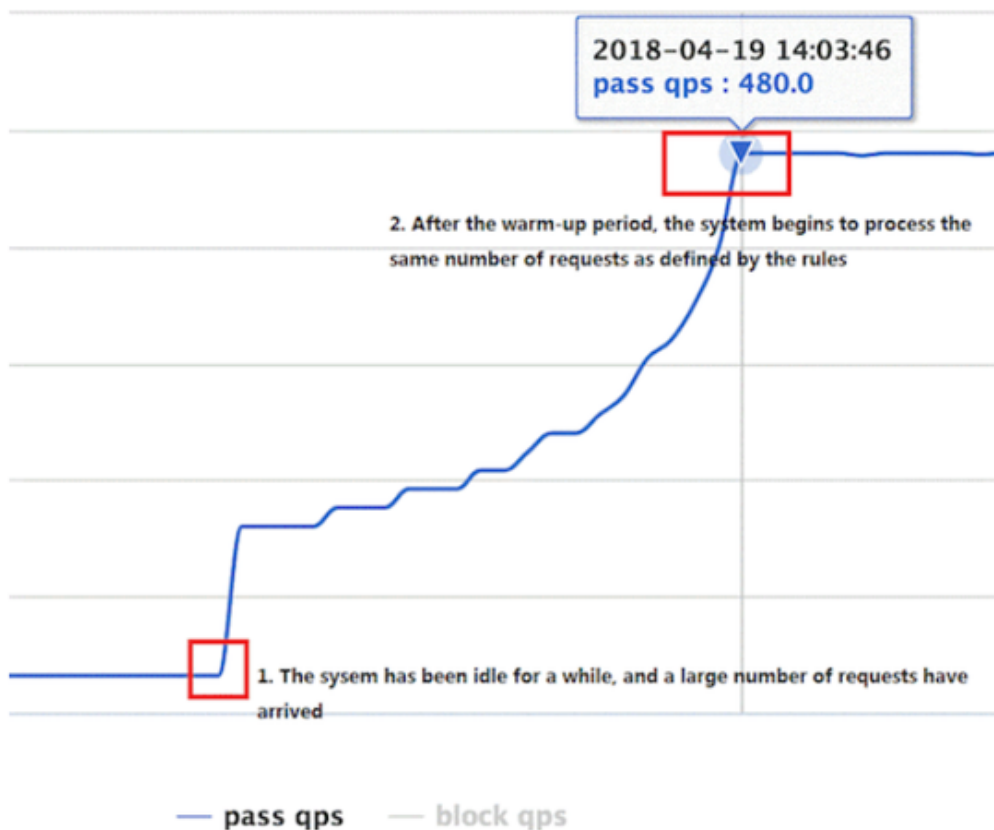
3) testWarmUP 刚开始每秒访问4次，就会报错，超过了 10/3=3，5s后每秒访问 1~10 次都正常；

网页预热规则配置

先在单机阈值10/3，3的时候，预热10秒后，慢慢将阈值升至20。

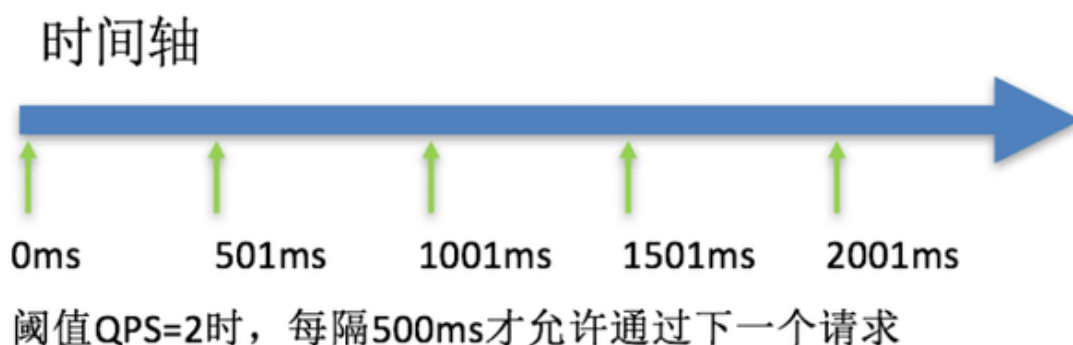
刚开始刷/testWarmUP，会出现默认错误，预热时间到了后，阈值增加，没超过阈值刷新，请求正常。

通常冷启动的过程系统允许通过的 QPS 曲线如下图所示：



4.4 排队等待模式

匀速排队 (RuleConstant.CONTROL_BEHAVIOR_RATE_LIMITER) 方式会严格控制请求通过的间隔时间，也即是让请求以均匀的速度通过，对应的是漏桶算法。阈值必须设置为QPS。



这种方式主要用于处理间隔性突发的流量，例如消息队列。

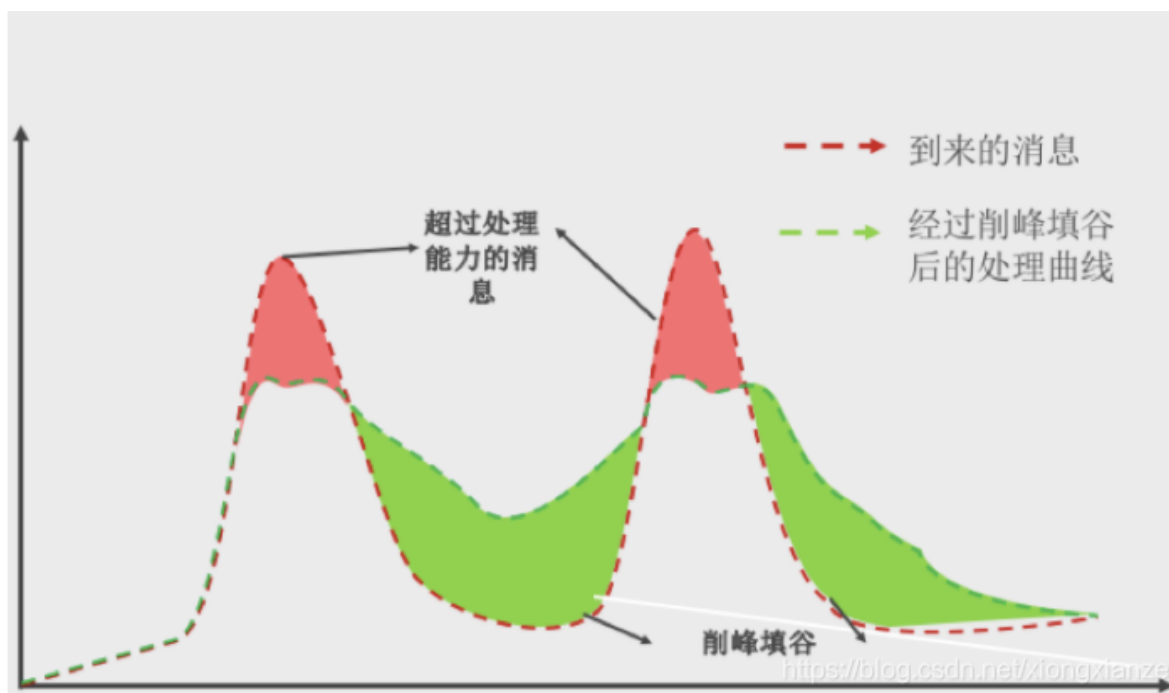
想象一下这样的场景，在某一秒有大量的请求到来，而接下来的几秒则处于空闲状态，我们希望系统能够在接下来的空闲期间逐渐处理这些请求，而不是在第一秒直接拒绝多余的请求。

某瞬时来了大流量的请求, 而如果此时要处理所有请求, 很可能会导致系统负载过高, 影响稳定性。但其实可能后面几秒之内都没有消息投递, 若直接把多余的消息丢掉则没有充分利用系统处理消息的能力。

Sentinel的Rate Limiter模式能在某一段时间间隔内以匀速方式处理这样的请求, 充分利用系统的处理能力, 也就是削峰填谷, 保证资源的稳定性。

Sentinel会以固定的间隔时间让请求通过, 访问资源。当请求到来的时候, 如果当前请求距离上个通过的请求通过的时间间隔不小于预设值, 则让当前请求通过; 否则, 计算当前请求的预期通过时间, 如果该请求的预期通过时间小于规则预设的 timeout 时间, 则该请求会等待直到预设时间到来通过; 反之, 则马上抛出阻塞异常。

使用Sentinel的这种策略, 简单点说, 就是使用一个时间段(比如20s的时间)处理某一瞬时产生的大量请求, 起到一个削峰填谷的作用, 从而充分利用系统的处理能力, 下图能很形象的展示这种场景: X轴代表时间, Y轴代表系统处理的请求。



示例

模拟2个用户同时并发的访问资源, 发出100个请求,

如果设置QPS阈值为1, 拒绝策略修改为Rate Limiter匀速

RuleConstant.CONTROL_BEHAVIOR_RATE_LIMITER方式, 还需要设置setMaxQueueingTimeMs(20 * 1000)表示每一请求最长等待时间, 这里等待时间大一点, 以保证让所有请求都能正常通过;

假设这里设置的排队等待时间过小的话, 导致排队等待的请求超时而抛出异常BlockException, 最终结果可能是这100个并发请求中只有一个请求或几个才能正常通过, 所以使用这种模式得根据访问资源的耗时间决定排队等待时间. 按照目前这种设置, QPS阈值为10的话, 每一个请求相当于是以匀速100ms左右通过。

使用注解定义资源

```

@SentinelResource(value = "testLineUp",
    blockHandler = "exceptionHandlerOftestLineUp")
@GetMapping("/testLineUp")
public String testLineUp()
{
    log.info(Thread.currentThread().getName() + "\t" + "...test1");
    return "-----hello baby, i am testLineUp";
}

```

代码限流规则

```

FlowRule lineUpRule = new FlowRule();
lineUpRule.setResource("testLineUp");
lineUpRule.setCount(10);
lineUpRule.setGrade(RuleConstant.FLOW_GRADE_QPS);
lineUpRule.setLimitApp("default");
lineUpRule.setMaxQueueingTimeMs(20 * 1000);
// CONTROL_BEHAVIOR_DEFAULT means requests more than threshold will be rejected
// immediately.
// CONTROL_BEHAVIOR_DEFAULT将超过阈值的流量立即拒绝掉。
lineUpRule.setControlBehavior(RuleConstant.CONTROL_BEHAVIOR_RATE_LIMITER);
rules.add(lineUpRule);

```

网页限流规则配置

编辑流控规则

资源名

testLineUp

针对来源

default

阈值类型

☒ QPS
 ☐ 线程数

单机阈值

10

是否集群

☐

流控模式

☒ 直接
 ☐ 关联
 ☐ 链路

流控效果

☐ 快速失败
 ☐ Warm Up
 ☒ 排队等待

超时时间

20000

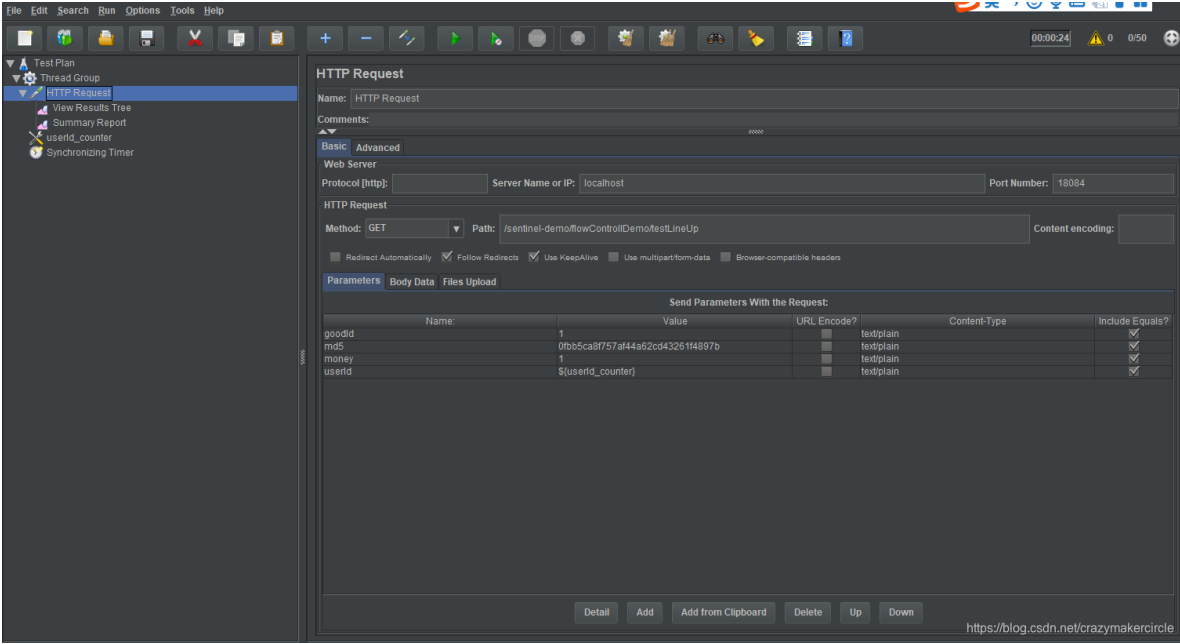
关闭高级选项

保存

取消

<https://blog.csdn.net/crazymakercircle>

通过jmeter进行测试



4.5 热点规则 (ParamFlowRule)

何为热点？

热点即经常访问的数据。很多时候我们希望统计某个热点数据中访问频次最高的 Top K 数据，并对其访问进行限制。比如：

- 商品 ID 为参数，统计一段时间内最常购买的商品 ID 并进行限制
- 用户 ID 为参数，针对一段时间内频繁访问的用户 ID 进行限制 热点参数限流会统计传入参数中的热点参数，并根据配置的限流阈值与模式，对包含热点参数的资源调用进行限流。热点参数限流可以看做是一种特殊的流量控制，仅对包含热点参数的资源调用生效。使用该规则需要引入依赖：

热点参数规则（ParamFlowRule）类似于流量控制规则（FlowRule）：

属性	说明	默认值
resource	资源名，必填	
count	限流阈值，必填	
grade	限流模式	QPS 模式
durationInSec	统计窗口时间长度（单位为秒）， 1.6.0 版本开始支持	1s
controlBehavior	流控效果（支持快速失败和匀速排队模式）， 1.6.0 版本开始支持	快速失败
maxQueueingTimeMs	最大排队等待时长（仅在匀速排队模式生效）， 1.6.0 版本开始支持	0ms
paramIdx	热点参数的索引，必填，对应 <code>sphU.entry(xxx, args)</code> 中的参数索引位置	
	参数例外项，可以针对指定的参数值单独设置限流阈值，	

属性	说明	默认值
ParamFlowItemList	不受前面 count 阈值的限制。仅支持基本类型和字符串类型	
clusterMode	是否是集群参数流控规则	false
clusterConfig	集群流控相关配置	

自定义资源

```
@GetMapping("/byHotKey")
@SentinelResource(value = "byHotKey",
    blockHandler = "userAccessError")
public String test4(@RequestParam(value = "userId", required = false) String
userId,
    @RequestParam(value = "goodId", required = false) int
goodId)
{
    log.info(Thread.currentThread().getName() + "\t" + "...byHotKey");
    return "-----by HotKey:  UserId";
}
```

限流规则代码：

可以通过 ParamFlowRuleManager 的 loadRules 方法更新热点参数规则，下面是官方实例：

```
ParamFlowRule rule = new ParamFlowRule(resourceName)
    .setParamIdx(0)
    .setCount(5);
// 针对 int 类型的参数 PARAM_B，单独设置限流 QPS 阈值为 10，而不是全局的阈值 5.
ParamFlowItem item = new ParamFlowItem().setObject(String.valueOf(PARAM_B))
    .setClassType(int.class.getName())
    .setCount(10);
rule.setParamFlowItemList(Collections.singletonList(item));

ParamFlowRuleManager.loadRules(Collections.singletonList(rule));
```

具体的限流代码如下：

```
ParamFlowRule pRule = new ParamFlowRule("byHotKey")
    .setParamIdx(1)
    .setCount(1);
// 针对 参数值1000，单独设置限流 QPS 阈值为 5，而不是全局的阈值 1.
ParamFlowItem item = new ParamFlowItem().setObject(String.valueOf(1000))
    .setClassType(int.class.getName())
    .setCount(5);
pRule.setParamFlowItemList(Collections.singletonList(item));

ParamFlowRuleManager.loadRules(Collections.singletonList(pRule));
```

编辑热点规则

资源名

byHotKey

限流模式

QPS 模式

参数索引

1

单机阈值

1

统计窗口时长

1

秒

是否集群

☐

参数例外项

参数类型

参数值

例外项参数值

限流阈值

限流阈值

+ 添加

参数值	参数类型	限流阈值	操作
1000	int	5	删除

关闭高级选项

https://blog.csdn.net/crazy_m/article

保存

取消

5、Sentinel 系统保护

系统保护的目

在开始之前，我们先了解一下系统保护的目

- 保证系统不被拖垮
- 在系统稳定的前提下，保持系统的吞吐量

长期以来，系统保护的思路是根据硬指标，即系统的负载 (load1) 来做系统过载保护。当系统负载高于某个阈值，就禁止或者减少流量的进入；当 load 开始好转，则恢复流量的进入。这个思路给我们带来了不可避免的两个问题：

- load 是一个“结果”，如果根据 load 的情况来调节流量的通过率，那么就始终有延迟性。也就意味着通过率的任何调整，都会过一段时间才能看到效果。当前通过率是使 load 恶化的一个动作，那么也至少要过 1 秒之后才能观测到；同理，如果当前通过率调整是让 load 好转的一个动作，也需要 1 秒之后才能继续调整，这样就浪费了系统的处理能力。所以我们看到的曲线，总是会有抖动。
- 恢复慢。想象一下这样的一个场景（真实），出现了这样一个问题，下游应用不可靠，导致应用 RT 很高，从而 load 到了一个很高的点。过了一段时间之后下游应用恢复了，应用 RT 也相应减

少。这个时候，其实应该大幅度增大流量的通过率；但是由于这个时候 load 仍然很高，通过率的恢复仍然不高。

系统保护的目的是 **在系统不被拖垮的情况下，提高系统的吞吐率，而不是 load 一定要低于某个阈值**。如果我们还是按照固有的思维，超过特定的 load 就禁止流量进入，系统 load 恢复就放开流量，这样做的结果是无论我们怎么调参数，调比例，都是按照果来调节因，都无法取得良好的效果。

Sentinel 在系统自适应保护的做法是，用 load1 作为启动自适应保护的因子，而允许通过的流量由处理请求的能力，即请求的响应时间以及当前系统正在处理的请求速率来决定。

系统保护规则的应用

系统规则支持以下的模式：

- **Load 自适应**（仅对 Linux/Unix-like 机器生效）：系统的 load1 作为启发指标，进行自适应系统保护。当系统 load1 超过设定的启发值，且系统当前的并发线程数超过估算的系统容量时才会触发系统保护（BBR 阶段）。系统容量由系统的 `maxQps * minRt` 估算得出。设定参考值一般是 `CPU cores * 2.5`。
- **CPU usage**（1.5.0+ 版本）：当系统 CPU 使用率超过阈值即触发系统保护（取值范围 0.0-1.0），比较灵敏。
- **平均 RT**：当单台机器上所有入口流量的平均 RT 达到阈值即触发系统保护，单位是毫秒。
- **并发线程数**：当单台机器上所有入口流量的并发线程数达到阈值即触发系统保护。
- **入口 QPS**：当单台机器上所有入口流量的 QPS 达到阈值即触发系统保护。

系统保护规则是从应用级别的入口流量进行控制，从单台机器的 load、CPU 使用率、平均 RT、入口 QPS 和并发线程数等几个维度监控应用指标，让系统尽可能跑在最大吞吐量的同时保证系统整体的稳定性。

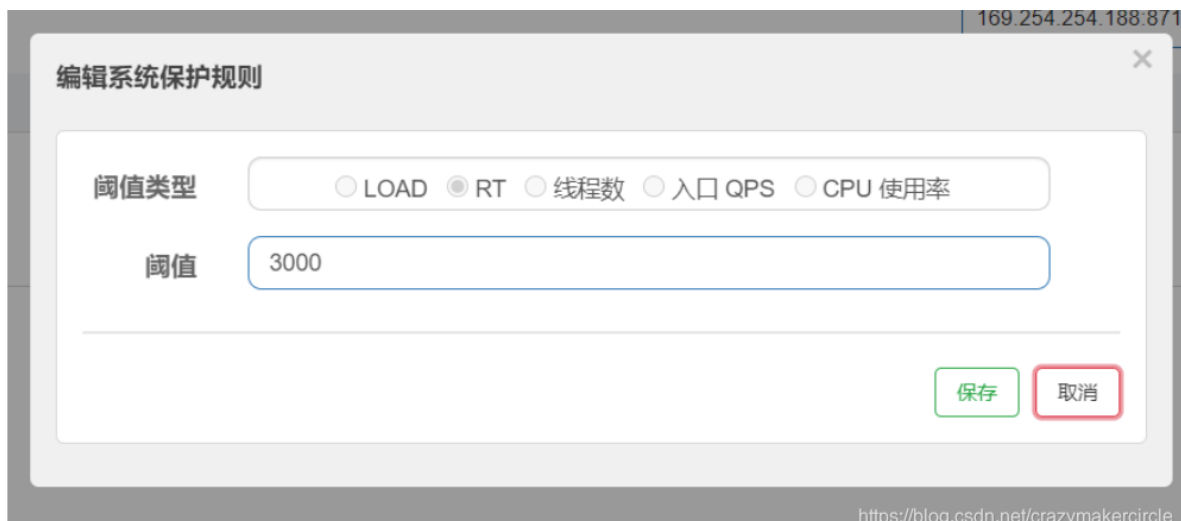
系统保护规则是应用整体维度的，而不是资源维度的，并且**仅对入口流量生效**。入口流量指的是进入应用的流量（`EntryType.IN`），比如 Web 服务或 Dubbo 服务端接收的请求，都属于入口流量。

系统规则的参数说明：

- `highestSystemLoad` 最大的 load1，参考值 -1 (不生效)
- `avgRt` 所有入口流量的平均响应时间 -1 (不生效)
- `maxThread` 入口流量的最大并发数 -1 (不生效)
- `qps` 所有入口资源的 QPS -1 (不生效)

硬编码的方式定义流量控制规则如下：

```
List<SystemRule> srules = new ArrayList<>();
SystemRule srule = new SystemRule();
srule.setAvgRt(3000);
srules.add(srule);
SystemRuleManager.loadRules(srules);
```



6、黑白名单规则

很多时候，我们需要根据调用方来限制资源是否通过，这时候可以使用 Sentinel 的访问控制（黑白名单）的功能。黑白名单根据资源的请求来源（origin）限制资源是否通过，若配置白名单则只有请求来源位于白名单内时才可通过；若配置黑名单则请求来源位于黑名单时不通过，其余的请求通过。

调用方信息通过 `ContextUtil.enter(resourceName, origin)` 方法中的 `origin` 参数传入。

访问控制规则 (AuthorityRule)

授权规则，即黑白名单规则（AuthorityRule）非常简单，主要有以下配置项：

- resource：资源名，即限流规则的作用对象
- limitApp：对应的黑名单/白名单，不同 origin 用, 分隔，如 appA,appB
- strategy：限制模式，AUTHORITY_WHITE 为白名单模式，AUTHORITY_BLACK 为黑名单模式，默认为白名单模式 比如我们希望控制对资源 test 的访问设置白名单，只有来源为 appA 和 appB 的请求才可通过，则可以配置如下白名单规则：

```
AuthorityRule rule = new AuthorityRule();
rule.setResource("test");
rule.setStrategy(RuleConstant.AUTHORITY_WHITE);
rule.setLimitApp("appA,appB");
AuthorityRuleManager.loadRules(Collections.singletonList(rule));
```

7、如何定义资源

Sentinel 可以简单的分为 Sentinel 核心库和 Dashboard。

核心库不依赖 Dashboard，可以通过代码手段定义资源。

我们说的资源，可以是任何东西，服务，服务里的方法，甚至是一段代码。

使用 Sentinel 来进行资源保护，主要分为几个步骤：

1. 定义资源
2. 定义规则
3. 检验规则是否生效

先把可能需要保护的资源定义好（埋点），之后再配置规则。

也可以理解为，只要有了资源，我们就可以在任何时候灵活地定义各种流量控制规则。在编码的时候，只需要考虑这个代码是否需要保护，如果需要保护，就将之定义为一个资源。

对于主流的框架，我们提供适配，只需要按照适配中的说明配置，Sentinel 就会默认定义提供的服务，方法等为资源。

方式一：主流框架的默认适配

为了减少开发的复杂程度，我们对大部分的主流框架，例如 Web Servlet、Dubbo、Spring Cloud、gRPC、Spring WebFlux、Reactor 等都做了适配。您只需要引入对应的依赖即可方便地整合 Sentinel。

可以参见: [主流框架的适配](#)。

方式二：抛出异常的方式定义资源

`SphU` 包含了 try-catch 风格的 API。用这种方式，当资源发生了限流之后会抛出 `BlockException`。这个时候可以捕捉异常，进行限流之后的逻辑处理。示例代码如下：

```
// 1.5.0 版本开始可以利用 try-with-resources 特性（使用有限制）
// 资源名可使用任意有业务语义的字符串，比如方法名、接口名或其它可唯一标识的字符串。
try (Entry entry = SphU.entry("resourceName")) {
    // 被保护的逻辑
    // do something here...
} catch (BlockException ex) {
    // 资源访问阻止，被限流或被降级
    // 在此处进行相应的处理操作
}
```

特别地，若 entry 的时候传入了热点参数，那么 exit 的时候也一定要带上对应的参数（`exit(count, args)`），否则可能会有统计错误。这个时候不能使用 try-with-resources 的方式。

另外通过 `Tracer.trace(ex)` 来统计异常信息时，由于 try-with-resources 语法中 catch 调用顺序的问题，会导致无法正确统计异常数，因此统计异常信息时也不能在 try-with-resources 的 catch 块中调用 `Tracer.trace(ex)`。

手动 exit 示例：

```
Entry entry = null;
// 务必保证 finally 会被执行
try {
    // 资源名可使用任意有业务语义的字符串，注意数目不能太多（超过 1K），超出几千请作为参数传入
    // 而不要直接作为资源名
    // EntryType 代表流量类型（inbound/outbound），其中系统规则只对 IN 类型的埋点生效
    entry = SphU.entry("自定义资源名");
    // 被保护的逻辑
    // do something...
}
```

```

} catch (BlockException ex) {
    // 资源访问阻止，被限流或被降级
    // 进行相应的处理操作
} catch (Exception ex) {
    // 若需要配置降级规则，需要通过这种方式记录业务异常
    Tracer.traceEntry(ex, entry);
} finally {
    // 务必保证 exit，务必保证每个 entry 与 exit 配对
    if (entry != null) {
        entry.exit();
    }
}
}

```

热点参数埋点示例：

```

Entry entry = null;
try {
    // 若需要配置例外项，则传入的参数只支持基本类型。
    // EntryType 代表流量类型，其中系统规则只对 IN 类型的埋点生效
    // count 大多数情况都填 1，代表统计为一次调用。
    entry = SphU.entry(resourceName, EntryType.IN, 1, paramA, paramB);
    // Your logic here.
} catch (BlockException ex) {
    // Handle request rejection.
} finally {
    // 注意：exit 的时候也一定要带上对应的参数，否则可能会有统计错误。
    if (entry != null) {
        entry.exit(1, paramA, paramB);
    }
}
}

```

`SphU.entry()` 的参数描述：

参数名	类型	解释	默认值
entryType	EntryType	资源调用的流量类型，是入口流量（EntryType.IN）还是出口流量（EntryType.OUT），注意系统规则只对 IN 生效	EntryType.OUT
count	int	本次资源调用请求的 token 数目	1
args	Object[]	传入的参数，用于热点参数限流	无

注意：`SphU.entry(xxx)` 需要与 `entry.exit()` 方法成对出现，匹配调用，否则会导致调用链记录异常，抛出 `ErrorEntryFreeException` 异常。常见的错误：

- 自定义埋点只调用 `SphU.entry()`，没有调用 `entry.exit()`
- 顺序错误，比如：`entry1 -> entry2 -> exit1 -> exit2`，应该为 `entry1 -> entry2 -> exit2 -> exit1`

方式三：返回布尔值方式定义资源

`SphO` 提供 if-else 风格的 API。用这种方式，当资源发生了限流之后会返回 `false`，这个时候可以根据返回值，进行限流之后的逻辑处理。示例代码如下：

```
// 资源名可使用任意有业务语义的字符串
if (SphO.entry("自定义资源名")) {
    // 务必保证finally会被执行
    try {
        /**
         * 被保护的逻辑
         */
    } finally {
        spho.exit();
    }
} else {
    // 资源访问阻止，被限流或被降级
    // 进行相应的处理操作
}
```

注意： `spho.entry(xxx)` 需要与 `SphO.exit()` 方法成对出现，匹配调用，位置正确，否则会导致调用链记录异常，抛出 `ErrorEntryFreeException` 异常。

方式四：注解方式定义资源

Sentinel 支持通过 `@SentinelResource` 注解定义资源并配置 `blockHandler` 和 `fallback` 函数来进行限流之后的处理。示例：

```
// 原本的业务方法。
@SentinelResource(blockHandler = "blockHandlerForGetUser")
public User getUserById(String id) {
    throw new RuntimeException("getUserById command failed");
}

// blockHandler 函数，原方法调用被限流/降级/系统保护的时候调用
public User blockHandlerForGetUser(String id, BlockException ex) {
    return new User("admin");
}
```

注意 `blockHandler` 函数会在原方法被限流/降级/系统保护的时候调用，而 `fallback` 函数会针对所有类型的异常。请注意 `blockHandler` 和 `fallback` 函数的形式要求，更多指引可以参见 [Sentinel 注解支持文档](#)。

方式五：异步调用支持

Sentinel 支持异步调用链路的统计。在异步调用中，需要通过 `SphU.asyncEntry(xxx)` 方法定义资源，并通常需要在异步的回调函数中调用 `exit` 方法。以下是一个简单的示例：

```
try {
    AsyncEntry entry = SphU.asyncEntry(resourceName);

    // 异步调用。
    doAsync(userId, result -> {
        try {
            // 在此处处理异步调用的结果。
        } finally {
            // 在回调结束后 exit。
            entry.exit();
        }
    });
}
```

```

} catch (BlockException ex) {
    // Request blocked.
    // Handle the exception (e.g. retry or fallback).
}

```

`SphU.asyncEntry(xxx)` 不会影响当前（调用线程）的 Context，因此以下两个 entry 在调用链上是平级关系（处于同一层），而不是嵌套关系：

```

// 调用链类似于：
// -parent
// ---asyncResource
// ---syncResource
asyncEntry = SphU.asyncEntry(asyncResource);
entry = SphU.entry(normalResource);

```

若在异步回调中需要嵌套其它的资源调用（无论是 `entry` 还是 `asyncEntry`），只需要借助 Sentinel 提供的上下文切换功能，在对应的地方通过 `ContextUtil.runOnContext(context, f)` 进行 Context 变换，将对应资源调用处的 Context 切换为生成的异步 Context，即可维持正确的调用链路关系。示例如下：

```

public void handleResult(String result) {
    Entry entry = null;
    try {
        entry = SphU.entry("handleResultForAsync");
        // Handle your result here.
    } catch (BlockException ex) {
        // Blocked for the result handler.
    } finally {
        if (entry != null) {
            entry.exit();
        }
    }
}

public void someAsync() {
    try {
        AsyncEntry entry = SphU.asyncEntry(resourceName);

        // Asynchronous invocation.
        doAsync(userId, result -> {
            // 在异步回调中进行上下文变换，通过 AsyncEntry 的 getAsyncContext 方法获取
            异步 Context
            ContextUtil.runOnContext(entry.getAsyncContext(), () -> {
                try {
                    // 此处嵌套正常的资源调用。
                    handleResult(result);
                } finally {
                    entry.exit();
                }
            });
        });
    } catch (BlockException ex) {
        // Request blocked.
        // Handle the exception (e.g. retry or fallback).
    }
}

```

此时的调用链就类似于：

```
-parent
---asyncInvocation
-----handleResultForAsync
```

更详细的示例可以参考 Demo 中的 [AsyncEntryDemo](#)，里面包含了普通资源与异步资源之间的各种嵌套示例。

8、核心组件 源码分析

Resource

resource是sentinel中最重要的一个概念，sentinel通过资源来保护具体的业务代码或其他后方服务。sentinel把复杂的逻辑给屏蔽掉了，用户只需要为受保护的代码或服务定义一个资源，然后定义规则就可以了，剩下的通通交给sentinel来处理了。并且资源和规则是解耦的，规则甚至可以在运行时动态修改。

定义完资源后，就可以通过在程序中埋点来保护你自己的服务了，埋点的方式有两种：

- try-catch 方式（通过 `sphu.entry(...)`），当 catch 到 `BlockException` 时执行异常处理(或 fallback)
- if-else 方式（通过 `spho.entry(...)`），当返回 `false` 时执行异常处理(或 fallback)

以上这两种方式都是通过硬编码的形式定义资源然后进行资源埋点的，对业务代码的侵入太大，从0.1.1版本开始，sentinel加入了注解的支持，可以通过注解来定义资源，具体的注解为： `SentinelResource`。

也可以使用Sentinel提供的注解 `@SentinelResource` 来定义资源，实例如下：

```
@SentinelResource("HelloWorld")
public void helloworld() {
    // 资源中的逻辑
    System.out.println("hello world");
}
```

通过注解除了可以定义资源外，还可以指定 `blockHandler` 和 `fallback` 方法。

在源码中，在sentinel中具体表示资源的类是： `ResourceWrapper`，他是一个抽象的包装类，包装了资源的 `Name` 和 `EntryType`。

他有两个实现类，分别是： `StringResourceWrapper` 和 `MethodResourceWrapper`。

顾名思义， `StringResourceWrapper` 是通过对一串字符串进行包装，是一个通用的资源包装类， `MethodResourceWrapper` 是对方法调用的包装。

Context

Context是对资源操作时的上下文环境，每个资源操作(针对Resource进行的entry/exit)必须属于一个Context，如果程序中未指定Context，会创建name为"sentinel_default_context"的默认Context。

一个Context生命周期内可能有多个资源操作，Context生命周期内的最后一个资源exit时会清理该Context，这也预示这整个Context生命周期的结束。

Context主要属性如下：

```
public class Context {
    // context名字，默认名字 "sentinel_default_context"
    private final String name;
    // context入口节点，每个context必须有一个entranceNode
    private DefaultNode entranceNode;
    // context当前entry，Context生命周期中可能有多个Entry，所有curEntry会有变化
    private Entry curEntry;
    // The origin of this context (usually indicate different invokers, e.g.
    // service consumer name or origin IP).
    private String origin = "";
    private final boolean async;
}
```

注意：一个Context生命期内Context只能初始化一次，因为是存到ThreadLocal中，并且只有在非null时才会进行初始化。

如果想在调用 SphU.entry() 或 SphO.entry() 前，自定义一个context，则通过ContextUtil.enter()方法来创建。

context是保存在ThreadLocal中的，每次执行的时候会优先到ThreadLocal中获取，为null时会调用MyContextUtil.myEnter(Constants.CONTEXT_DEFAULT_NAME, "", resourceWrapper.getType()) 创建一个context。

当Entry执行exit方法时，如果entry的parent节点为null，表示是当前Context中最外层的Entry了，此时将ThreadLocal中的context清空。

Context的创建与销毁

首先我们要清楚的一点就是，每次执行entry()方法，试图冲破一个资源时，都会生成一个上下文。这个上下文中会保存着调用链的根节点和当前的入口。

Context是通过ContextUtil创建的，具体的方法是trueEntry，代码如下：

```
protected static Context trueEnter(String name, String origin) {
    // 先从ThreadLocal中获取
    Context context = contextHolder.get();
    if (context == null) {
        // 如果ThreadLocal中获取不到Context
        // 则根据name从map中获取根节点，只要是相同的资源名，就能直接从map中获取到node
        Map<String, DefaultNode> localCacheNameMap = contextNameNodeMap;
        DefaultNode node = localCacheNameMap.get(name);
        if (node == null) {
            // 省略部分代码
            try {
```

```

        LOCK.lock();
        node = contextNameNodeMap.get(name);
        if (node == null) {
            // 省略部分代码
            // 创建一个新的入口节点
            node = new EntranceNode(new StringResourceWrapper(name,
EntryType.IN), null);
            Constants.ROOT.addChild(node);
            // 省略部分代码
        }
    } finally {
        LOCK.unlock();
    }
}
// 创建一个新的Context，并设置Context的根节点，即设置EntranceNode
context = new Context(node, name);
context.setOrigin(origin);
// 将该Context保存到ThreadLocal中去
contextHolder.set(context);
}
return context;
}

```

上面的代码中我省略了部分代码，只保留了核心的部分。从源码中还是可以比较清晰的看出生成Context的过程：

- 1.先从ThreadLocal中获取，如果能获取到直接返回，如果获取不到则继续第2步
- 2.从一个static的map中根据上下文的名称获取，如果能获取到则直接返回，否则继续第3步
- 3.加锁后进行一次double check，如果还是没能从map中获取到，则创建一个EntranceNode，并把该EntranceNode添加到一个全局的ROOT节点中去，然后将该节点添加到map中去(这部分代码在上述代码中省略了)
- 4.根据EntranceNode创建一个上下文，并将该上下文保存到ThreadLocal中去，下一个请求可以直接获取

那保存在ThreadLocal中的上下文什么时候会清除呢？从代码中可以看到具体的清除工作在ContextUtil的exit方法中，当执行该方法时，会将保存在ThreadLocal中的context对象清除，具体的代码非常简单，这里就不贴代码了。

那ContextUtil.exit方法什么时候会被调用呢？有两种情况：一是主动调用ContextUtil.exit的时候，二是当一个入口Entry要退出，执行该Entry的trueExit方法的时候，此时会触发ContextUtil.exit的方法。但是有一个前提，就是当前Entry的父Entry为null时，此时说明该Entry已经是最顶层的根节点了，可以清除context。

Entry

刚才在Context身影中也看到了Entry的出现，现在就谈谈Entry。

每次执行 SphU.entry() 或 SphO.entry() 都会返回一个Entry，Entry表示一次资源操作，内部会保存当前invocation信息。

在一个Context生命周期中多次资源操作，也就是对应多个Entry，这些Entry形成parent/child结构保存在Entry实例中，entry类CtEntry结构如下：

```

class CtEntry extends Entry {
    protected Entry parent = null;
    protected Entry child = null;
}

```

```

protected ProcessorSlot<Object> chain;
protected Context context;
}
public abstract class Entry implements AutoCloseable {
    private long createTime;
    private Node curNode;
    /**
     * {@link Node} of the specific origin, Usually the origin is the Service
     * Consumer.
     */
    private Node originNode;
    private Throwable error; // 是否出现异常
    protected ResourceWrapper resourceWrapper; // 资源信息
}

```

Entry实例代码中出现了Node，这个又是什么东东呢，接着往下看：

DefaultNode

Node（关于StatisticNode的讨论放到下一小节）默认实现类DefaultNode，该类还有一个子类EntranceNode；context有一个entranceNode属性，Entry中有一个curNode属性。

- **EntranceNode**：该类的创建是在初始化Context时完成的（ContextUtil.trueEnter方法），注意该类是针对Context维度的，也就是一个context有且仅有一个EntranceNode。
- **DefaultNode**：该类的创建是在NodeSelectorSlot.entry完成的，当不存在context.name对应的DefaultNode时会新建（new DefaultNode(resourceWrapper, null)，对应resource）并保存到本地缓存（NodeSelectorSlot中private volatile Map<String, DefaultNode> map）；获取到context.name对应的DefaultNode后会将该DefaultNode设置到当前context的curEntry.curNode属性，也就是说，在NodeSelectorSlot中是一个context有且仅有一个DefaultNode。

看到这里，你是不是有疑问？

为什么一个context有且仅有一个DefaultNode，我们的resource跑哪去了呢？

其实，这里的一个context有且仅有一个DefaultNode是在NodeSelectorSlot范围内，NodeSelectorSlot是ProcessorSlotChain中的一环，获取ProcessorSlotChain是根据Resource维度来的。

总结为一句话就是：

- 针对同一个Resource，多个context对应多个DefaultNode；
- 针对不同Resource，(不管是否是同一个context)对应多个不同DefaultNode。

```

public class DefaultNode extends StatisticNode {
    private ResourceWrapper id;
    /**
     * The list of all child nodes.
     * 子节点集合
     */
    private volatile Set<Node> childList = new HashSet<>();
    /**
     * Associated cluster node.
     */
    private ClusterNode clusterNode;
}

```

一个Resource只有一个clusterNode，多个defaultNode对应一个clusterNode，
如果defaultNode.clusterNode为null，则在ClusterBuilderSlot.entry中会进行初始化。

同一个Resource，对应同一个ProcessorSlotChain，这块处理逻辑在lookProcessChain方法中，如下：

```

ProcessorSlot<Object> lookProcessChain(ResourceWrapper resourceWrapper) {
    ProcessorSlotChain chain = chainMap.get(resourceWrapper);
    if (chain == null) {
        synchronized (LOCK) {
            chain = chainMap.get(resourceWrapper);
            if (chain == null) {
                // Entry size limit.
                if (chainMap.size() >= Constants.MAX_SLOT_CHAIN_SIZE) {
                    return null;
                }

                chain = slotChainProvider.newSlotChain();
                Map<ResourceWrapper, ProcessorSlotChain> newMap =
                newHashMap<ResourceWrapper, ProcessorSlotChain>(
                    chainMap.size() + 1);
                newMap.putAll(chainMap);
                newMap.put(resourceWrapper, chain);
                chainMap = newMap;
            }
        }
    }
    return chain;
}

```

StatisticNode

StatisticNode中保存了资源的实时统计数据（基于滑动时间窗口机制），通过这些统计数据，sentinel才能进行限流、降级等一系列操作。

StatisticNode属性如下：

```

public class StatisticNode implements Node {
    /**

```

```

    * 秒级的滑动时间窗口（时间窗口单位500ms）
    */
    private transient volatile Metric rollingCounterInSecond =
newArrayMetric(SampleCountProperty.SAMPLE_COUNT,

IntervalProperty.INTERVAL);
/**
    * 分钟级的滑动时间窗口（时间窗口单位1s）
    */
    private transient Metric rollingCounterInMinute = new ArrayMetric(60, 60 *
1000,false);
/**
    * The counter for thread count.
    * 线程个数用户触发线程数流控
    */
    private LongAdder curThreadNum = new LongAdder();
}
public class ArrayMetric implements Metric {
    private final LeapArray<MetricBucket> data;
}
public class MetricBucket {
    // 保存统计值
    private final LongAdder[] counters;
    // 最小rt
    private volatile long minRt;
}

```

其中MetricBucket.counters数组大小为MetricEvent枚举值的个数，每个枚举对应一个统计项，比如PASS表示通过个数，限流可根据通过的个数和设置的限流规则配置count大小比较，得出是否触发限流操作，所有枚举值如下：

```

public enum MetricEvent {
    PASS, // Normal pass.
    BLOCK, // Normal block.
    EXCEPTION,
    SUCCESS,
    RT,
    OCCUPIED_PASS
}

```

9、插槽Slot 源码分析

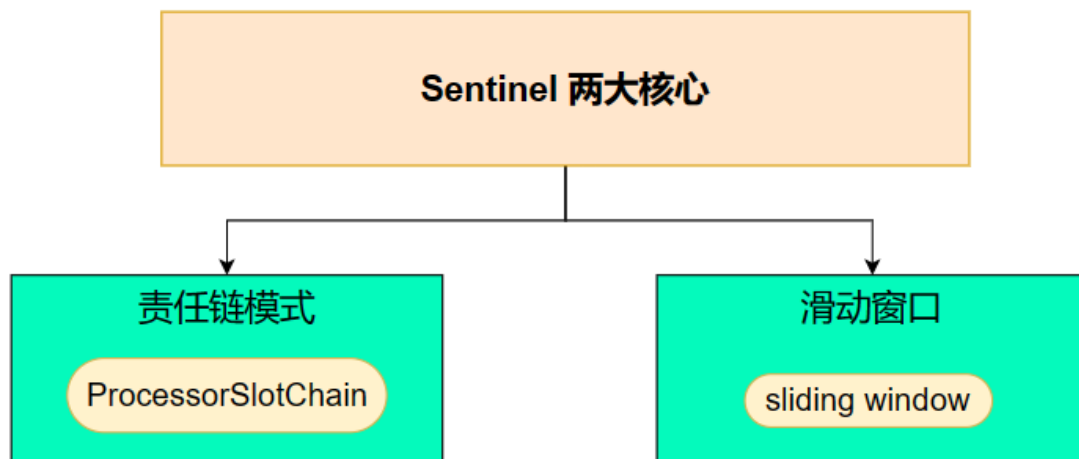
slot是另一个sentinel中非常重要的概念，sentinel的工作流程就是围绕着一个个插槽所组成的插槽链来展开的。

需要注意的是每个插槽都有自己的职责，他们各司其职完美的配合，通过一定的编排顺序，来达到最终的限流降级的目的。

默认的各个插槽之间的顺序是固定的，因为有的插槽需要依赖其他的插槽计算出来的结果才能进行工作。

Sentinel中的责任链模式

Sentinel的整体工具流程就是使用责任链模式将所有的 ProcessorSlot 按照一定的顺序串成一个单向链表。实现将 ProcessorSlot 串成一个单向链表的是 ProcessorSlotChain，这个 ProcessorSlotChain 是由 SlotChainBuilder 构造的。

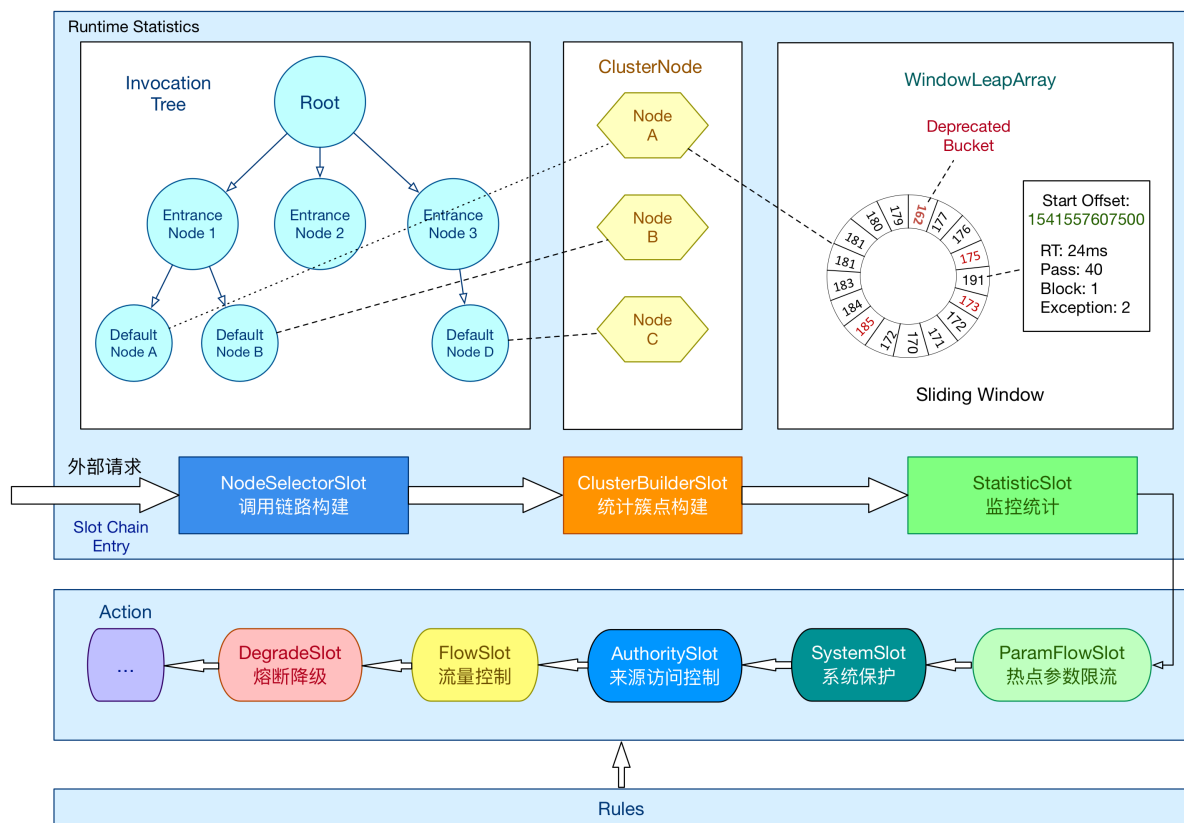


这个和Netty类似，Netty也是一个责任链，在责任链上的每一个处理器，叫做handler。

entinel中的责任链模式，在责任链上的每一个处理器，叫做Slot。

换句话说，Sentinel在内部创建了一个责任链，责任链是由一系列ProcessorSlot接口的实现类组成的，每个ProcessorSlot对象负责不同的功能，外部请求想要访问资源需要责任链层层校验和处理。

每个Slot 需要执行（例如配置过降级规则DegradeSlot）则处理，不需要执行则交给下一个Slot。每一个Slot如果处理失败或者/如果校验失败，会抛出BlockException异常。



责任链上的处理或者/校验顺序，大致如下：

- 降级
- 黑白名单
- 构建ClusterNode对象（统计QPS,RT等）
- 校验QPS
- RT等
- 流控
- 打印日志

ProcessorSlot接口：

是一个基于责任链模式的接口，定义了一个entry()方法，用于处理入口参数和出口参数的限流和降级逻辑；一个exit()方法，用于将权限交给下一个抽象处理人（实际会传参具体处理人）。

ProcessorSlot实现类，大致如下：

- **DegradeSlot**：用于服务降级。如果发现服务超时次数或者报错次数超过限制，DegradeSlot将禁止再次访问服务，等待一段时间后，DegradeSlot试探性的放过一个请求，然后根据该请求的处理情况，决定是否再次降级。
- **AuthoritySlot**：黑白名单校验，按照字符串匹配，如果在黑名单，则禁止访问。
- **ClusterBuilderSlot**：构建ClusterNode对象，该对象用于统计访问资源的QPS、线程数、异常、响应时间等，每个资源对应一个ClusterNode对象。
- **SystemSlot**：校验QPS、并发线程数、系统负载、CPU使用率、平均响应时间是否超过限制，使用滑动窗口算法统计上述这些数据。
- **StatisticSlot**：用于从多个维度（入口流量、调用者、当前被访问资源）统计响应时间、并发线程数、处理失败个数、处理成功个数等。
- **FlowSlot**：用于流控，可以根据QPS或者每秒并发线程数控制，当QPS或者并发线程数超过设定值，便会抛出FlowException异常。FlowSlot依赖于StatisticSlot的统计数据。
- **NodeSelectorSlot**：负责收集资源路径，并将这些资源的调用路径，以树状结构存储起来，用于根据调用路径来限流降级、数据统计。
- **LogSlot**：打印日志。

责任链SlotChain 如何创建？

但是这并不意味着我们只能按照框架的定义来，sentinel 通过 SlotChainBuilder 作为 SPI 接口，使得 Slot Chain 具备了扩展的能力。

我们可以通过实现 SlotsChainBuilder 接口加入自定义的 slot 并自定义编排各个 slot 之间的顺序，从而可以给 sentinel 添加自定义的功能。

那SlotChain是在哪创建的呢？

是在 CtSph.lookupProcessChain() 方法中创建的，并且该方法会根据当前请求的资源先去一个静态的 HashMap 中获取，如果获取不到才会创建，创建后会保存到HashMap中。

这就意味着，同一个资源会全局共享一个SlotChain。默认生成ProcessorSlotChain为：

```
// DefaultSlotChainBuilder
public ProcessorSlotChain build() {
    ProcessorSlotChain chain = new DefaultProcessorsSlotChain();
    chain.addLast(new NodeSelectorsSlot());
    chain.addLast(new ClusterBuildersSlot());
    chain.addLast(new LogSlot());
    chain.addLast(new StatisticsSlot());
    chain.addLast(new SystemSlot());
    chain.addLast(new AuthoritySlot());
    chain.addLast(new FlowSlot());
    chain.addLast(new DegradesSlot());

    return chain;
}
```

这里大概的介绍下每种Slot的功能职责：

- `NodeSelectorsSlot` 负责收集资源的路径，并将这些资源的调用路径，以树状结构存储起来，用于根据调用路径来限流降级；
- `ClusterBuildersSlot` 则用于存储资源的统计信息以及调用者信息，例如该资源的 RT, QPS, thread count 等等，这些信息将用作为多维度限流，降级的依据；
- `StatisticsSlot` 则用于记录，统计不同维度的 runtime 信息；
- `SystemSlot` 则通过系统的状态，例如 load1 等，来控制总的入口流量；
- `AuthoritySlot` 则根据黑白名单，来做黑白名单控制；
- `FlowSlot` 则用于根据预设的限流规则，以及前面 slot 统计的状态，来进行限流；
- `DegradesSlot` 则通过统计信息，以及预设的规则，来做熔断降级；

每个Slot执行完业务逻辑处理后，会调用fireEntry()方法，该方法将会触发下一个节点的entry方法，下一个节点又会调用他的fireEntry，以此类推直到最后一个Slot，由此就形成了sentinel的责任链。

下面我们就来详细研究下这些Slot的原理。

责任链模式的重要性

在责任链模式中，客户只需要将请求发送到责任链上即可，无须关心请求的处理细节和请求的传递过程，请求会自动进行传递。所以责任链将请求的发送者和请求的处理者解耦了。

责任链模式是一种对象行为型模式，其主要**优点**如下。

- 降低了对象之间的耦合度。该模式使得一个对象无须知道到底是哪一个对象处理其请求以及链的结构，发送者和接收者也无须拥有对方的明确信息。

- 增强了系统的可扩展性。可以根据需要增加新的请求处理类，满足开闭原则。
- 增强了给对象指派职责的灵活性。当工作流程发生变化，可以动态地改变链内的成员或者调动它们的次序，也可动态地新增或者删除责任。
- 责任链简化了对象之间的连接。每个对象只需保持一个指向其后继者的引用，不需保持其他所有处理者的引用，这避免了使用众多的 if 或者 if...else 语句。
- 责任分担。每个类只需要处理自己该处理的工作，不该处理的传递给下一个对象完成，明确各类的责任范围，符合类的单一职责原则。

其主要缺点如下。

- 不能保证每个请求一定被处理。由于一个请求没有明确的接收者，所以不能保证它一定会被处理，该请求可能一直传到链的末端都得不到处理。
- 对比较长的职责链，请求的处理可能涉及多个处理对象，系统性能将受到一定影响。
- 职责链建立的合理性要靠客户端来保证，增加了客户端的复杂性，可能会由于职责链的错误设置而导致系统出错，如可能会造成循环调用。

责任链模式的重要性

- Netty 中用了责任链模式（具体参见尼恩的视频）
- Spring 核心源码中，大量用了责任链模式（具体参见尼恩的视频）
- Sentinel 中用了责任链模式

NodeSelectorSlot 原理分析+ 源码分析

`NodeSelectorSlot` 是用来构造调用链的，具体的是将资源的调用路径，封装成一个一个的节点，再组成一个树状的结构来形成一个完整的调用链，`NodeSelectorSlot` 是所有 Slot 中最关键也是最复杂的一个 Slot，

`NodeSelectorSlot` 涉及到以下几个核心的概念：

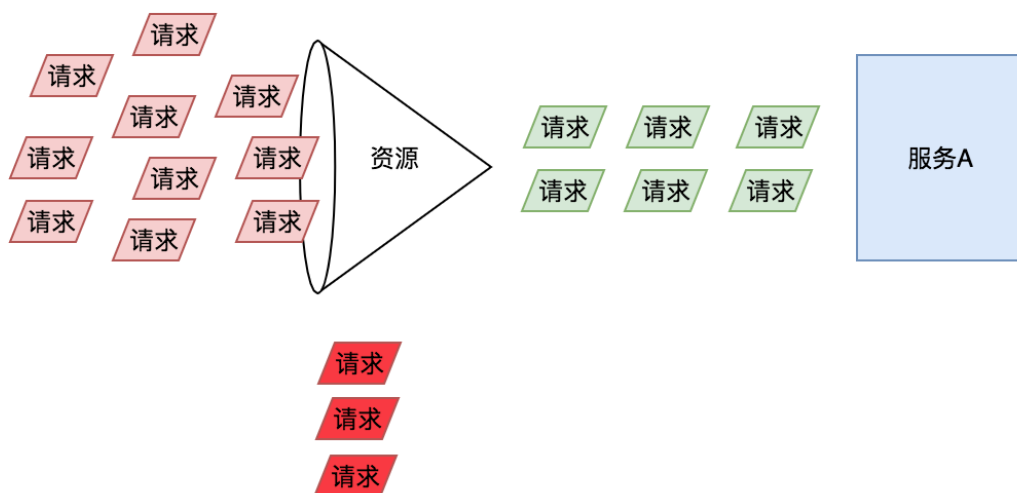
核心的概念1: Resource

资源是 Sentinel 的关键概念。它可以是 Java 应用程序中的任何内容，例如，由应用程序提供的服务，或由应用程序调用的其它服务，甚至可以是一段代码。

只要通过 Sentinel API 定义的代码，就是资源，能够被 Sentinel 保护起来。大部分情况下，可以使用方法签名，URL，甚至服务名称作为资源名来标示资源。

打个比方，我有一个服务A，请求非常多，经常会被陡增的流量冲垮，

为了防止这种情况，简单的做法，我们可以定义一个 Sentinel 的资源，通过该资源来对请求进行调整，使得允许通过的请求不会把服务A搞崩溃。



每个资源的状态也是不同的，这取决于资源后端的服务，有的资源可能比较稳定，有的资源可能不太稳定。

那么在整个调用链中，Sentinel 需要对不稳定资源进行控制。

当调用链路中某个资源出现不稳定，例如表现为 timeout，或者异常比例升高的时候，则对这个资源的调用进行限制，并让请求快速失败，避免影响到其它的资源，最终导致雪崩的后果。

Sentinel 中，资源就是 用来保护系统的一个媒介。

源码中用来包装资源的类是：`com.alibaba.csp.sentinel.slotchain.ResourceWrapper`，

他有两个子类：

- `StringResourceWrapper`
- 和 `MethodResourceWrapper`，

通过名字就知道可以将一段字符串或一个方法包装为一个资源。

核心的概念2: Context

上下文是一个用来保存调用链当前状态的元数据的类，每次进入一个资源时，就会创建一个上下文。

相同的资源名可能会创建多个上下文。

一个Context中包含了三个核心的对象：

- 1) 当前调用链的根节点：EntranceNode
- 2) 当前的入口：Entry
- 3) 当前入口所关联的节点：Node

上下文中只会保存一个当前正在处理的入口Entry，另外还会保存调用链的根节点。

需要注意的是，每次进入一个新的资源时，都会创建一个新的上下文。

核心的概念3: Entry

每次调用 `sphU#entry()` 都会生成一个Entry入口，该入口中会保存了以下数据：

- 入口的创建时间，

- 当前入口所关联的节点，
- 当前入口所关联的调用源对应的节点。

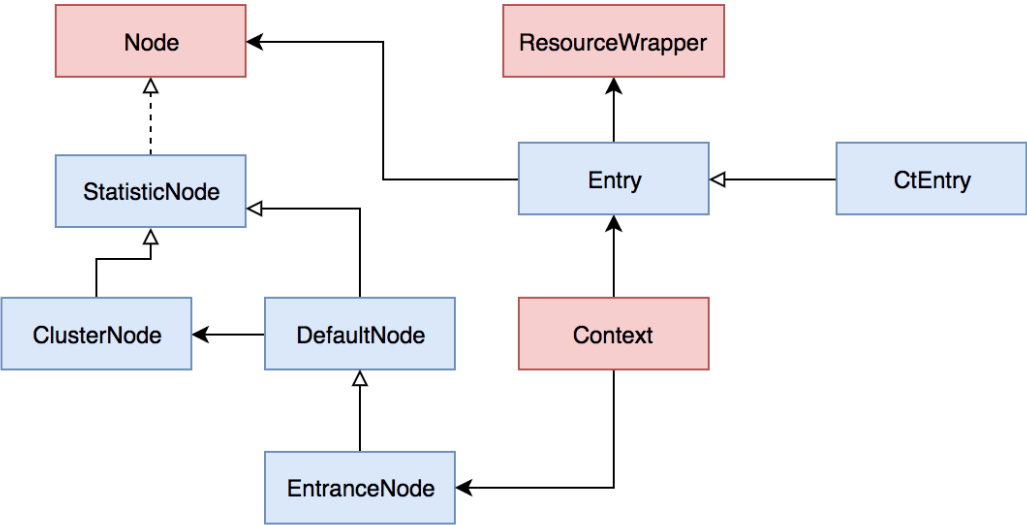
Entry是一个抽象类，他只有一个实现类，在CtSph中的一个静态类： CtEntry

核心的概念4: Node

节点是用来保存某个资源的各种实时统计信息的，他是一个接口，通过访问节点，就可以获取到对应资源的实时状态，以此为依据进行限流和降级操作。

可能看到这里，大家还是比较懵，这么多类到底有什么用，

接下来就让我们更进一步，挖掘一下这些类的作用，在这之前，我先给大家展示一下他们之间的关系，如下图所示：

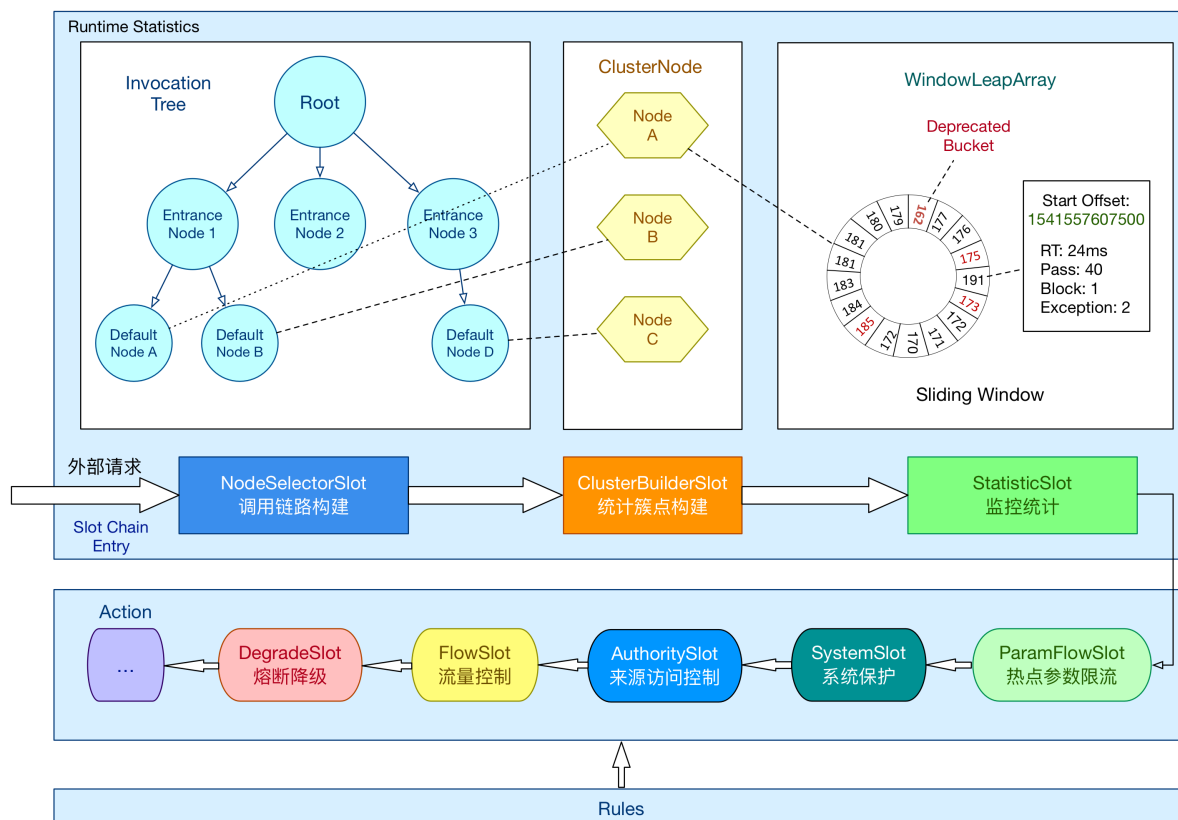


这里把几种Node的作用先大概介绍下：

节点	作用
StatisticNode	执行具体的资源统计操作
DefaultNode	该节点持有指定上下文中指定资源的统计信息，当在同一个上下文中多次调用entry方法时，该节点可能下会创建有一系列的子节点。另外每个DefaultNode中会关联一个ClusterNode
ClusterNode	该节点中保存了资源的总体的运行时统计信息，包括rt，线程数，qps等等，相同的资源会全局共享同一个ClusterNode，不管他属于哪个上下文
EntranceNode	该节点表示一棵调用链树的入口节点，通过他可以获取调用链树中所有的子节点

调用链树

当在一个上下文中多次调用了 SphU#entry() 方法时，就会创建一棵调用链树。



具体的代码在entry方法中创建CtEntry对象时：

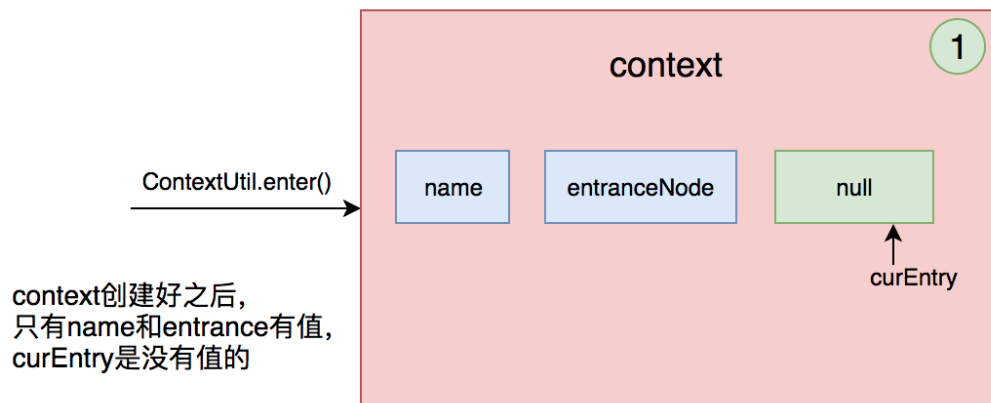
```
CtEntry(ResourceWrapper resourceWrapper, ProcessorSlot<Object> chain, Context context) {
    super(resourceWrapper);
    this.chain = chain;
    this.context = context;
    // 获取「上下文」中上一次的入口
    parent = context.getCurEntry();
    if (parent != null) {
        // 然后将当前入口设置为上一次入口的子节点
        ((CtEntry)parent).child = this;
    }
    // 设置「上下文」的当前入口为该类本身
    context.setCurEntry(this);
}
```

这里可能看代码没有那么直观，可以用一些图形来描述一下这个过程。

构造树干

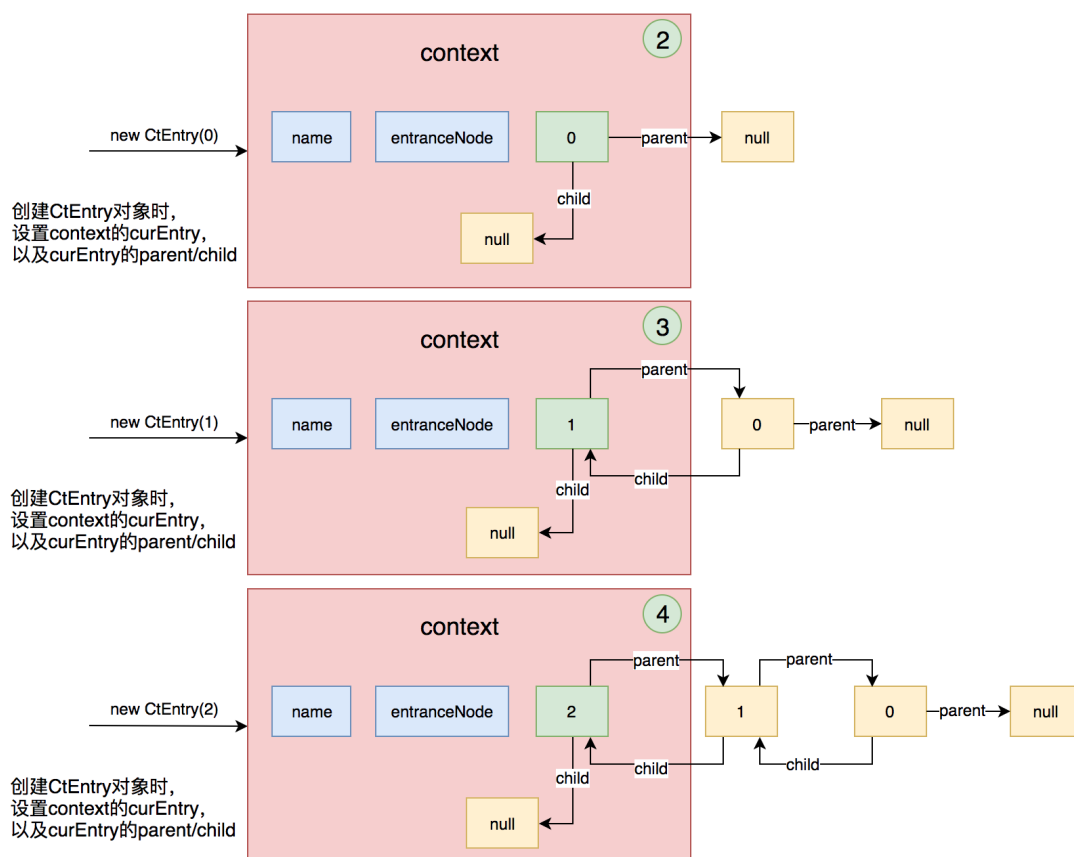
创建context

context的创建在上面已经分析过了，初始化的时候，context中的curEntry属性是没有值的，如下图所示：



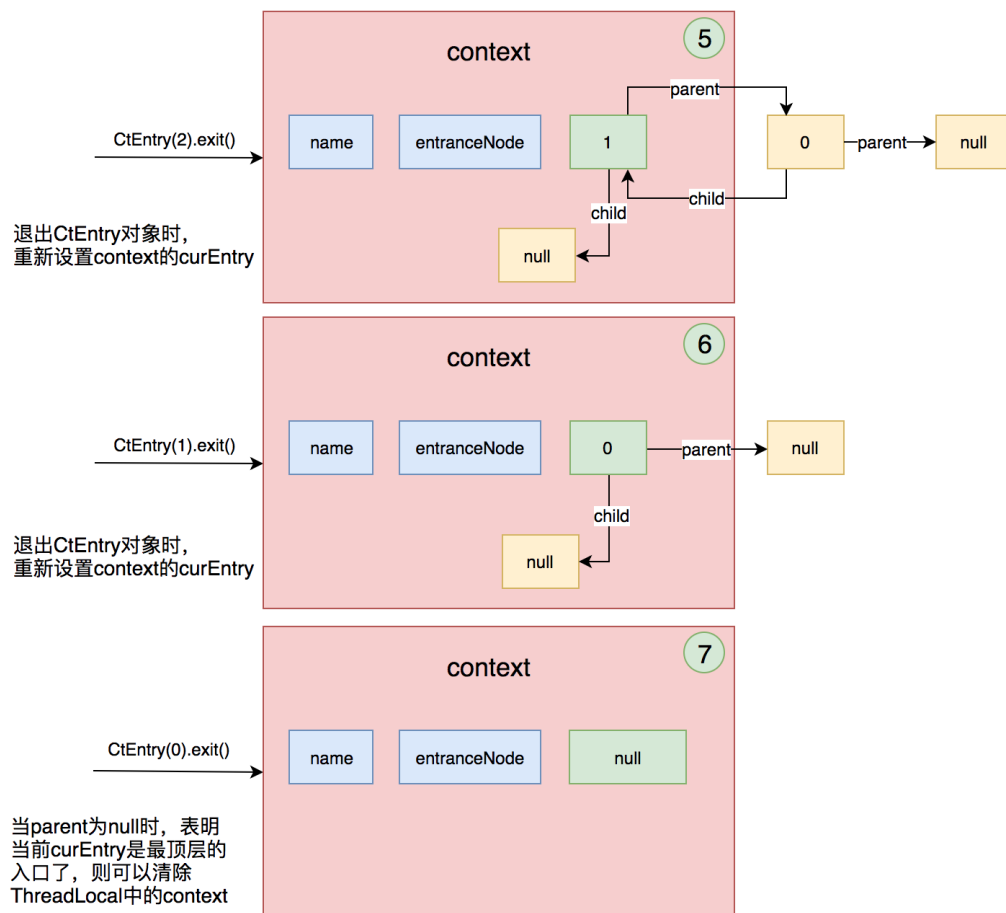
创建Entry

每创建一个新的Entry对象时，都会重新设置context的curEntry，并将context原来的curEntry设置为该新Entry对象的父节点，如下图所示：



退出Entry

某个Entry退出时，将会重新设置context的curEntry，当该Entry是最顶层的一个入口时，将会把ThreadLocal中保存的context也清除掉，如下图所示：



构造叶子节点

上面的过程是构造了一棵调用链的树，但是这棵树只有树干，没有叶子，那叶子节点是在什么时候创建的呢？

DefaultNode就是叶子节点，在叶子节点中保存着目标资源在当前状态下的统计信息。

通过分析，我们知道了叶子节点是在NodeSelectorSlot的entry方法中创建的。

具体的代码如下：

```
@Override
public void entry(Context context, Resourcewrapper resourcewrapper, Object obj,
int count, Object... args) throws Throwable {
    // 根据「上下文」的名称获取DefaultNode
    // 多线程环境下，每个线程都会创建一个context，
    // 只要资源名相同，则context的名称也相同，那么获取到的节点就相同
    DefaultNode node = map.get(context.getName());
    if (node == null) {
        synchronized (this) {
            node = map.get(context.getName());
            if (node == null) {
                // 如果当前「上下文」中没有该节点，则创建一个DefaultNode节点
                node = Env.nodeBuilder.buildTreeNode(resourcewrapper, null);
                // 省略部分代码
            }
            // 将当前node作为「上下文」的最后一个节点的子节点添加进去
            // 如果context的curEntry.parent.curNode为null，则添加到entranceNode中去
            // 否则添加到context的curEntry.parent.curNode中去
        }
    }
}
```

```

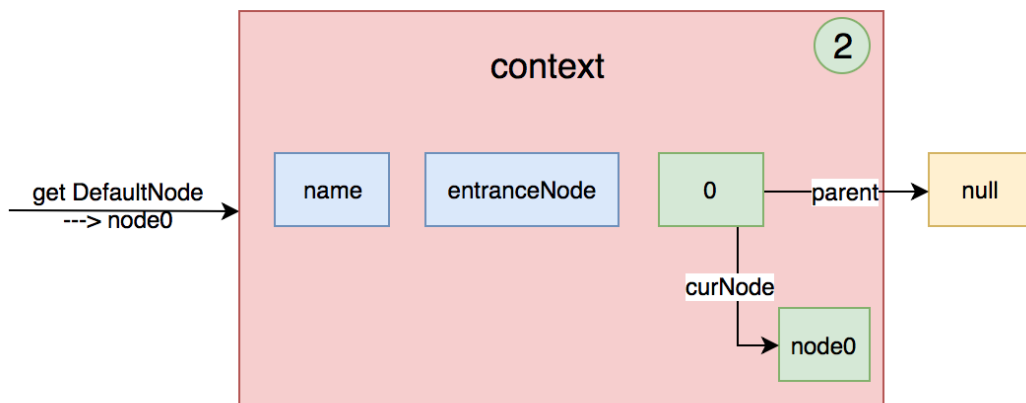
        ((DefaultNode)context.getLastNode()).addChild(node);
    }
}
// 将该节点设置为「上下文」中的当前节点
// 实际是将当前节点赋值给context中curEntry的curNode
// 在Context的getLastNode中会用到在此处设置的curNode
context.setCurNode(node);
fireEntry(context, resourcewrapper, node, count, args);
}

```

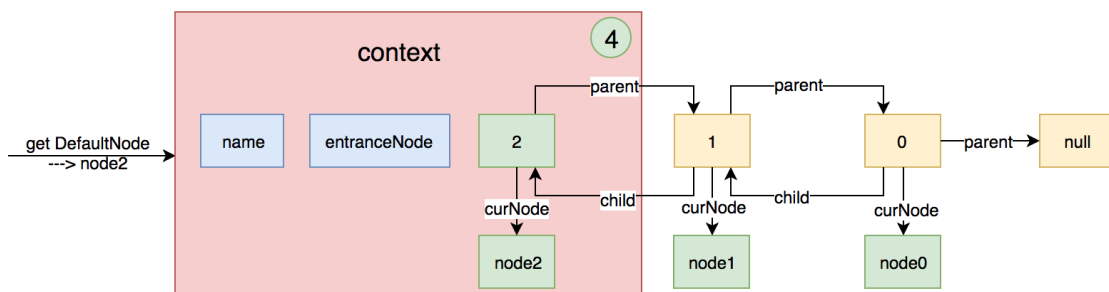
上面的代码可以分解成下面这些步骤：

- 1) 获取当前上下文对应的DefaultNode，如果没有的话会为当前的调用新生成一个DefaultNode节点，它的作用是对资源进行各种统计度量以便进行流控；
- 2) 将新创建的DefaultNode节点，添加到context中，作为「entranceNode」或者「curEntry.parent.curNode」的子节点；
- 3) 将DefaultNode节点，添加到context中，作为「curEntry」的curNode。

上面的第2步，不是每次都会执行。我们先看第3步，把当前DefaultNode设置为context的curNode，实际上是把当前节点赋值给context中curEntry的curNode，用图形表示就是这样：



多次创建不同的Entry，并且执行NodeSelectorSlot的entry方法后，就会变成这样一棵调用链树：



PS：这里图中的node0，node1，node2可能是相同的node，因为在同一个context中从map中获取的node是同一个，这里只是为了表述的更清楚所以用了不同的节点名。

保存子节点

上面已经分析了叶子节点的构造过程，叶子节点是保存在各个Entry的curNode属性中的。

我们知道context中只保存了入口节点和当前Entry，那子节点是什么时候保存的呢，其实子节点就是上面代码中的第2步中保存的。

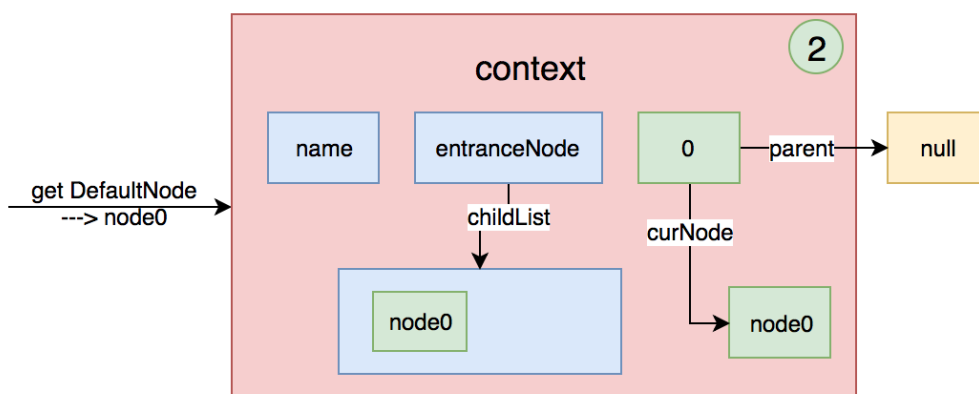
下面我们来分析上面的第2步的情况：

第一次调用NodeSelectorSlot的entry方法时，map中肯定是没有DefaultNode的，那就会进入第2步中，创建一个node，创建完成后会把该节点加入到context的lastNode的子节点中去。我们先看一下context的getLastNode方法：

```
public Node getLastNode() {  
    // 如果curEntry不存在时，返回entranceNode  
    // 否则返回curEntry的lastNode，  
    // 需要注意的是curEntry的lastNode是获取的parent的curNode，  
    // 如果每次进入的资源不同，就会每次都创建一个CtEntry，则parent为null，  
    // 所以curEntry.getLastNode()也为null  
    if (curEntry != null && curEntry.getLastNode() != null) {  
        return curEntry.getLastNode();  
    } else {  
        return entranceNode;  
    }  
}
```

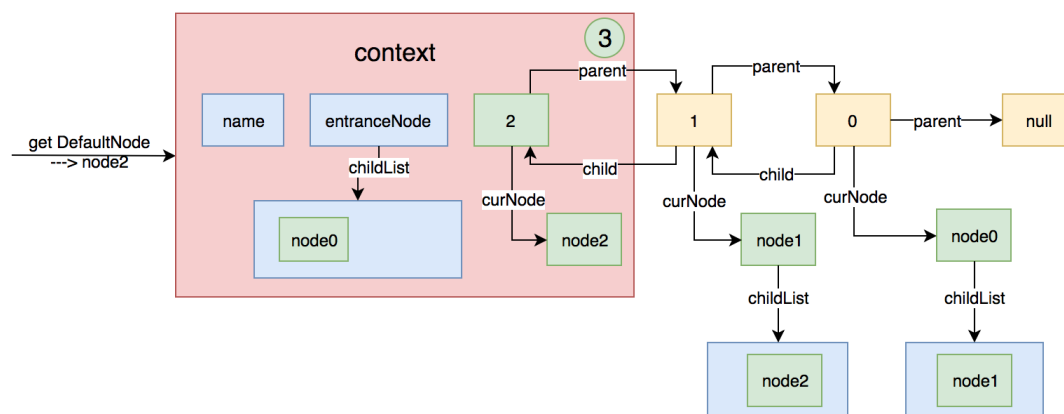
代码中我们可以知道，lastNode的值可能是context中的entranceNode也可能是curEntry.parent.curNode，但是他们都是「DefaultNode」类型的节点，DefaultNode的所有子节点是保存在一个HashSet中的。

第一次调用getLastNode方法时，context中curEntry是null，因为curEntry是在第3步中才赋值的。所以，lastNode最初的值就是context的entranceNode。那么将node添加到entranceNode的子节点中去之后就变成了下面这样：



紧接着再进入一次，资源名不同，会再次生成一个新的Entry，上面的图形就变成下图这样：

假如再创建一个Entry，然后再进入一次不同的资源名，上面的图就变成下面这样：



至此NodeSelectorSlot的基本功能已经大致分析清楚了。

PS：以上的分析是基于每次执行SphU.entry(name)时，资源名都是不一样的前提下。如果资源名都一样的话，那么生成的node都相同，则只会再第一次把node加入到entranceNode的子节点中去，其他的时候，只会创建一个新的Entry，然后替换context中的curEntry的值。

ClusterBuilderSlot

NodeSelectorSlot的entry方法执行完之后，会调用fireEntry方法，此时会触发ClusterBuilderSlot的entry方法。

ClusterBuilderSlot的entry方法比较简单，具体代码如下：

```
@Override
public void entry(Context context, Resourcewrapper resourcewrapper, DefaultNode
node, int count, Object... args) throws Throwable {
    if (clusterNode == null) {
        synchronized (lock) {
            if (clusterNode == null) {
                // Create the cluster node.
                clusterNode = Env.nodeBuilder.buildClusterNode();
                // 将clusterNode保存到全局的map中去
                HashMap<Resourcewrapper, ClusterNode> newMap = new
                HashMap<Resourcewrapper, ClusterNode>(16);
                newMap.putAll(clusterNodeMap);
                newMap.put(node.getId(), clusterNode);

                clusterNodeMap = newMap;
            }
        }
    }
    // 将clusterNode塞到DefaultNode中去
    node.setClusterNode(clusterNode);

    // 省略部分代码

    fireEntry(context, resourcewrapper, node, count, args);
}
```

NodeSelectorSlot的职责比较简单，主要做了两件事：

- 一、为每个资源创建一个clusterNode，然后把clusterNode塞到DefaultNode中去
- 二、将clusterNode保持到全局的map中去，用资源作为map的key

PS：一个资源只有一个ClusterNode，但是可以有多个DefaultNode

StatisticSlot

StatisticSlot负责来统计资源的实时状态，具体的代码如下：

```
@Override
public void entry(Context context, ResourceWrapper resourceWrapper, DefaultNode
node, int count, Object... args) throws Throwable {
    try {
        // 触发下一个Slot的entry方法
        fireEntry(context, resourceWrapper, node, count, args);
        // 如果能通过SlotChain中后面的Slot的entry方法，说明没有被限流或降级
        // 统计信息
        node.increaseThreadNum();
        node.addPassRequest();
        // 省略部分代码
    } catch (BlockException e) {
        context.getCurEntry().setError(e);
        // Add block count.
        node.increaseBlockedQps();
        // 省略部分代码
        throw e;
    } catch (Throwable e) {
        context.getCurEntry().setError(e);
        // should not happen
        node.increaseExceptionQps();
        // 省略部分代码
        throw e;
    }
}

@Override
public void exit(Context context, ResourceWrapper resourceWrapper, int count,
Object... args) {
    DefaultNode node = (DefaultNode)context.getCurNode();
    if (context.getCurEntry().getError() == null) {
        long rt = TimeUtil.currentTimeMillis() -
context.getCurEntry().getCreateTime();
        if (rt > Constants.TIME_DROP_VALVE) {
            rt = Constants.TIME_DROP_VALVE;
        }
        node.rt(rt);
        // 省略部分代码
        node.decreaseThreadNum();
        // 省略部分代码
    }
    fireExit(context, resourceWrapper, count);
}
```

代码分成了两部分，第一部分是entry方法，该方法首先会触发后续slot的entry方法，即SystemSlot、FlowSlot、DegradeSlot等的规则，如果规则不通过，就会抛出BlockException，则会在node中统计被block的数量。反之会在node中统计通过的请求数和线程数等信息。第二部分是在exit方法中，当退出该Entry入口时，会统计rt的时间，并减少线程数。

这些统计的实时数据会被后续的校验规则所使用，具体的统计方式是通过 `滑动窗口` 来实现的。

SystemSlot

SystemSlot就是根据总的请求统计信息，来做流控，主要是防止系统被搞垮，具体的代码如下：

```
@Override
public void entry(Context context, ResourceWrapper resourceWrapper, DefaultNode
node, int count,
                boolean prioritized, Object... args) throws Throwable {
    SystemRuleManager.checkSystem(resourceWrapper);
    fireEntry(context, resourceWrapper, node, count, prioritized, args);
}
```

```
public static void checkSystem(ResourceWrapper resourceWrapper) throws
BlockException {
    if (resourceWrapper == null) {
        return;
    }
    // Ensure the checking switch is on.
    if (!checkSystemStatus.get()) {
        return;
    }

    // for inbound traffic only
    if (resourceWrapper.getEntryType() != EntryType.IN) {
        return;
    }

    // total qps
    double currentQps = Constants.ENTRY_NODE == null ? 0.0 :
Constants.ENTRY_NODE.successQps();
    if (currentQps > qps) {
        throw new SystemBlockException(resourceWrapper.getName(), "qps");
    }

    // total thread
    int currentThread = Constants.ENTRY_NODE == null ? 0 :
Constants.ENTRY_NODE.curThreadNum();
    if (currentThread > maxThread) {
        throw new SystemBlockException(resourceWrapper.getName(), "thread");
    }

    double rt = Constants.ENTRY_NODE == null ? 0 : Constants.ENTRY_NODE.avgRt();
```

```

        if (rt > maxRt) {
            throw new SystemBlockException(resourcewrapper.getName(), "rt");
        }

        // BBR算法
        // load. BBR algorithm.
        if (highestSystemLoadIsSet && getCurrentSystemAvgLoad() > highestSystemLoad)
        {
            if (!checkBbr(currentThread)) {
                throw new SystemBlockException(resourcewrapper.getName(), "load");
            }
        }

        // cpu usage
        if (highestCpuUsageIsSet && getCurrentCpuUsage() > highestCpuUsage) {
            throw new SystemBlockException(resourcewrapper.getName(), "cpu");
        }
    }
}

```

其中的Constants.ENTRY_NODE是一个全局的ClusterNode，该节点的值是在StatisticsSlot中进行统计的。

当前的统计值和系统配置的进行比较，各个维度超过范围抛BlockException

AuthoritySlot

AuthoritySlot做的事也比较简单，主要是根据黑白名单进行过滤，只要有一条规则校验不通过，就抛出异常。

```

@Override
public void entry(Context context, Resourcewrapper resourcewrapper, DefaultNode
node, int count, boolean prioritized, Object... args)
    throws Throwable {
    checkBlackwhiteAuthority(resourcewrapper, context);
    fireEntry(context, resourcewrapper, node, count, prioritized, args);
}

void checkBlackwhiteAuthority(Resourcewrapper resource, Context context) throws
AuthorityException {
    // 通过监听来的规则集
    Map<String, Set<AuthorityRule>> authorityRules =
    AuthorityRuleManager.getAuthorityRules();

    if (authorityRules == null) {
        return;
    }

    // 根据资源名称获取相应的规则
    Set<AuthorityRule> rules = authorityRules.get(resource.getName());
    if (rules == null) {
        return;
    }

    for (AuthorityRule rule : rules) {

```

```

        // 黑名单白名单验证
        // 只要有一条规则校验不通过，就抛出AuthorityException
        if (!AuthorityRuleChecker.passCheck(rule, context)) {
            throw new AuthorityException(context.getOrigin(), rule);
        }
    }
}

```

FlowSlot

FlowSlot主要是根据前面统计好的信息，与设置的限流规则进行匹配校验，如果规则校验不通过则进行限流，具体的代码如下：

```

@Override
public void entry(Context context, Resourcewrapper resourcewrapper, DefaultNode
node, int count,
                boolean prioritized, Object... args) throws Throwable {
    checkFlow(resourcewrapper, context, node, count, prioritized);

    fireEntry(context, resourcewrapper, node, count, prioritized, args);
}

void checkFlow(Resourcewrapper resource, Context context, DefaultNode node, int
count, boolean prioritized)
    throws BlockException {
    checker.checkFlow(ruleProvider, resource, context, node, count,
prioritized);
}

```

DegradeSlot

DegradeSlot主要是根据前面统计好的信息，与设置的降级规则进行匹配校验，如果规则校验不通过则进行降级，具体的代码如下：

```

@Override
public void entry(Context context, Resourcewrapper resourcewrapper, DefaultNode
node, int count,
                boolean prioritized, Object... args) throws Throwable {
    performChecking(context, resourcewrapper);

    fireEntry(context, resourcewrapper, node, count, prioritized, args);
}

void performChecking(Context context, Resourcewrapper r) throws BlockException {
    List<CircuitBreaker> circuitBreakers =
        DegradeRuleManager.getCircuitBreakers(r.getName());

    if (circuitBreakers == null || circuitBreakers.isEmpty()) {
        return;
    }
    for (CircuitBreaker cb : circuitBreakers) {
        if (!cb.tryPass(context)) {
            throw new DegradeException(cb.getRule().getLimitApp(),
cb.getRule());
        }
    }
}

```

```

    }
}
}

```

DefaultProcessorSlotChain

Chain 是链条的意思，从build的方法可看出，ProcessorSlotChain 是一个链表，里面添加了很多个 Slot。都是 ProcessorSlot 的子类。具体的实现需要到 DefaultProcessorSlotChain 中去看。

```

public class DefaultProcessorSlotChain extends ProcessorSlotChain {

    AbstractLinkedProcessorSlot<?> first = new
AbstractLinkedProcessorsSlot<Object>() {

        @Override
        public void entry(Context context, ResourceWrapper resourceWrapper,
Object t, int count, boolean prioritized, Object... args)
            throws Throwable {
            super.fireEntry(context, resourceWrapper, t, count, prioritized,
args);
        }

        @Override
        public void exit(Context context, ResourceWrapper resourceWrapper, int
count, Object... args) {
            super.fireExit(context, resourceWrapper, count, args);
        }

    };

    AbstractLinkedProcessorSlot<?> end = first;

    @Override
    public void addFirst(AbstractLinkedProcessorSlot<?> protocolProcessor) {
        protocolProcessor.setNext(first.getNext());
        first.setNext(protocolProcessor);
        if (end == first) {
            end = protocolProcessor;
        }
    }

    @Override
    public void addLast(AbstractLinkedProcessorSlot<?> protocolProcessor) {
        end.setNext(protocolProcessor);
        end = protocolProcessor;
    }

    /**
     * Same as {@link #addLast(AbstractLinkedProcessorSlot)}.
     *
     * @param next processor to be added.
     */
    @Override
    public void setNext(AbstractLinkedProcessorSlot<?> next) {
        addLast(next);
    }

    @Override

```

```

    public AbstractLinkedProcessorSlot<?> getNext() {
        return first.getNext();
    }

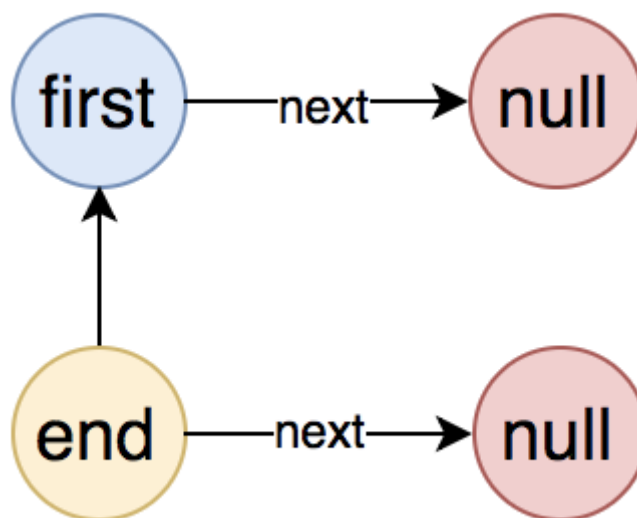
    @Override
    public void entry(Context context, ResourceWrapper resourceWrapper, Object
t, int count, boolean prioritized, Object... args)
        throws Throwable {
        first.transformEntry(context, resourceWrapper, t, count, prioritized,
args);
    }

    @Override
    public void exit(Context context, ResourceWrapper resourceWrapper, int
count, Object... args) {
        first.exit(context, resourceWrapper, count, args);
    }
}

```

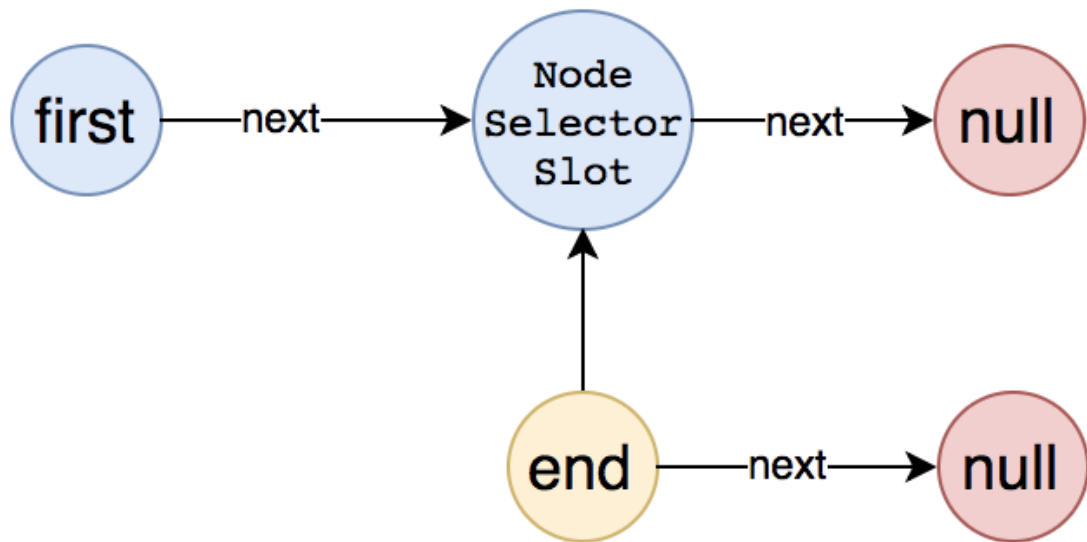
DefaultProcessorSlotChain中有两个AbstractLinkedProcessorSlot类型的变量：first和end，这就是链表的头结点和尾节点。

创建DefaultProcessorSlotChain对象时，首先创建了首节点，然后把首节点赋值给了尾节点，可以用下图表示：



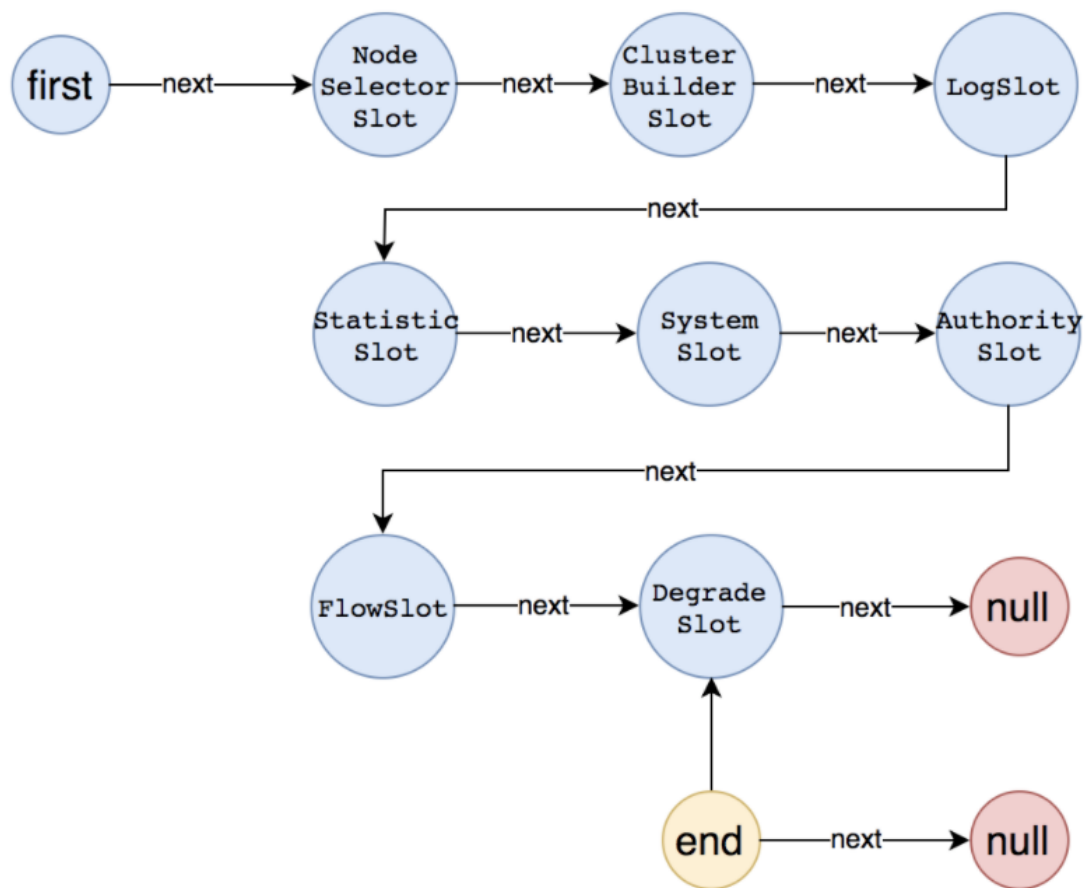
<https://blog.csdn.net/hxyascx>

将第一个节点添加到链表中后，整个链表的结构变成了如下图这样：



<https://blog.csdn.net/hxyascx>

将所有的节点都加入到链表中后，整个链表的结构变成了如下图所示：

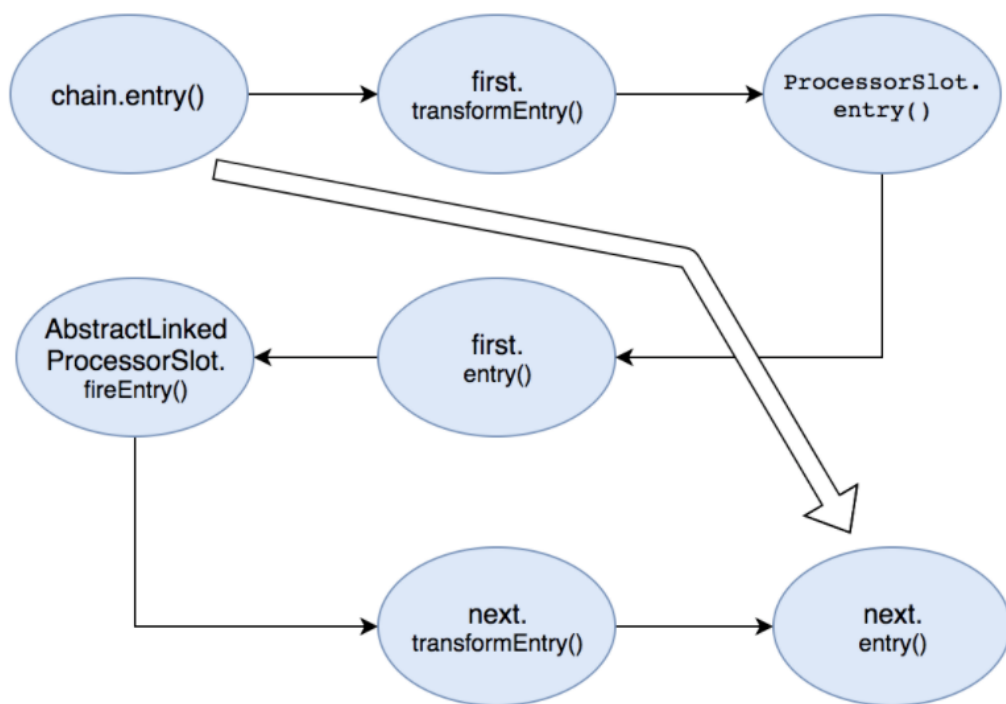


<https://blog.csdn.net/hxyascx>

这样就将所有的Slot对象添加到了链表中去了，每一个Slot都是继承自AbstractLinkedProcessorSlot。而AbstractLinkedProcessorSlot是一种责任链的设计，每个对象中都有一个next属性，指向的是另一个AbstractLinkedProcessorSlot对象。其实责任链模式在很多框架中都有，比如Netty中是通过pipeline来实现的。

知道了SlotChain是如何创建的了，那接下来就要看下是如何执行Slot的entry方法的了

从这里可以看到，从fireEntry方法中就开始传递执行entry了，这里会执行当前节点的下一个节点transformEntry方法，上面已经分析过了，transformEntry方法会触发当前节点的entry，也就是说fireEntry方法实际是触发了下一个节点的entry方法。



<https://blog.csdn.net/hxyascx>

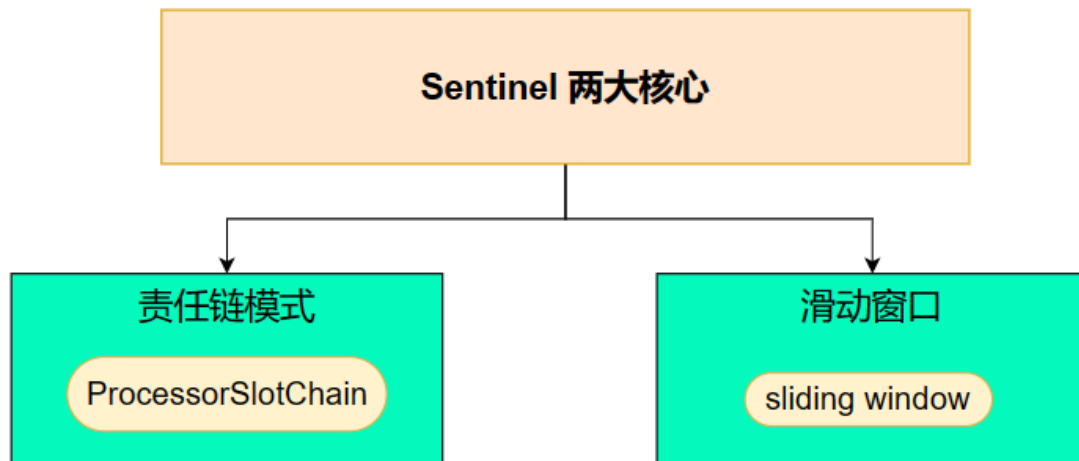
从最初的调用Chain的entry()方法，转变成了调用SlotChain中Slot的entry()方法。从@SpiOrder(-10000)知道，SlotChain中的第一个Slot节点是NodeSelectorSlot。

slot总结

sentinel的限流降级等功能，主要是通过一个SlotChain实现的。在链式插槽中，有7个核心的Slot，这些Slot各司其职，可以分为以下几种类型：

- 一、进行资源调用路径构造的NodeSelectorSlot和ClusterBuilderSlot
 - 二、进行资源的实时状态统计的StatisticsSlot
 - 三、进行系统保护，限流，降级等规则校验的SystemSlot、AuthoritySlot、FlowSlot、DegradeSlot
- 后面几个Slot依赖于前面几个Slot统计的结果。至此，每种Slot的功能已经基本分析清楚了。

10、sentinel滑动窗口 sliding window 源码分析



Sentinel统计QPS使用的是滑动窗口：时间窗口+Bucket，通过循环复用Bucket以减少对内存的占用，在统计QPS时，更是利用当前时间戳定位Bucket，使用LongAdder统计时间窗口内的请求成功数、失败数、总耗时等指标数据优化了并发锁，通过定时任务递增时间戳避免每次都使用System获取当前时间。

Sentinel会为每个资源创建一个保存一分钟，时间窗口为1秒的Bucket数组，以及一个保存一秒钟内以500ms为时间窗口的Bucket数组，将这两个数组包装为一个Node，以统计该接口的请求数据。

每个Bucket记录一个时间窗口内的请求总数、失败总数、总耗时（通过总耗时可计算平均耗时）、被限流或者被熔断的请求总数这些指标数据。

```
StatisticNode
1 public class StatisticNode implements Node {
2
3     private transient volatile Metric rollingCounterInSecond = new ArrayMetric(SampleCountProperty.SAMPLE_COUNT,
4         IntervalProperty.INTERVAL);
5
6     private transient Metric rollingCounterInMinute = new ArrayMetric(60, 60 * 1000, false);
7 }
```

因此，Sentinel消耗的内存，至少是资源总数乘以每个资源对应的Node占用的内存大小，每个Node占用的内存大小即为一个大小为2的Bucket数组和一个大小为60的Bucket数组所占用的内存。

可以看到Sentinel在性能方面所做出的努力，Sentinel尽最大可能降低自身对应用的影响。

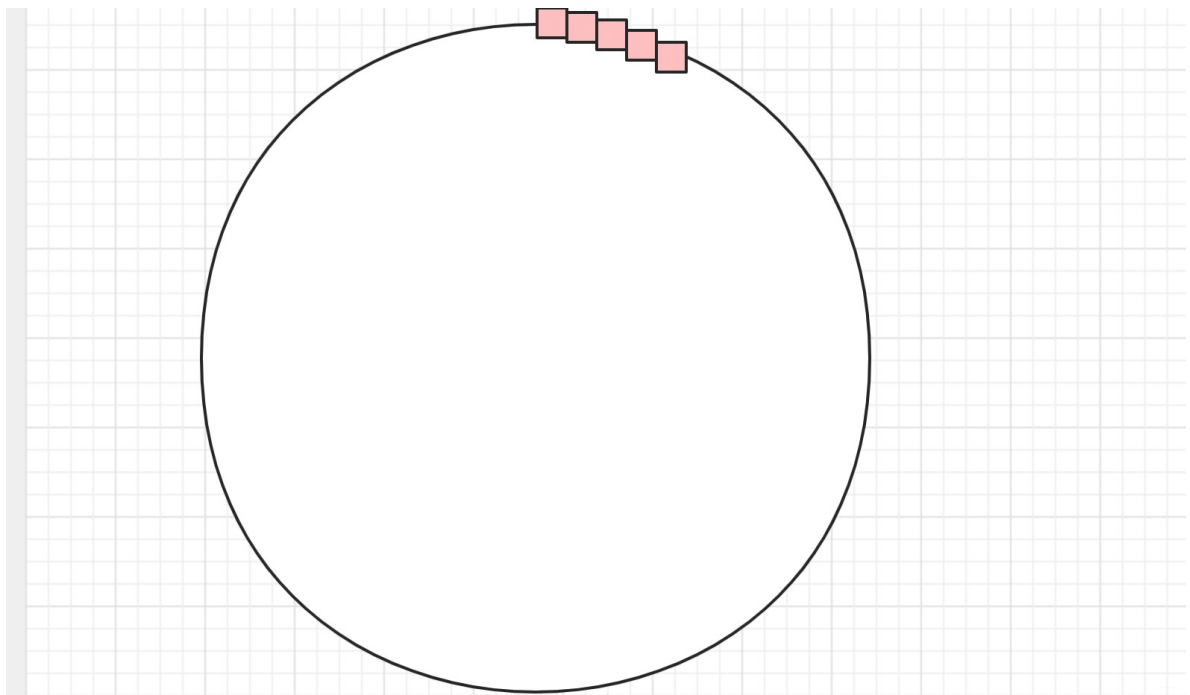
10.1 基本原理

滑动窗口可以先拆为滑动跟窗口两个词，先介绍下窗口，

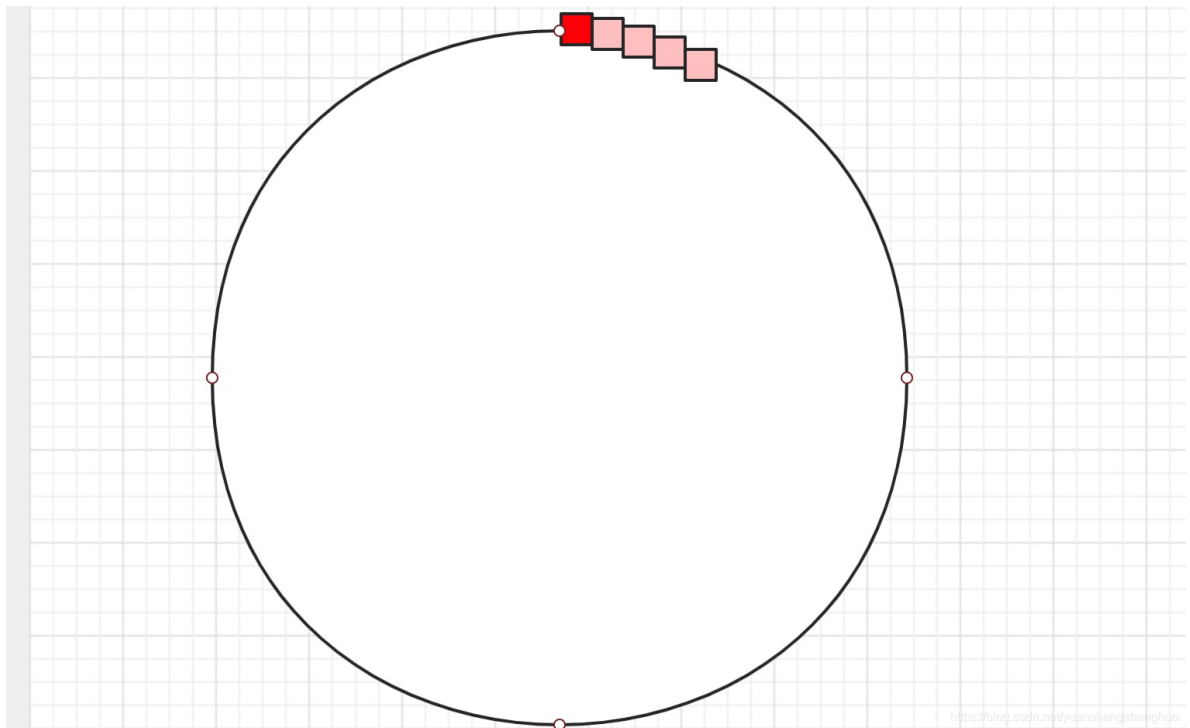
你可以这么理解，一段是时间就是窗口，比如说我们可以把这个1s认为是1个窗口

这个样子我们就能将1分钟就可以划分成60个窗口了，这个没毛病吧。

如下图我们就分成了60个窗口（这个多了我们就画5个表示一下）

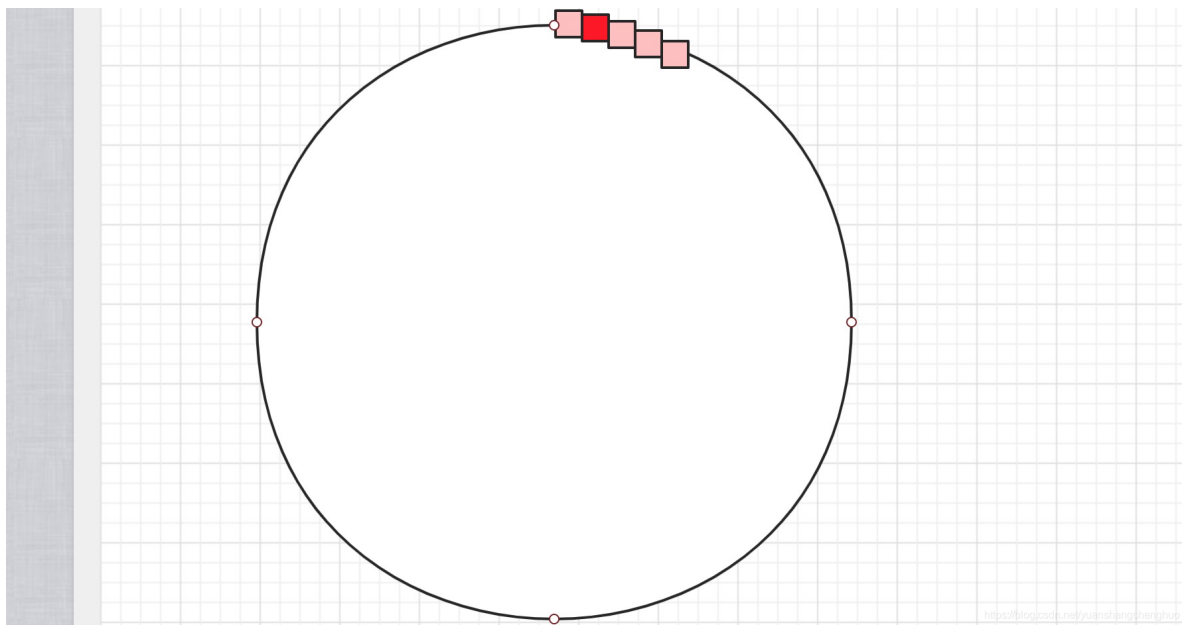


比如现在处于第1秒上，那1s那个窗口就是 当前窗口，就如下图中红框表示。



好了，窗口就介绍完了，现在在来看下 滑动，

滑动很简单，比如说现在时间由第1秒变成了第2秒，就是从当前这个窗口---->下一个窗口就可以了，这个时候下一个窗口就变成了当前窗口，之前那个当前窗口就变成了上一个窗口，这个过程其实就是滑动。



好了，介绍完了滑动窗口，我们再来介绍下这个sentinel的滑动窗口的实现原理。

其实你要是理解了上面这个滑动窗口的意思，sentinel实现原理就简单了。

先是介绍下窗口中里面都存储些啥。也就是上面这个小框框有啥。

1. 它得有个开始时间吧，不然你怎么知道这个窗口是什么时候开始的
2. 还得有个窗口的长度吧，不然你咋知道窗口啥时候结束，通过这个开始时间+窗口长度=窗口结束时间，就比如说上面的1s，间隔1s
3. 最后就是要在窗口里面统计的东西，你总不能白搞些窗口，搞些滑动吧。所以这里就存储了一堆要统计的指标（qps，rt等等）

说完了这一个小窗口里面的东西，就得来说说是怎么划分这个小窗口，怎么管理这些小窗口的了，也就是我们的视野得往上提高一下了，不能总聚在这个小窗口上。

1. 要知道有多少个小窗口，在sentinel中也就是sampleCount，比如说我们有60个窗口。
2. 还有就是intervalInMs，这个intervalInMs是用来计算这个窗口长度的，intervalInMs/窗口数量=窗口长度。也就是我给你1分钟，你给我分成60个窗口，这个时候窗口长度就是1s了，那如果我给你1s，你给我分2个窗口，这个时候窗口长度就是500毫秒了，这个1分钟，就是intervalInMs。
3. 再就是存储这个窗口的容器（这里是数组），毕竟那么多窗口，还得提供计算当前时间窗口的方法等等

最后我们就来看看这个当前时间窗口是怎么计算的。

咱们就拿 60个窗口，这个60个窗口放在数组中，窗口长度是1s 来计算，看看当前时间戳的一个时间窗口是在数组中哪个位置。

比如说当前时间戳是1609085401454 ms，算出秒 = 1609085401454 /1000（窗口长度）

在数组的位置 = 算出秒 %数组长度

我们再来计算下 某个时间戳对应窗口的起始时间，还是以1609085401454 来计算

窗口startTime = 1609085401454 - 1609085401454%1000（窗口长度）=454

这里1609085401454%1000（窗口长度）能算出来它的毫秒值，也就是454，减去这个后就变成了1609085401000

好了，sentinel 滑动窗口原理就介绍完成了。

10.2 sentinel使用滑动窗口都统计啥

我们来介绍下使用这个滑动窗口都来统计啥

```
public enum MetricEvent {  
    /**  
     * Normal pass.  
     */  
    PASS, // 通过  
    /**  
     * Normal block.  
     */  
    BLOCK, // 拒绝的  
    EXCEPTION, // 异常  
    SUCCESS, // 成功  
    RT, // 耗时  
    /**  
     * Passed in future quota (pre-occupied, since 1.5.0).  
     */  
    OCCUPIED_PASS  
}
```

这是最基本的指标，然后通过这些指标，又可以计算出来比如说最大，最小，平均等等的一些指标。

10.3 滑动窗口源码实现

我们先来看下这个窗口里面的统计指标的实现 MetricBucket

3.1 MetricBucket 源码分析

这个MetricBucket 是由LongAdder数组组成的，一个LongAdder就是一个MetricEvent，也就是第二小节里面的PASS，BLOCK等等。

我们稍微看下就可以了



```
27  /**  
28  public class MetricBucket {  
29      /**  
30       * 存储各事件的计数，比如异常总数、请求总数等  
31       */  
32      private final LongAdder[] counters;  
33      /**  
34       * 这段事件内的最小耗时  
35       */  
36      private volatile long minRt;  
37      public MetricBucket() {  
38          // 创建各个事件  
39          MetricEvent[] events = MetricEvent.values();  
40          this.counters = new LongAdder[events.length];  
41          for (MetricEvent event : events) {  
42              counters[event.ordinal()] = new LongAdder();  
43          }  
44          // 初始化最小rt  
45          initMinRt();  
46      }  
47      public MetricBucket reset(MetricBucket bucket) {  
48          for (MetricEvent event : MetricEvent.values()) {  
49              counters[event.ordinal()].reset();  
50              counters[event.ordinal()].add(bucket.get(event));  
51          }  
52          initMinRt();  
53          return this;  
54      }  
55  }  
56  /**  
57  }
```

可以看到它在实例化的时候创建一个LongAdder 数据，个数就是那堆event的数量。这个LongAdder是jdk8里面的原子操作类，你可以把它简单认为AtomicLong。然后下面就是一堆get 跟add 的方法了，这里我们就不看了。

接下来再来看看那这个窗口的实现WindowWrap类

3.2 WindowWrap 源码分析

先来看下这个成员

```
27  /**
28   * Time length of a single window bucket in milliseconds.
29   * 窗口长度
30   */
31   private final long windowLengthInMs;
32
33   /**
34   * Start timestamp of the window in milliseconds.
35   *
36   * 窗口开始时间 ms
37   */
38   private long windowStart;
39
40   /**
41   * Statistic data.
42   *
43   * 统计数据
44   */
45   private T value;
```

窗口长度，窗口startTime，指标统计的都有了，下面的就没啥好看的了，我们再来看下的一个方法吧

```
/**
 * Check whether given timestamp is in current bucket.
 * 判断某个时间戳有没有在这个窗口中
 * @param timeMillis valid timestamp in ms
 * @return true if the given time is in current bucket, otherwise false
 * @since 1.5.0
 */
public boolean isTimeInWindow(long timeMillis) {
    // 就是 窗口起始时间 与结束时间之间
    return windowStart <= timeMillis && timeMillis < windowStart + windowLengthInMs;
}
```

就是判断某个时间戳是不是在这个窗口中
时间戳要大于等于窗口开始时间 && 小于这个结束时间。

接下来再来看下这个管理窗口的类LeapArray

3.3 LeapArray 源码分析

```
2  protected int windowLengthInMs; // 窗口长度
3  protected int sampleCount; // 样本数
4  protected int intervalInMs; // 总时间间隔
5
6  // 窗口数组
7  protected final AtomicReferenceArray<WindowWrap<T>> array;
8
9  /**
10   * The conditional (predicate) update lock is used only when current bucket is deprecated.
11   */
12   private final ReentrantLock updateLock = new ReentrantLock();
13
14  /**
15   * The total bucket count is: {@code sampleCount = intervalInMs / windowLengthInMs}.
16   */
```

看下它的成员，窗口长度，样本数sampleCount 也就是窗口个数，intervalInMs，再就是窗口数组
看看它的构造方法

```
/**
 * @param intervalInMs the total time interval of this {@code LeapArray} in milliseconds
 */
public LeapArray(int sampleCount, int intervalInMs) {
    AssertUtil.isTrue(value: sampleCount > 0, message: "bucket count is invalid: " + sampleCount);
    AssertUtil.isTrue(value: intervalInMs > 0, message: "total time interval of the sliding window should be positive");
    AssertUtil.isTrue(value: intervalInMs % sampleCount == 0, message: "time span needs to be evenly divided");
    // 窗口大小
    this.windowLengthInMs = intervalInMs / sampleCount;
    // 总时间间隔
    this.intervalInMs = intervalInMs;
    // 样本数
    this.sampleCount = sampleCount;
    // 创建数组
    this.array = new AtomicReferenceArray<>(sampleCount);
}
```

这个构造方法其实就是计算出来这个窗口长度，创建了窗口数组。

11 使用Nacos存储规则及双向同步

1、Sentinel 动态规则扩展

Sentinel 的理念是开发者只需要关注资源的定义，当资源定义成功后可以动态增加各种流控降级规则。

Sentinel 提供两种方式修改规则：

- 通过 API 直接修改 (loadRules) (硬编码方式)
- 通过 DataSource 适配不同数据源修改

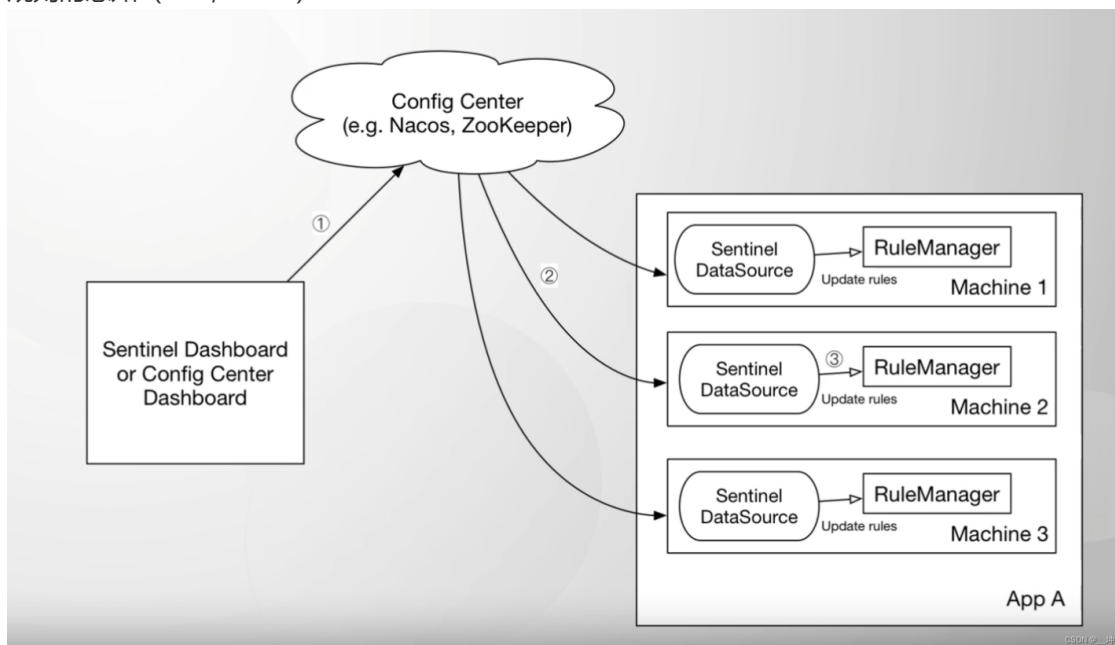
手动通过 API 修改比较直观，可以通过以下几个 API 修改不同的规则：

```
FlowRuleManager.loadRules(List<FlowRule> rules); // 修改流控规则
DegradRuleManager.loadRules(List<DegradRule> rules); // 修改降级规则
```

手动修改规则（硬编码方式）一般仅用于测试和演示或者小规模的规则场景

大量规则场景，一般通过动态规则源的方式来动态管理规则。

- config center: zookeeper / apollo / etcd / redis / Consul / Eureka / Nacos
- 规则的感知 (Pull, Push)



2、规则管理及推送

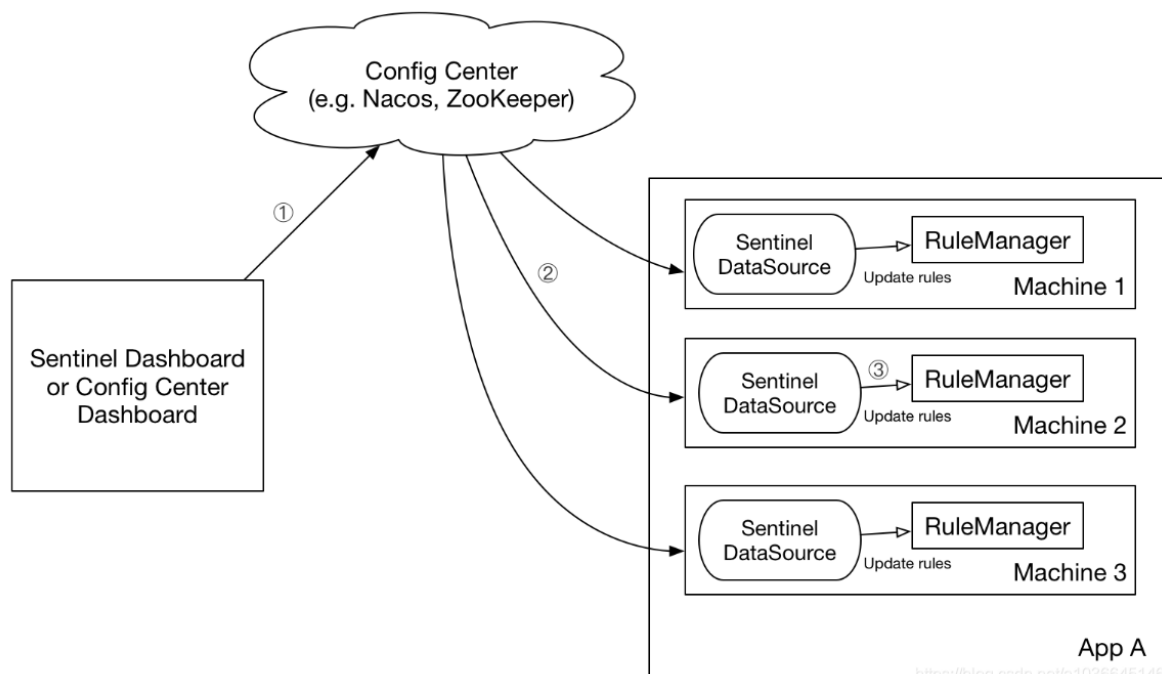
一般来说，规则的推送有下面三种模式：

推送模式	说明	优点	缺点
原始模式	API 将规则推送至客户端并直接更新到内存中，扩展写数据源（ WritableDataSource ）	简单，无任何依赖	不保证一致性；规则保存在内存中，重启即消失。严重不建议用于生产环境
Pull 模式	扩展写数据源（ WritableDataSource ），客户端主动向某个规则管理中心定期轮询拉取规则，这个规则中心可以是 RDBMS、文件等	简单，无任何依赖；规则持久化	不保证一致性；实时性不保证，拉取过于频繁也可能会有性能问题。
Push 模式	扩展读数据源（ ReadableDataSource ），规则中心统一推送，客户端通过注册监听器的方式时刻监听变化，比如使用 Nacos、Zookeeper 等配置中心。这种方式有更好的实时性和一致性保证。 生产环境下一般采用 push 模式的数据源。	规则持久化；一致性；快速	引入第三方依赖

3、DataSource 扩展

上述 `loadRules()` 方法只接受内存态的规则对象，但更多时候规则存储在文件、数据库或者配置中心当中。`DataSource` 接口给我们提供了对接任意配置源的能力。相比直接通过 API 修改规则，实现 `DataSource` 接口是更加可靠的做法。

生产环境下一般更常用的是 push 模式的数据源。对于 push 模式的数据源,如远程配置中心（ZooKeeper, Nacos, Apollo 等等），推送的操作不应由 Sentinel 客户端进行，而应该经控制台统一进行管理，直接进行推送，数据源仅负责获取配置中心推送的配置并更新到本地。因此推送规则正确做法应该是 **配置中心控制台/Sentinel 控制台 → 配置中心 → Sentinel 数据源 → Sentinel**，而不是经 Sentinel 数据源推送至配置中心。这样的流程就非常清晰了：



`DataSource` 扩展常见的实现方式有:

- **拉模式**: 客户端主动向某个规则管理中心定期轮询拉取规则, 这个规则中心可以是 RDBMS、文件, 甚至是 VCS 等。这样做的方式是简单, 缺点是无法及时获取变更;
- **推模式**: 规则中心统一推送, 客户端通过注册监听器的方式时刻监听变化, 比如使用 [Nacos](#)、[Zookeeper](#) 等配置中心。这种方式有更好的实时性和一致性保证。

Sentinel 目前支持以下数据源扩展:

- Pull-based: 文件、Consul
- Push-based: ZooKeeper, Redis, Nacos, Apollo, etcd

详细说明, 可参考官网:

<https://github.com/alibaba/Sentinel/wiki/%E5%8A%A8%E6%80%81%E8%A7%84%E5%88%99%E6%89%A9%E5%B1%95>

DataSource(接口)

1、导入依赖

```
<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
</dependency>

<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-datasource-nacos</artifactId>
</dependency>

<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-nacos-config</artifactId>
</dependency>
```

```

<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-nacos-
discovery</artifactId>
</dependency>

```

2、配置

```

#开启sentinel
feign.sentinel.enabled=true
spring.cloud.sentinel.transport.port=8719
#sentinel控制台
spring.cloud.sentinel.transport.dashboard=localhost:8080
#服务启动直接建立心跳连接
spring.cloud.sentinel.eager=true

#Sentinel使用Nacos存储规则
spring.cloud.sentinel.datasource.ds1.nacos.server-addr=192.168.100.80:8848
spring.cloud.sentinel.datasource.ds1.nacos.dataId=${spring.application.name}-
flow-rules
spring.cloud.sentinel.datasource.ds1.nacos.groupId=myGroupId
spring.cloud.sentinel.datasource.ds1.nacos.namespace=@namespace@
spring.cloud.sentinel.datasource.ds1.nacos.data-type=json
spring.cloud.sentinel.datasource.ds1.nacos.rule-type=flow

```

注：如果你的Nacos配置了不同的隔离环境 **namespace**，则需要指定具体哪一个namespace，否则会加载不到规则配置，报错如下：

```

[c.a.c.s.datasource.converter.SentinelConverter    ] line 80 : converter can
not convert rules because source is empty

```

naocs 如下：



- `spring.cloud.sentinel.transport.dashboard`: sentinel dashboard的访问地址, 根据上面准备工作中启动的实例配置
- `spring.cloud.sentinel.datasource.ds.nacos.server-addr`: nacos的访问地址, 根据上面准备工作中启动的实例配置
- `spring.cloud.sentinel.datasource.ds.nacos.groupId`: nacos中存储规则的groupId
- `spring.cloud.sentinel.datasource.ds.nacos.dataId`: nacos中存储规则的数据Id
- `spring.cloud.sentinel.datasource.ds.nacos.rule-type`: 该参数是spring cloud alibaba升级到0.2.2之后增加的配置, 用来定义存储的规则类型。所有的规则类型可查看枚举类:
`org.springframework.cloud.alibaba.sentinel.datasource.RuleType`, 每种规则的定义格式可以通过各枚举值中定义的规则对象来查看, 比如限流规则可查看:
`com.alibaba.csp.sentinel.slots.block.flow.FlowRule`

这里对于dataId使用了 `${spring.application.name}` 变量, 这样可以根据应用名来区分不同的规则配置。

3、Nacos中创建限流规则的配置

在Nacos控制台, 对应的namespace, 新建一个json配置文件: **service-order-flow-rules**, 如下:

新建配置

* Data ID:

* Group:

[更多高级选项](#)

描述:

配置格式: ☐ TEXT ☒ JSON ☐ XML ☐ YAML ☐ HTML ☐ Properties

* 配置内容: ? :

```

1  [
2      {
3          "resource": "/test",
4          "limitApp": "default",
5          "grade": 1,
6          "count": 10,
7          "strategy": 0,
8          "controlBehavior": 0,
9          "clusterMode": false
10     }
11 ]
  
```

其中: Data ID、Group 就是上面第5点中配置的内容。

配置格式选择JSON, 并在配置内容中填入下面的内容:

```
[
  {
    "resource": "/test",
    "limitApp": "default",
    "grade": 1,
    "count": 10,
    "strategy": 0,
    "controlBehavior": 0,
    "clusterMode": false
  }
]
```

可以看到上面配置规则是一个**数组类型**，数组中的每个对象是针对每一个保护资源的配置对象，每个对象中的属性解释如下：

- resource：资源名，即限流规则的作用对象
- limitApp：流控针对的调用来源，若为 default 则不区分调用来源
- grade：限流阈值类型（QPS 或并发线程数）；0 代表根据并发数量来限流，1 代表根据QPS来进行流量控制
- count：限流阈值
- strategy：调用关系限流策略
- controlBehavior：流量控制效果（直接拒绝、Warm Up、匀速排队）
- clusterMode：是否为集群模式

配置管理 | public 查询结果：共查询到 1 条满足要求的配置。

Data ID: Group: [查询](#) [高级查询](#) [导出查询结果](#) [导入配置](#)

☐	Data Id	Group	归属应用:	操作
☐	service-order-flow-rules	DEFAULT_GROUP		详情 示例代码 编辑 删除 更多

[删除](#) [导出选中的配置](#) [克隆](#) 每页显示: 10

启动service-order 应用，注册到nacos到，打开Sentinel控制台，可以看到上面nacos新建的限流规则，如下：

Sentinel 控制台 1.7.2

应用名: 搜索

service-order (1/1)

实时监控 节点链路 **流控规则** 降级规则

service-order [+ 新增流控规则](#)

流控规则 192.168.239.18720 关键字 [刷新](#)

资源名	来源应用	流控模式	阈值类型	阈值	阈值模式	流控效果	操作
/test	default	直接	QPS	10	单机	快速失败	编辑 删除

共 1 条记录, 每页 10 条记录

注意：

在完成了上面的整合之后，对于接口流控规则的修改就存在两个地方了：**Sentinel控制台、Nacos控制台**。

这个时候，通过Nacos修改该条规则是可以同步到Sentinel的，但是通过Sentinel控制台修改或新增却不可以同步到Nacos。

因为当前版本的Sentinel控制台不具备同步修改Nacos配置的能力，而Nacos由于可以通过在客户端中使用Listener来实现自动更新。

所以，在整合了Nacos做规则存储之后，需要知道在下面两个地方修改存在不同的效果：

- Sentinel控制台中修改规则：仅存在于服务的**内存**中，不会修改Nacos中的配置值，重启后恢复原来的值。
- Nacos控制台中修改规则：服务的内存中规则会更新，Nacos中持久化规则也会更新，重启后依然保持。

下面我们进通过修改，使得Nacos与Sentinel可以互相同步限流规则：

12、Nacos与Sentinel互相同步限流规则

1、流控推送规则

要通过 Sentinel 控制台配置集群流控规则，需要对控制台进行改造。主要改造规则可以参考：

```
https://github.com/alibaba/Sentinel/wiki/Sentinel-控制台（集群流控管理）#规则配置
```

其控制台推送规则：

- 将规则推送到Nacos或其他远程配置中心
- Sentinel客户端链接Nacos，获取规则配置；并监听Nacos配置变化，如发生变化，就更新本地缓存。

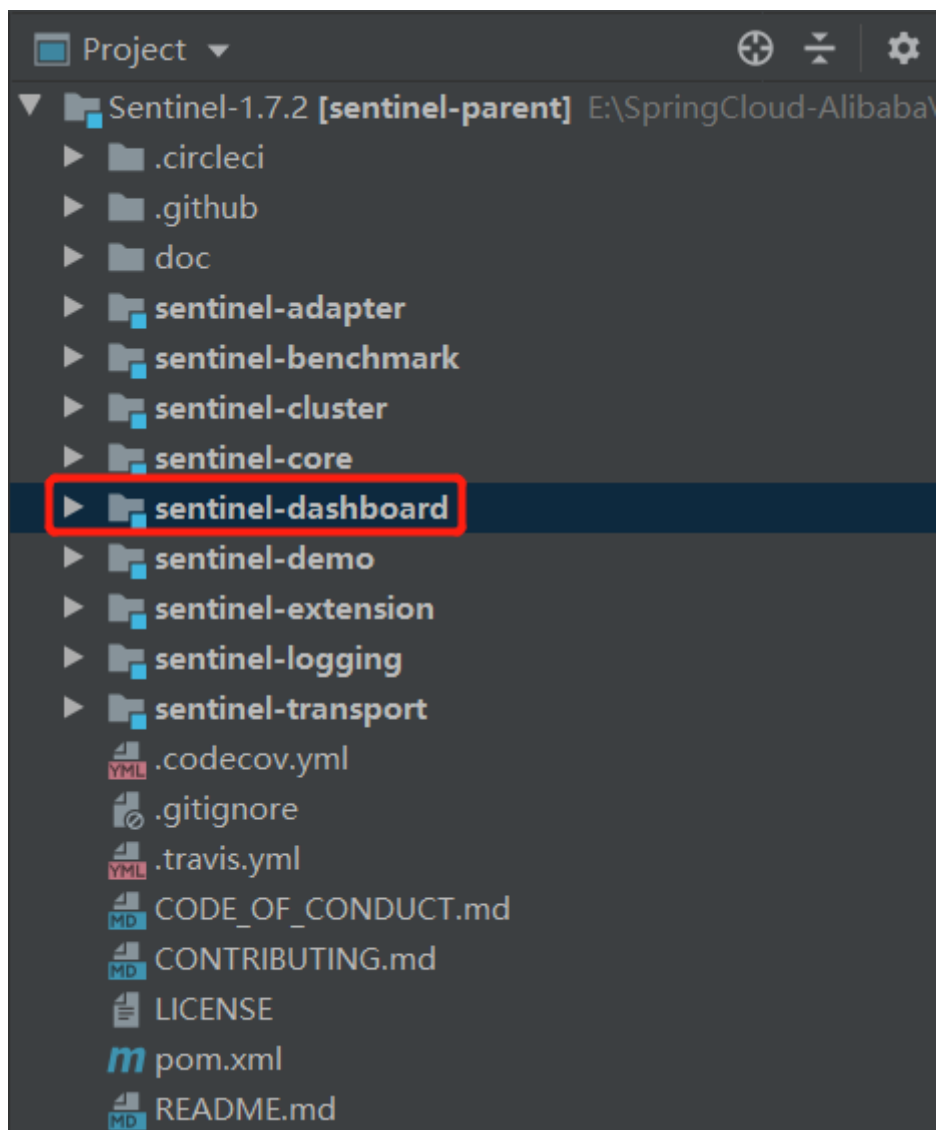
控制台监听Nacos配置变化，如发生变化就更新本地缓存。从而让控制台本地缓存总是和Nacos一致。

2、改造sentinel-dashboard

2.1 通关git官网下载Sentinel 源代码，如下：

```
https://github.com/alibaba/Sentinel/archive/1.7.2.zip
```

下载后解压，使用IDEA打开如下：



2.2 修改sentinel-dashboard 控制台模块的pom.xml，将test注释掉

```
<dependency>
  <groupId>com.alibaba.csp</groupId>
  <artifactId>sentinel-datasource-nacos</artifactId>
  <!--<scope>test</scope>-->
</dependency>
```

2.3 修改sidebar.html页面 (sentinel控制台左边菜单栏)

```
sentinel-
dashboard/src/main/webapp/resources/app/scripts/directives/sidebar.html
```

并找到如下代码段后，并把注释打开，名称也稍作修改。

修改前：

```

<!--<li ui-sref-active="active" ng-if="entry.appType==0">
  <a ui-sref="dashboard.flow({app: entry.app})">
    <i class="glyphicon glyphicon-filter"></i>&nbsp;&nbsp;&nbsp;流控规则 V1</a>
</li>-->

<li ui-sref-active="active" ng-if="entry.isGateway">
  <a ui-sref="dashboard.gatewayApi({app: entry.app})">
    <i class="glyphicon glyphicon-tags"></i> API 管理</a>
</li>
<li ui-sref-active="active" ng-if="entry.isGateway">
  <a ui-sref="dashboard.gatewayFlow({app: entry.app})">
    <i class="glyphicon glyphicon-filter"></i> 流控规则</a>
</li>

<li ui-sref-active="active" ng-if="!entry.isGateway">
  <a ui-sref="dashboard.flowV1({app: entry.app})">
    <i class="glyphicon glyphicon-filter"></i> &nbsp;&nbsp;&nbsp;流控规则</a>
</li>

```

修改后:

```

<li ui-sref-active="active" ng-if="entry.appType==0">
  <a ui-sref="dashboard.flowV1({app: entry.app})">
    <i class="glyphicon glyphicon-filter"></i>&nbsp;&nbsp;&nbsp;流控规则_V1</a>
</li>

<li ui-sref-active="active" ng-if="entry.isGateway">
  <a ui-sref="dashboard.gatewayApi({app: entry.app})">
    <i class="glyphicon glyphicon-tags"></i> API 管理</a>
</li>
<li ui-sref-active="active" ng-if="entry.isGateway">
  <a ui-sref="dashboard.gatewayFlow({app: entry.app})">
    <i class="glyphicon glyphicon-filter"></i> 流控规则</a>
</li>

<li ui-sref-active="active" ng-if="!entry.isGateway">
  <!-- <a ui-sref="dashboard.flowV1({app: entry.app})">-->
  <a ui-sref="dashboard.flow({app: entry.app})">
    <i class="glyphicon glyphicon-filter"></i> &nbsp;&nbsp;&nbsp;流控规则_V2(nacos)</a>
</li>

```

2.4 修改nacos相关java代码

找到如下目录 (位于test目录)

```

sentinel-
dashboard/src/test/java/com/alibaba/csp/sentinel/dashboard/rule/nacos

```

将整个目录拷贝到

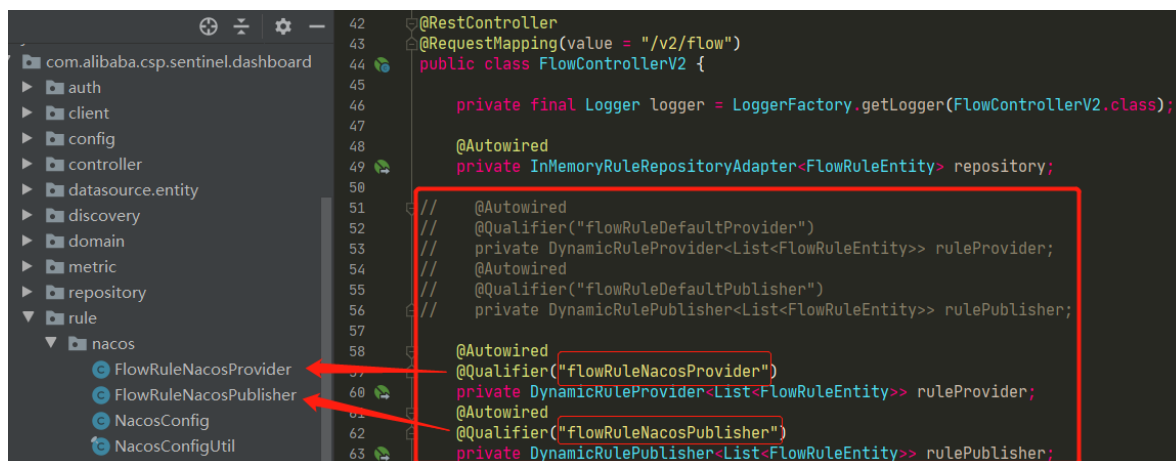
```

sentinel-
dashboard/src/main/java/com/alibaba/csp/sentinel/dashboard/rule/nacos

```

修改com.alibaba.csp.sentinel.dashboard.controller.v2.FlowControllerV2.java

修改如下:



其中，注入的两个bean：

- flowRuleNacosProvider，就是实现Nacos的限流规则配置拉取。
- flowRuleNacosPublisher，实现Nacos的配置推送。

FlowRuleNacosProvider.java 如下，无需修改

```
@Component("flowRuleNacosProvider")
public class FlowRuleNacosProvider implements DynamicRuleProvider<List<FlowRuleEntity>> {

    @Autowired
    private ConfigService configService;
    @Autowired
    private Converter<String, List<FlowRuleEntity>> converter;

    @Override
    public List<FlowRuleEntity> getRules(String appName) throws Exception {
        String rules = configService.getConfig(appName + NacosConfigUtil.FLOW_DATA_ID_POSTFIX,
            NacosConfigUtil.GROUP_ID, 3000);
        if (StringUtil.isEmpty(rules)) {
            return new ArrayList<>();
        }
        return converter.convert(rules);
    }
}
```

- getRules 方法中的 appName 参数是Sentinel中的服务名称。
- configService.getConfig 方法是从Nacos中获取配置信息的具体操作。其中，DataId和GroupId分别对应客户端使用时候的对应配置。

如下：

```
spring.cloud.sentinel.datasource.ds1.nacos.server-addr=192.168.100.80:8848
spring.cloud.sentinel.datasource.ds1.nacos.dataId=${spring.application.name}-flow-rules
```

FlowRuleNacosPublisher.java如下，无需修改：

```

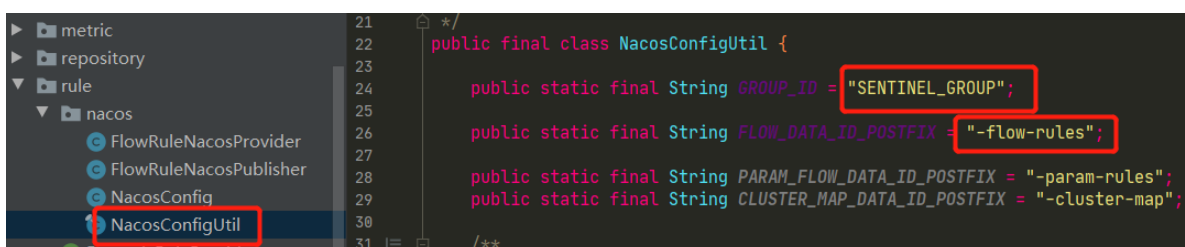
@Component("flowRuleNacosPublisher")
public class FlowRuleNacosPublisher implements DynamicRulePublisher<List<FlowRuleEntity>> {

    @Autowired
    private ConfigService configService;
    @Autowired
    private Converter<List<FlowRuleEntity>, String> converter;

    @Override
    public void publish(String app, List<FlowRuleEntity> rules) throws Exception {
        AssertUtil.notEmpty(app, "app name cannot be empty");
        if (rules == null) {
            return;
        }
        configService.publishConfig(app + NacosConfigUtil.FLOW_DATA_ID_POSTFIX,
            NacosConfigUtil.GROUP_ID, converter.convert(rules));
    }
}

```

打开 **NacosConfigUtil.java**，如下两个地方，需要和上面使用nacos存储时的配置一致。注意：两边的DataId和GroupId必须对应上。



```

21  */
22  public final class NacosConfigUtil {
23
24      public static final String GROUP_ID = "SENTINEL_GROUP";
25
26      public static final String FLOW_DATA_ID_POSTFIX = "-flow-rules";
27
28      public static final String PARAM_FLOW_DATA_ID_POSTFIX = "-param-rules";
29      public static final String CLUSTER_MAP_DATA_ID_POSTFIX = "-cluster-map";
30
31  /**

```

打开 **NacosConfig.java**，修改如下，主要是nacos配置中心的地址与namespace隔离环境的配置修改，如果没有设置namespace，就可以不设置 PropertyKeyConst.NAMESPACE。

```

@Configuration
public class NacosConfig {

    @Bean
    public Converter<List<FlowRuleEntity>, String> flowRuleEntityEncoder() {
        return JSON::toJSONString;
    }

    @Bean
    public Converter<String, List<FlowRuleEntity>> flowRuleEntityDecoder() {
        return s -> JSON.parseArray(s, FlowRuleEntity.class);
    }

    @Bean
    public ConfigService nacosConfigService() throws Exception {
        Properties properties = new Properties();
        //nacos配置中心地址
        properties.put(PropertyKeyConst.SERVER_ADDR, "192.168.100.80:8848");
        // nacos配置中心的namespace
        properties.put(PropertyKeyConst.NAMESPACE, "e95689b2-72d7-457e-b99b-231c1390be53");//dev
        properties.put(PropertyKeyConst.NAMESPACE, "a5a8124b-0588-457e-a54d-b533c1355cc7");//local
        return ConfigFactory.createConfigService(properties);
    }
}

```

经过以上步骤就已经把流控规则改造成**推模式**持久化了。

2.5 编译生成jar包

执行命令

```
mvn clean package -DskipTests
```

编译成功后，在项目的 target 目录可以找到sentinel-dashboard.jar，执行以下命令可以启动控制台：

```
java -jar sentinel-dashboard.jar
```

打开Sentinel控制台，可以看到上面通过nacos新建的限流规则



我们可以尝试在Sentinel控制台修改该规则，看是否能同步推送到Nacos，这里我们修改阈值为15，打开Nacos配置中心，可以看到已经更新过来了。

编辑配置

* Data ID: service-order-flow-rules.json

* Group: greenchoice-b2b

更多高级选项

描述:

Beta发布: ☐ 默认不要勾选。

配置格式: ☐ TEXT ☐ JSON ☐ XML ☐ YAML ☐ HTML ☐ Properties

配置内容 ? :

```
1 [{"app": "service-order", "clusterConfig": {"fallbackToLocalWhenFail": true, "sampleCount": 10, "strategy": 0, "thresholdType": 0, "windowIntervalMs": 1000}, "clusterMode": false, "controlBehavior": 0, "count": 15.0, "gmtCreate": 1596695021020, "gmtModified": 1596783916338, "grade": 1, "id": 3, "ip": "192.168.239.1", "limitApp": "default", "port": 8720, "resource": "/test", "strategy": 0}]
```

下面我们通过修改Nacos将阈值再修改为20，刷新Sentinel,也能同步过来，如下：

资源名	来源应用	流控模式	阈值类型	阈值	阈值模式	流控效果	操作
/test	default	直接	QPS	20	单机	快速失败	<button>编辑</button> <button>删除</button>

共 1 条记录, 每页 10 条记录

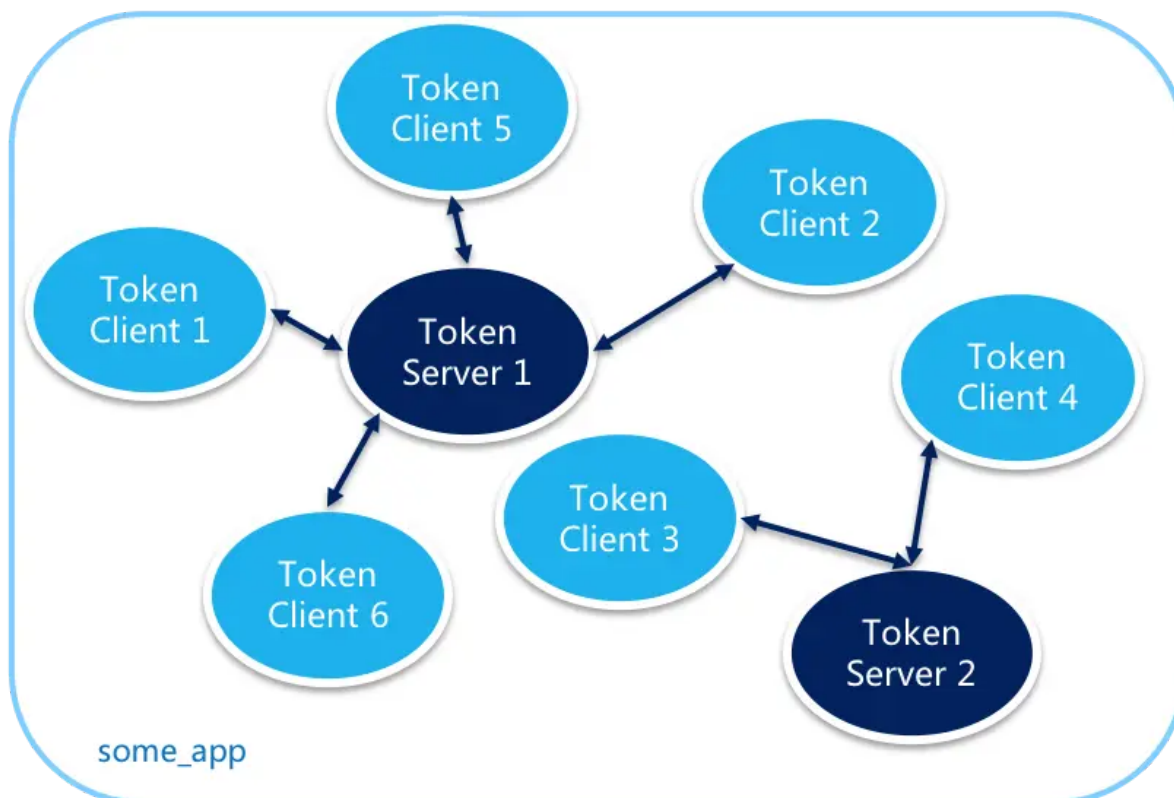
通过测试发现，在Sentinel控制台修改规则可以同步到Nacos，或者在Nacos上修改规则也可以同步到Sentinel控制台。

13 Sentinel+nacos实现集群限流

使用集群的方式设置限流规则

Sentinel的集群限流也没那么神秘，核心设计就是采用一个中心化的Token Server来分配令牌来实行，每次请求都会通过实例的Token Client请求Token Server获取token并调用。

如下图所示：



注意这里Client和Server是通过tcp长连接的方式通信的，需要有reconnect的机制，比如这里Client连接不上Server，会等待 $n * 2000ms$ 的方式一直尝试连接。

Sentinel 集群限流服务端有两种启动方式：

- 嵌入模式（Embedded）适合应用级别的限流，部署简单，但对应用性能有影响；
- 独立模式（Alone）适合全局限流，需要独立部署。

集群限流使用场景

场景一、qps量小比机器数还少

假设我们希望给某个用户限制调用某个 API 的总 QPS 为 50，但机器数可能很多（比如有 100 台）。这时候我们很自然地就想到，找一个 server 来专门来统计总的调用量，其它的实例都与这台 server 通信来判断是否可以调用。

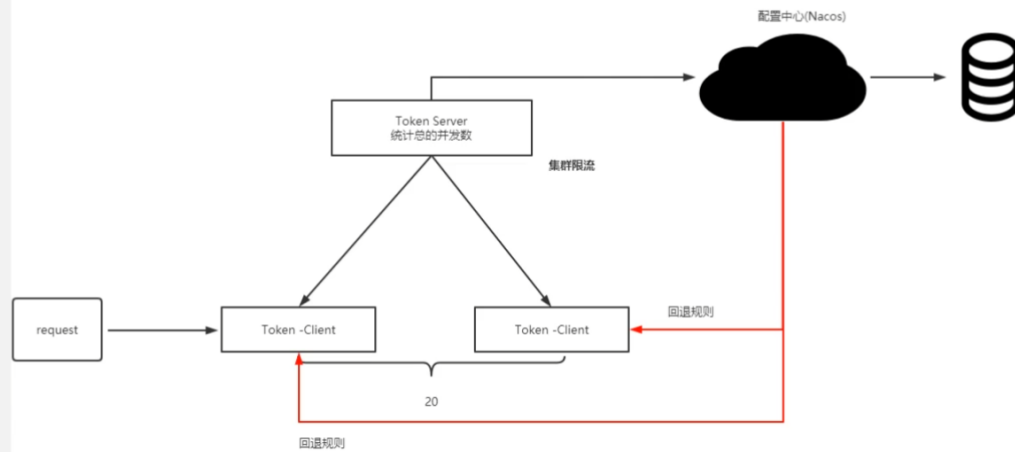
场景二、机器弹性伸缩、数目变化频繁

假设一个服务访问量呈锯齿状，开启了弹性伸缩，机器数目不透明，那么这时候通过预估单实例qps * 机器数的方式计算就会不准确。

场景三、机器配置不一样，不同实例需要根据整体qps和机器水位综合限流

这种场景比较特殊，一般业务上不会遇到，不做展开讨论。

集群架构示意图



CSDN @_坤

建立 sentinel-token-sever

project sentinel-token-sever

3、添加依赖

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.chaoxing.sentinel</groupId>
  <artifactId>sentinel-token-sever</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>sentinel-token-sever</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.7</maven.compiler.source>
    <maven.compiler.target>1.7</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>com.alibaba.csp</groupId>
      <artifactId>sentinel-cluster-server-default</artifactId>
      <version>1.8.0</version>
    </dependency>
    <dependency>
      <groupId>com.alibaba.csp</groupId>
```

```

    <artifactId>sentinel-datasource-nacos</artifactId>
    <version>1.8.0</version>
  </dependency>
  <dependency>
    <groupId>com.alibaba.csp</groupId>
    <artifactId>sentinel-transport-simple-http</artifactId>
    <version>1.8.0</version>
  </dependency>***
</dependencies>

<build>
  <pluginManagement><!-- lock down plugins versions to avoid using Maven
defaults (may be moved to parent pom) -->
    <plugins>
      <!-- clean lifecycle, see https://maven.apache.org/ref/current/maven-
core/lifecycles.html#clean_Lifecycle -->
      <plugin>
        <artifactId>maven-clean-plugin</artifactId>
        <version>3.1.0</version>
      </plugin>
      <!-- default lifecycle, jar packaging: see
https://maven.apache.org/ref/current/maven-core/default-
bindings.html#Plugin_bindings_for_jar_packaging -->
      <plugin>
        <artifactId>maven-resources-plugin</artifactId>
        <version>3.0.2</version>
      </plugin>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.0</version>
      </plugin>
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.22.1</version>
      </plugin>
      <plugin>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.0.2</version>
      </plugin>
      <plugin>
        <artifactId>maven-install-plugin</artifactId>
        <version>2.5.2</version>
      </plugin>
      <plugin>
        <artifactId>maven-deploy-plugin</artifactId>
        <version>2.8.2</version>
      </plugin>
      <!-- site lifecycle, see https://maven.apache.org/ref/current/maven-
core/lifecycles.html#site_Lifecycle -->
      <plugin>
        <artifactId>maven-site-plugin</artifactId>
        <version>3.7.1</version>
      </plugin>
    </plugins>
  </pluginManagement>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>

```

```

        <artifactId>maven-compiler-plugin</artifactId>
        <configuration>
            <source>8</source>
            <target>8</target>
        </configuration>
    </plugin>
</plugins>
</build>
</project>

```

4、建立服务类ClusterServer

```

package com.chaoxing.sentinel;

import com.alibaba.csp.sentinel.cluster.client.ClusterTokenClient;
import com.alibaba.csp.sentinel.cluster.server.ClusterTokenServer;
import com.alibaba.csp.sentinel.cluster.server.SentinelDefaultTokenServer;
import com.alibaba.csp.sentinel.cluster.server.config.ClusterServerConfigManager;
import com.alibaba.csp.sentinel.cluster.server.config.ServerTransportConfig;

import java.util.Collections;

/**
 * @Author :Kun
 * @Date 2022/6/27 16:38
 */
public class ClusterServer {
    public static void main(String[] args) throws Exception {
        ClusterTokenServer tokenServer = new SentinelDefaultTokenServer();
        //手动载入namespace和ClusterServerConfigManager
        //集群限制流服务端通信相关配置
        ClusterServerConfigManager.loadGlobalTransportConfig(new
        ServerTransportConfig().setIdleSeconds(600).setPort(9999));
        //加载namespace集合列表

        ClusterServerConfigManager.loadServerNamespacesSet(Collections.singleton("APP-
        Test"));
        tokenServer.start();
        //Token-client会上报自己的project.name到token-server会根据namespace来统一
    }
}

```

5、建立获取配置类FlowRuleInitFunc

```

package com.chaoxing.sentinel;

import com.alibaba.csp.sentinel.cluster.flow.rule.ClusterFlowRuleManager;
import com.alibaba.csp.sentinel.datasource.ReadableDataSource;
import com.alibaba.csp.sentinel.datasource.nacos.NacosDataSource;
import com.alibaba.csp.sentinel.init.InitFunc;
import com.alibaba.csp.sentinel.slots.block.flow.ClusterFlowConfig;

```

```

import com.alibaba.csp.sentinel.slots.block.flow.FlowRule;
import com.alibaba.csp.sentinel.slots.block.flow.FlowRuleManager;
import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.TypeReference;

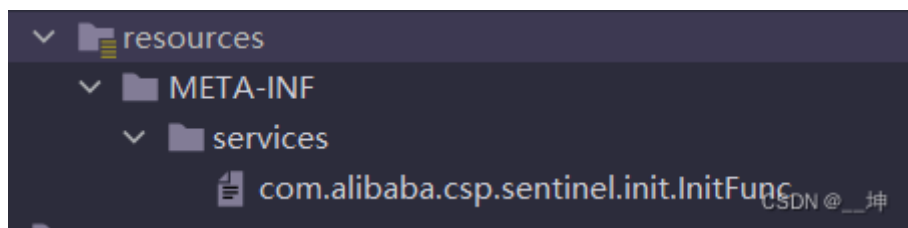
import java.util.List;

/**
 * @Author :Kun
 * @Date 2022/6/27 15:37
 */
public class FlowRuleInitFunc implements InitFunc {

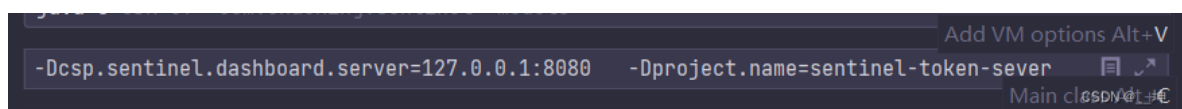
    private final String nacosAddress = "127.0.0.1:8848";
    private final String groupId = "SENTINEL_GROUP";
    private final String dataId = "-flow-rules";
    @Override
    /**
     * 定义限流规则,初始化规则方便使用
     */
    public void init() throws Exception {
        ClusterFlowRuleManager.setPropertySupplier(namespace->{
            ReadableDataSource<String,List<FlowRule>> flowRuleDs =
                new NacosDataSource<List<FlowRule>>
(nacosAddress,groupId,namespace+dataId,
                    s -> JSON.parseObject(s,new
TypeReference<List<FlowRule>>(){}));
            return flowRuleDs.getProperty();
        });
    }
}

```

6、添加从nacos获取配置的配置文件



7、添加VM 参数



```

-Dcsp.sentinel.dashboard.server=127.0.0.1:8080 -Dproject.name=sentinel-token-sever

```

8、在nacos中配置

配置详情

* Data ID: App-Test-flow-rules

* Group: SENTINEL_GROUP

[更多高级选项](#)

描述:

* MD5: 3960bbda6eac06ea3cb9853376ed9094

* 配置内容:

```
1 [ {
2   "resource": "resource_Name",
3   "grade": 1,
4   "count": 5,
5   "clusterMode": true,
6   "clusterConfig": {
7     "flowId": "100001",
8     "thresholdType": 1,
9     "fallbackToLocalWhenFail": true
10  }
11 }
```

9、运行

应用名 搜索

🏠 首页

SpringBoot-sentinel (1/1) ✓

sentinel-dashboard (1/1) ✓

sentinel-token-sever (1/1) ✓

进行客户端的限流参数上报

修改FlowRuleInitFunc方法

```
package com.chaoxing.springboot_sentinel;

import com.alibaba.csp.sentinel.cluster.client.config.ClusterClientAssignConfig;
import com.alibaba.csp.sentinel.cluster.client.config.ClusterClientConfig;
import com.alibaba.csp.sentinel.cluster.client.config.ClusterClientConfigManager;
import com.alibaba.csp.sentinel.datasource.ReadableDataSource;
import com.alibaba.csp.sentinel.datasource.nacos.NacosDataSource;
import com.alibaba.csp.sentinel.init.InitFunc;
import com.alibaba.csp.sentinel.slots.block.RuleConstant;
import com.alibaba.csp.sentinel.slots.block.flow.FlowRule;
import com.alibaba.csp.sentinel.slots.block.flow.FlowRuleManager;
import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.TypeReference;
```

```

import java.util.ArrayList;
import java.util.List;

/**
 * @Author :Kun
 * @Date 2022/6/27 15:37
 */
public class FlowRuleInitFunc implements InitFunc {

    private final String nacosAddress = "127.0.0.1:8848";
    private final String groupId = "SENTINEL_GROUP";
    private final String dataId = "-flow-rules";
    private final String clusterServerHost="localhost";
    private final int clusterServerPort=9999;
    private final int requestTimeOut=20000;
    private final String appname = "App-Test";
    @Override
    /**
     *定义限流规则,初始化规则方便使用
     */
    public void init() throws Exception {
        loadClusterConfig();
        registerFlowRule();
    }
    private void loadClusterConfig(){
        ClusterClientAssignConfig assignConfig = new ClusterClientAssignConfig();
        assignConfig.setServerHost(clusterServerHost);//放到配置中心
        assignConfig.setServerPort(clusterServerPort);
        ClusterClientConfigManager.applyNewAssignConfig(assignConfig);
        ClusterClientConfig clientConfig = new ClusterClientConfig();
        clientConfig.setRequestTimeout(requestTimeOut);//放到配置中心
        ClusterClientConfigManager.applyNewConfig(clientConfig);
    }
    private void registerFlowRule(){
        ReadableDataSource<String,List<FlowRule>> flowRuleDs =
            new NacosDataSource<List<FlowRule>>
(nacosAddress,groupId,appname+dataId,
                s -> JSON.parseObject(s,new TypeReference<List<FlowRule>>
(){}));
        FlowRuleManager.register2Property(flowRuleDs.getProperty());
    }
}

```

11、设置主方法为客户端

```

package com.chaoxing.springboot_sentinel;

import com.alibaba.csp.sentinel.cluster.ClusterStateManager;
import com.alibaba.csp.sentinel.slots.block.RuleConstant;
import com.alibaba.csp.sentinel.slots.block.flow.FlowRule;
import com.alibaba.csp.sentinel.slots.block.flow.FlowRuleManager;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

```

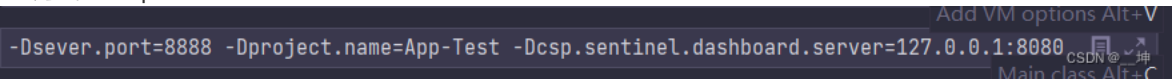
```
import java.util.ArrayList;
import java.util.List;

@SpringBootApplication
public class SpringBootSentinelApplication {

    public static void main(String[] args) {
        ClusterStateManager.applyState(ClusterStateManager.CLUSTER_CLIENT);
        SpringApplication.run(SpringBootSentinelApplication.class, args);
    }

}
```

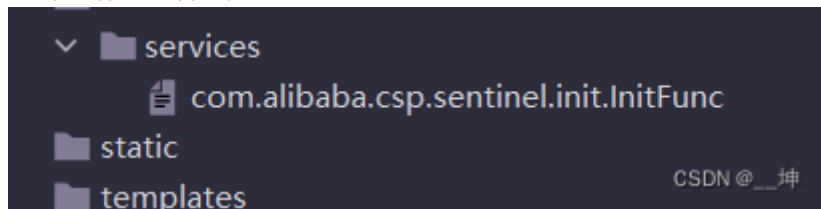
12 调试vm options



```
-Dsever.port=8888 -Dproject.name=App-Test -Dcsp.sentinel.dashboard.server=127.0.0.1:8080
```

```
-Dsever.port=8888 -Dproject.name=App-Test -
Dcsp.sentinel.dashboard.server=127.0.0.1:8080
```

13 确定配置文件可用



```
services
├── com.alibaba.csp.sentinel.init.InitFunc
├── static
└── templates
```

14 运行测试

The first screenshot shows the Sentinel 1.8.4 console with the 'sentinel-token-sever' page. The left sidebar has 'sentinel-token-sever' selected. The main area shows a table of Token Servers with one entry: 192.168.56.1@8721, Port 9999, App-Test, Standalone mode, 1 total connection, 0 QPS. The '连接详情' (Connection Details) button is highlighted.

The second screenshot shows the same console with the '集群流控' (Cluster Flow Control) option selected in the left sidebar. The '连接详情' button is still highlighted.

The third screenshot shows the '连接详情' dialog box. It displays the Token Server 192.168.56.1@8721 and a table of connections:

命名空间	连接数	连接详情
default	0	[]
App-Test	1	[127.0.0.1:50155]

连接成功

未完待续.....

作者介绍:

本文1作：德鲁，资深架构师，《Java 高并发核心编程 加强版》作者之1。

本文2作：尼恩，40岁资深老架构师，《Java 高并发核心编程 加强版 卷1、卷2、卷3》创世作者，著名博主。《K8S学习圣经》《Docker学习圣经》等11个PDF圣经的作者。

技术自由的实现路径 PDF:

实现你的 架构自由：

《[吃透8图1模板，人人可以做架构](#)》

《[10Wqps评论中台，如何架构？B站是这么做的!!!](#)》

《[阿里二面：千万级、亿级数据，如何性能优化？教科书级 答案来了](#)》

《[峰值21WQps、亿级DAU，小游戏《羊了个羊》是怎么架构的？](#)》

《[100亿级订单怎么调度，来一个大厂的极品方案](#)》

《[2个大厂 100亿级 超大流量 红包 架构方案](#)》

... 更多架构文章，正在添加中

实现你的 响应式 自由：

《[响应式圣经：10W字，实现Spring响应式编程自由](#)》

这是老版本 《[Flux、Mono、Reactor 实战（史上最全）](#)》

实现你的 spring cloud 自由：

《[Spring cloud Alibaba 学习圣经](#)》

《[分库分表 Sharding-JDBC 底层原理、核心实战（史上最全）](#)》

《[一文搞定：SpringBoot、SLF4j、Log4j、Logback、Netty之间混乱关系（史上最全）](#)》

实现你的 linux 自由：

《[Linux命令大全：2W多字，一次实现Linux自由](#)》

实现你的 网络 自由：

《[TCP协议详解\(史上最全\)](#)》

《[网络三张表：ARP表, MAC表, 路由表，实现你的网络自由!!](#)》

实现你的 分布式锁 自由：

《[Redis分布式锁（图解 - 秒懂 - 史上最全）](#)》

《[Zookeeper 分布式锁 - 图解 - 秒懂](#)》

实现你的 王者组件 自由：

《[队列之王：Disruptor 原理、架构、源码 一文穿透](#)》

《[缓存之王：Caffeine 源码、架构、原理（史上最全，10W字 超级长文）](#)》

《[缓存之王：Caffeine 的使用（史上最全）](#)》

《[Java Agent 探针、字节码增强 ByteBuddy \(史上最全\)](#)》

实现你的 面试题 自由：

[4000页《尼恩Java面试宝典》40个专题](#)

以上尼恩 架构笔记、面试题 的PDF文件更新，请到《技术自由圈》公号获取↓↓↓

免费获取11个技术圣经 PDF：

技术自由圈 技术圣经系列

技术自由圈 7个 学习圣经 PDF

-  **1 《SpringCloud Alibaba 学习圣经》** 377页 PDF
v1 版，发布日期：2023年03月15日
- 2 《Docker 学习圣经》** 145页 PDF
v2 版，发布日期：2023年03月12日
-  **3 《Kubernetes 学习圣经》** 530页 PDF
v1 版，发布日期：2023年04月11日
-  **4 《响应式圣经》** 114页 PDF
v2 版，发布日期：2023年02月14日
- 5 《缓存之王 Caffeine 红宝书》** 175页 PDF
v2 版，发布日期：2022年11月30日
- 6 《队列之王 Disruptor 红宝书》** 75页 PDF
v1 版，发布日期：2022年10月05日
- 7 《Prometheus+grafna 红宝书》** 100页 PDF
v1 版，发布日期：2022年10月08日

技术自由圈 3个 高并发圣经 PDF

-  **8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》** 609页 PDF
加强版，发布日期：2023年01月01日
-  **9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》** 448页 PDF

加强版，发布日期：2022年11月14日

10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》

481页 PDF

加强版，发布日期：2022年11月14日

技术自由圈 面试圣经 40个面试题 PDF

11 《尼恩 Java 面试宝典》40个专题

4000页 PDF

v60 版，发布日期：2023年03月11日

领取方式： 技术自由圈 公众号

CSDN @40岁资深老架构师尼恩